

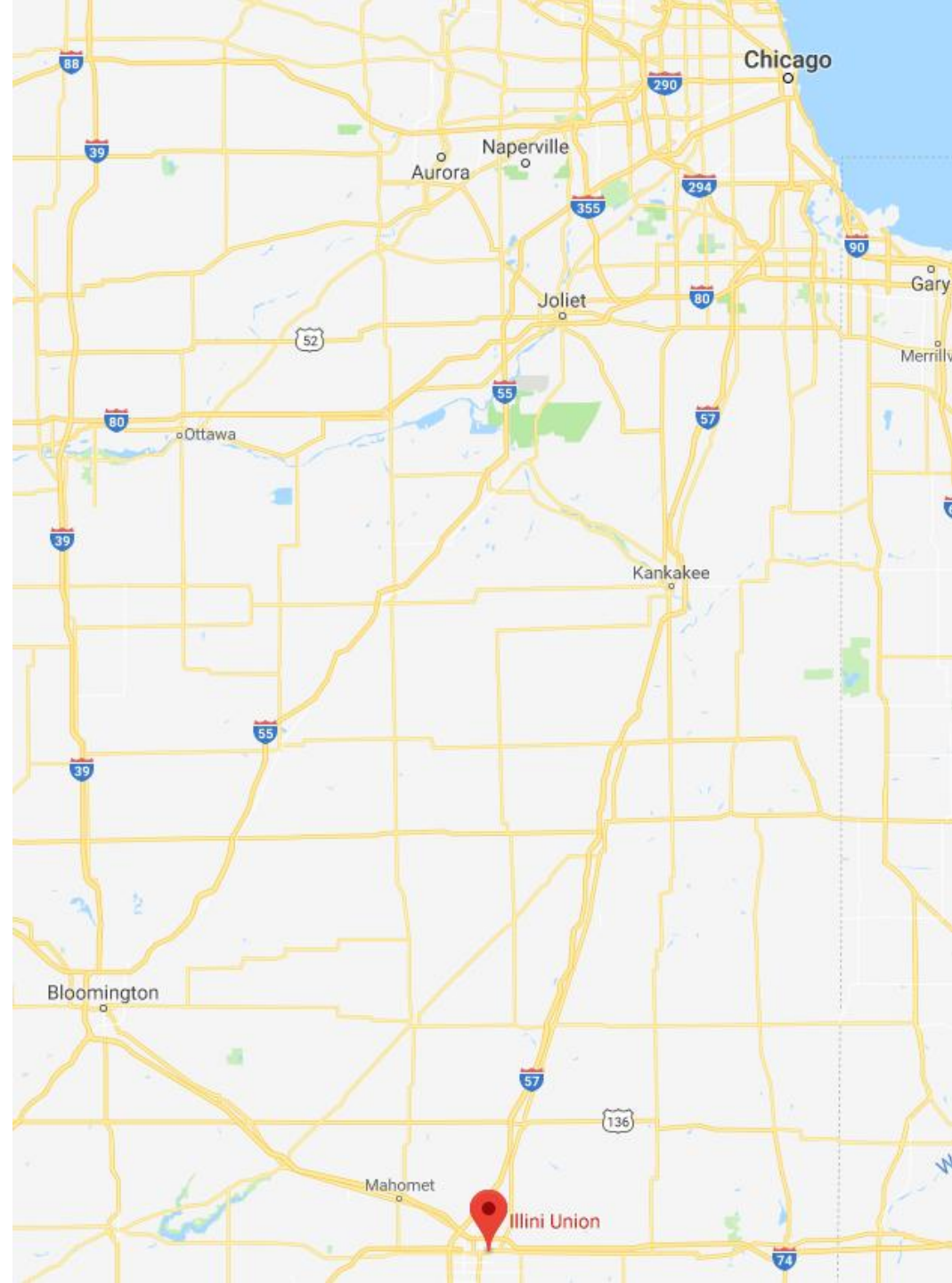
CS 225

Data Structures

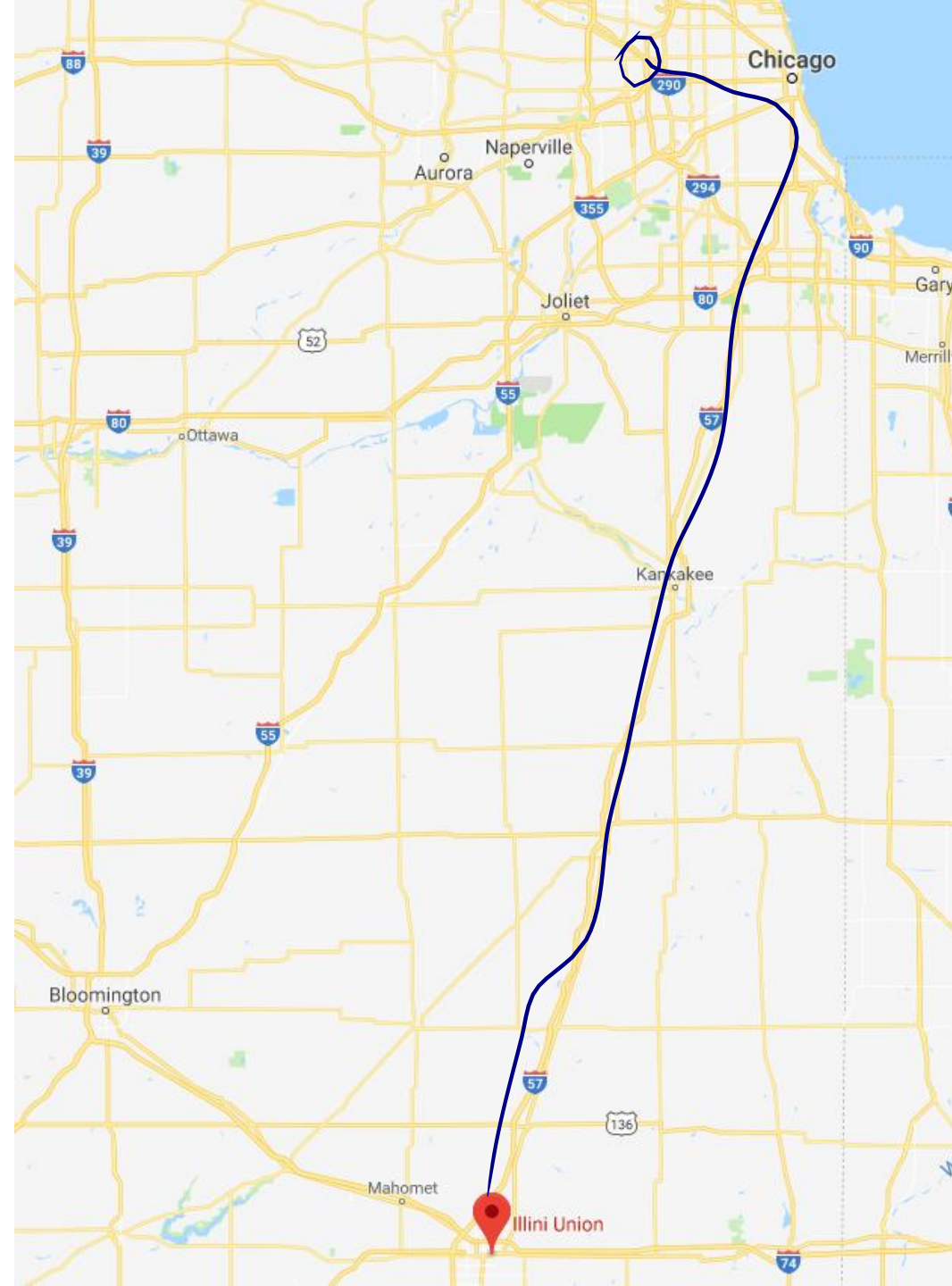
Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

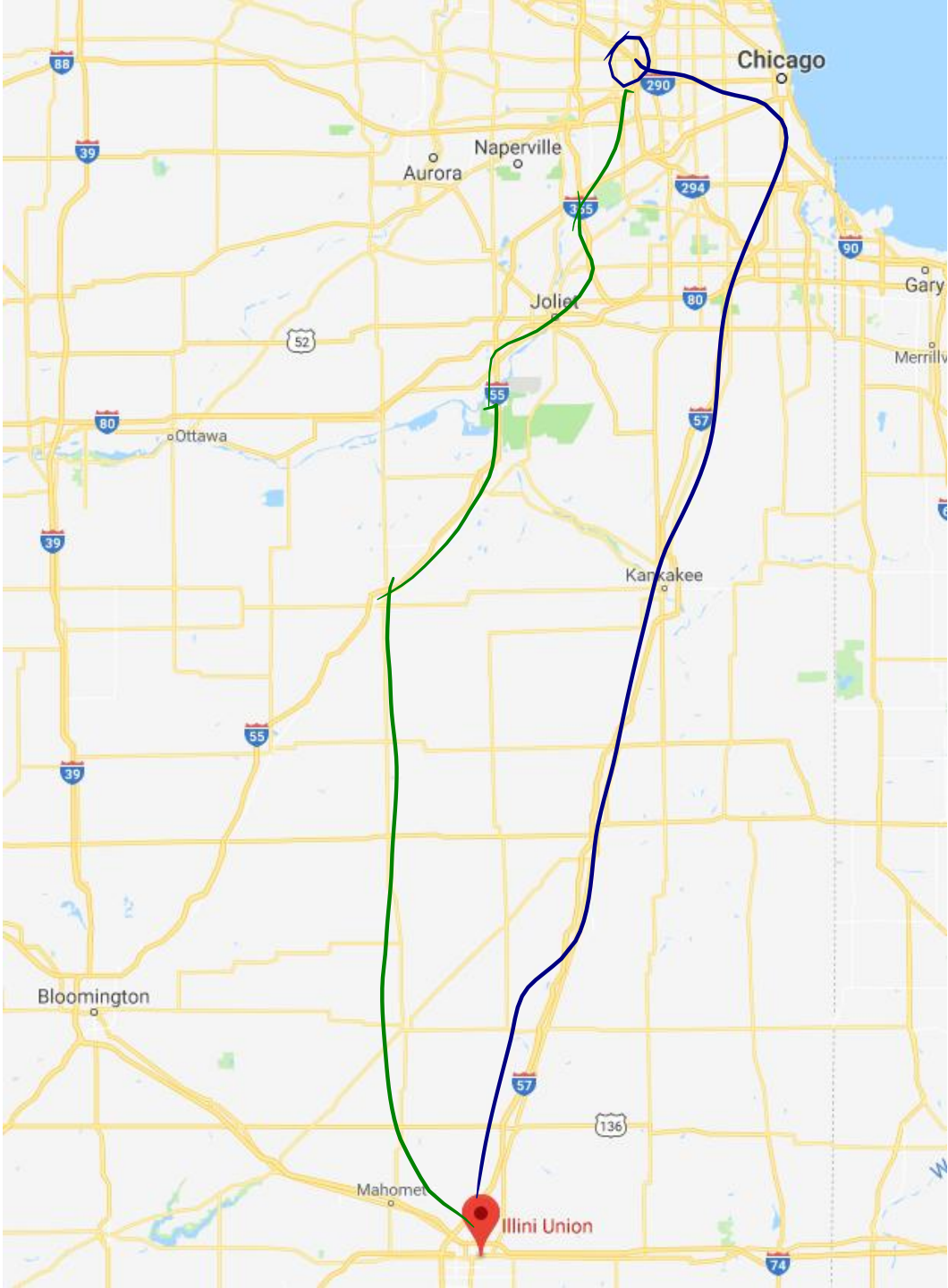
Shortest Path



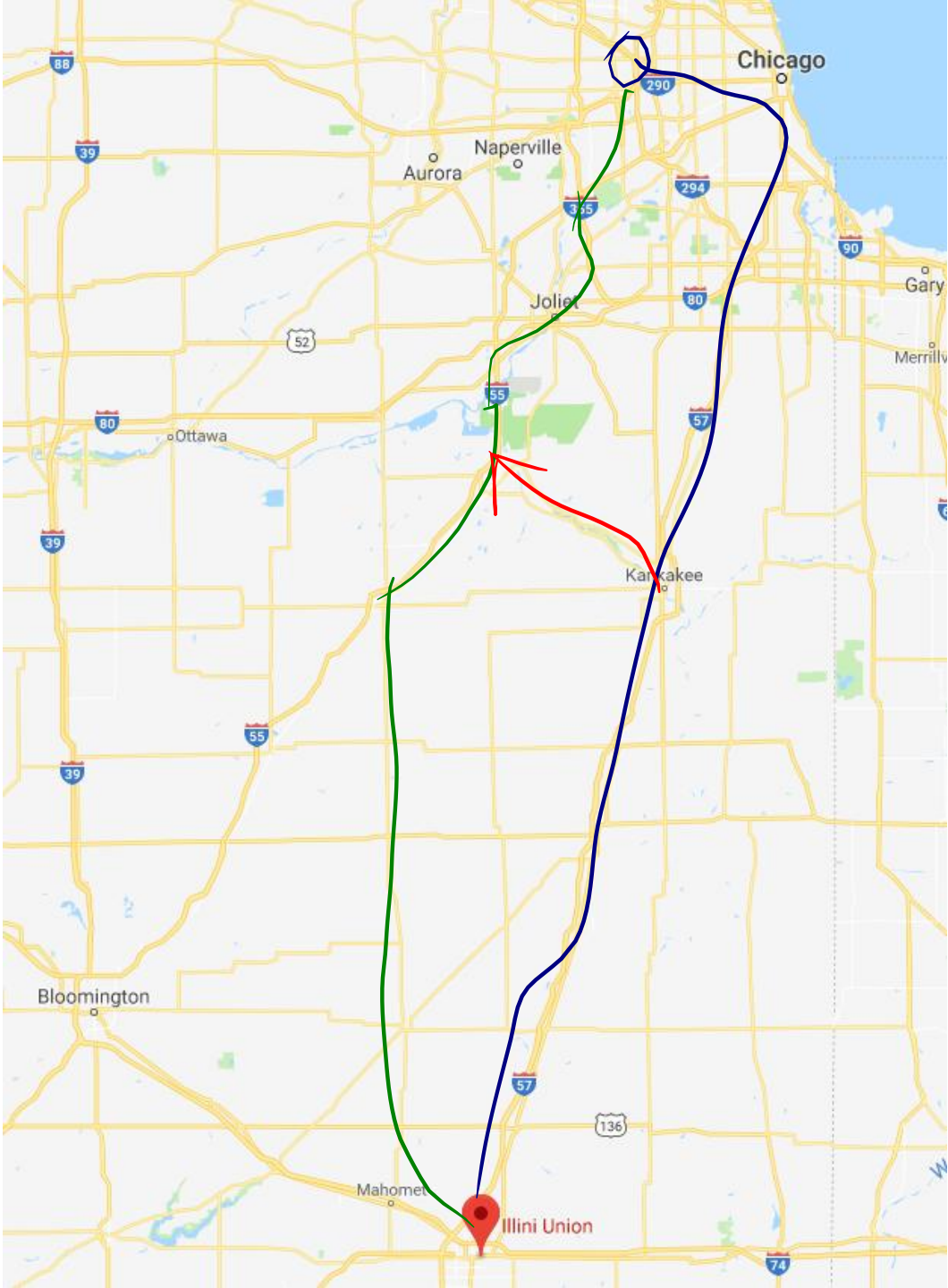
Shortest Path



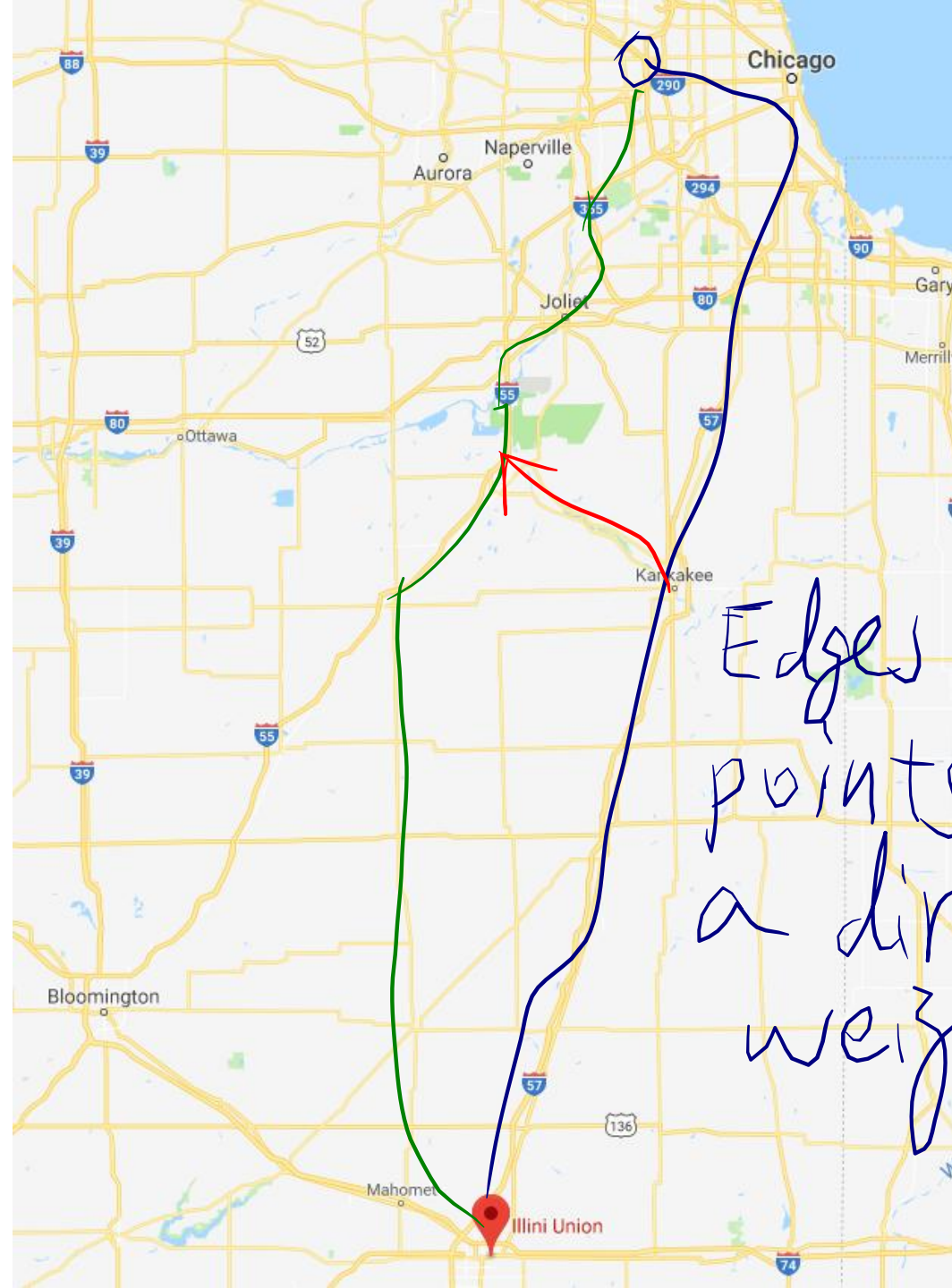
Shortest Path



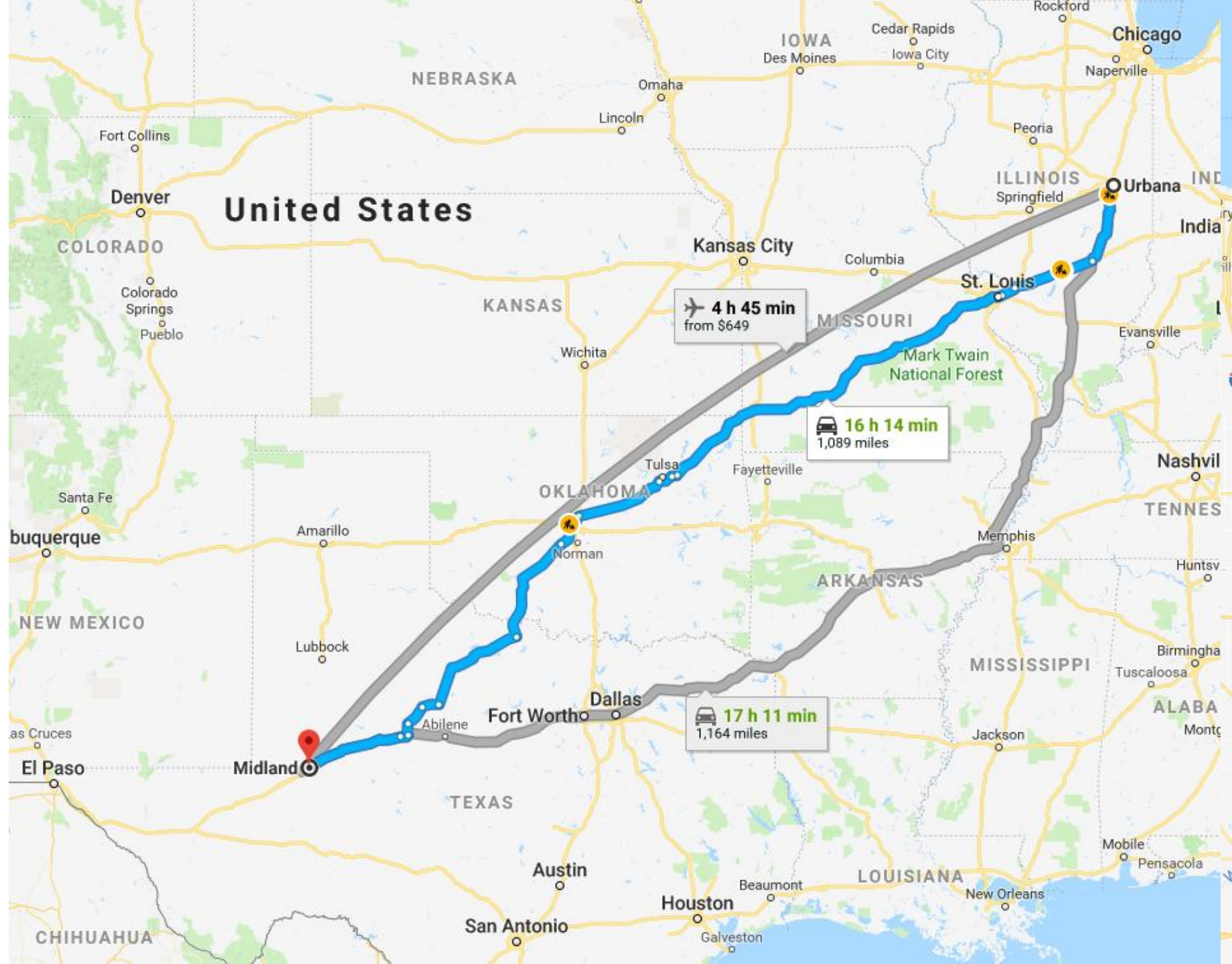
Shortest Path



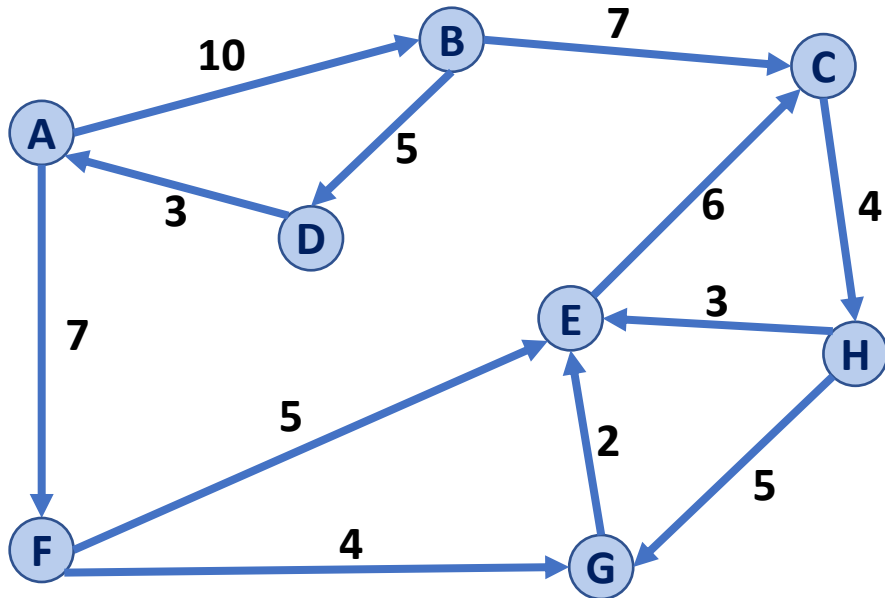
Shortest Path



Edges B/T
points have
a directed
weight



Dijkstra's Algorithm (SSSP)



```
DijkstraSSSP(G, s):
```

```
6  foreach (Vertex v : G):
```

```
7      d[v] = +inf
```

```
8      p[v] = NULL
```

```
9  d[s] = 0
```

```
10
```

```
11  PriorityQueue Q // min distance, defined by d[v]
```

```
12  Q.buildHeap(G.vertices())
```

```
13  Graph T          // "labeled set"
```

```
14
```

```
15  repeat n times:
```

```
16      Vertex u = Q.removeMin()
```

```
17      T.add(u)
```

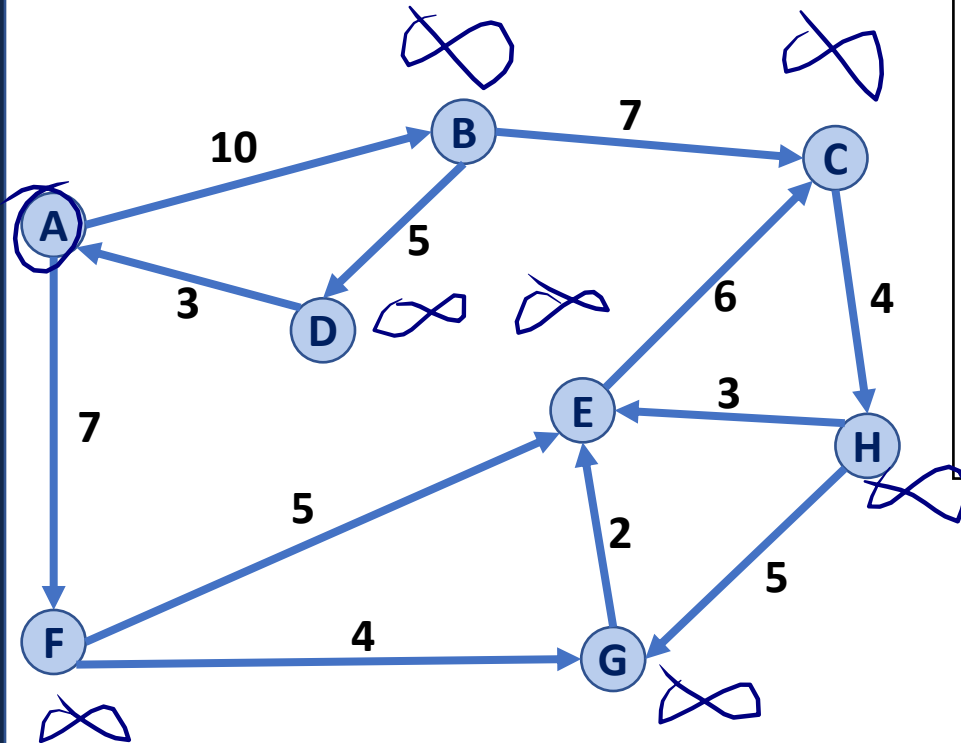
```
18      foreach (Vertex v : neighbors of u not in T):
```

```
19          if _____ < d[v]:
```

```
20              d[v] = _____
```

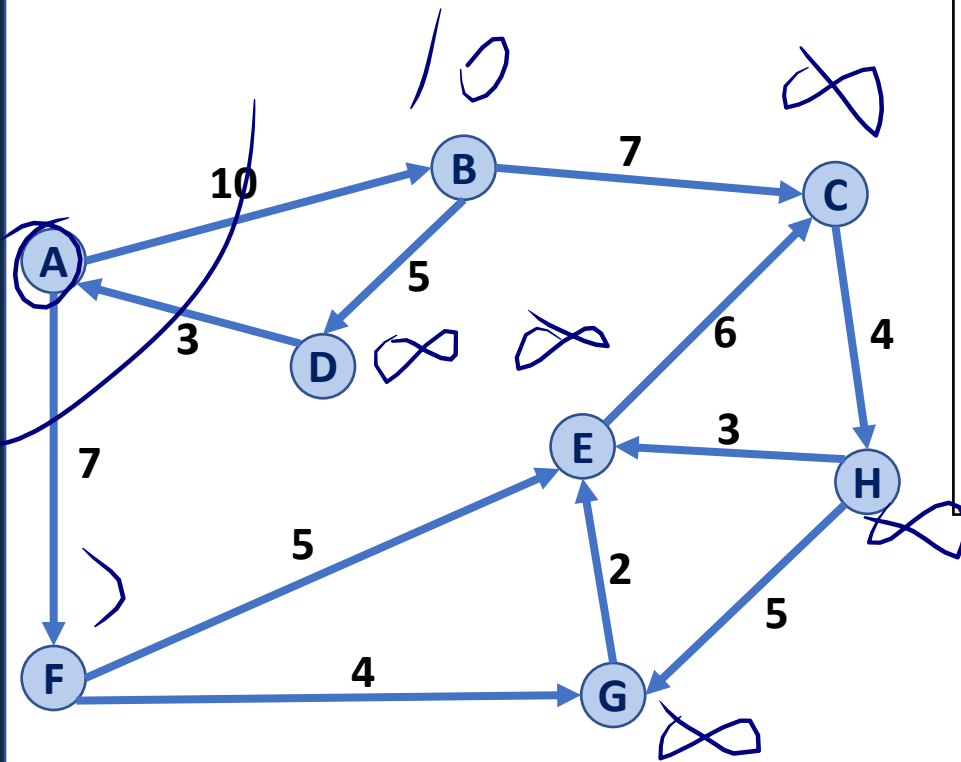
```
21              p[v] = m
```

Dijkstra's Algorithm (SSSP)



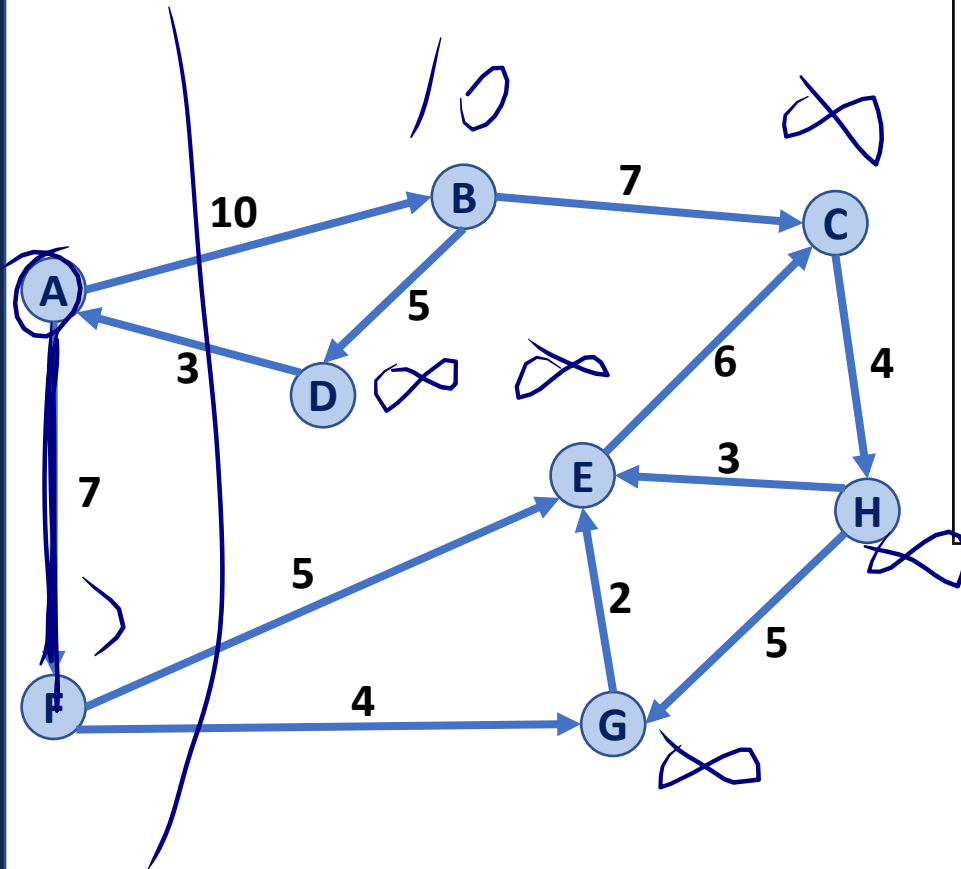
```
DijkstraSSSP(G, s):  
6  foreach (Vertex v : G):  
7      d[v] = +inf  
8      p[v] = NULL  
9  d[s] = 0  
10  
11  PriorityQueue Q // min distance, defined by d[v]  
12  Q.buildHeap(G.vertices())  
13  Graph T          // "labeled set"  
14  
15  repeat n times:  
16      Vertex u = Q.removeMin()  
17      T.add(u)  
18      foreach (Vertex v : neighbors of u not in T):  
19          if _____ < d[v]:  
20              d[v] = _____  
21              p[v] = m
```

Dijkstra's Algorithm (SSSP)



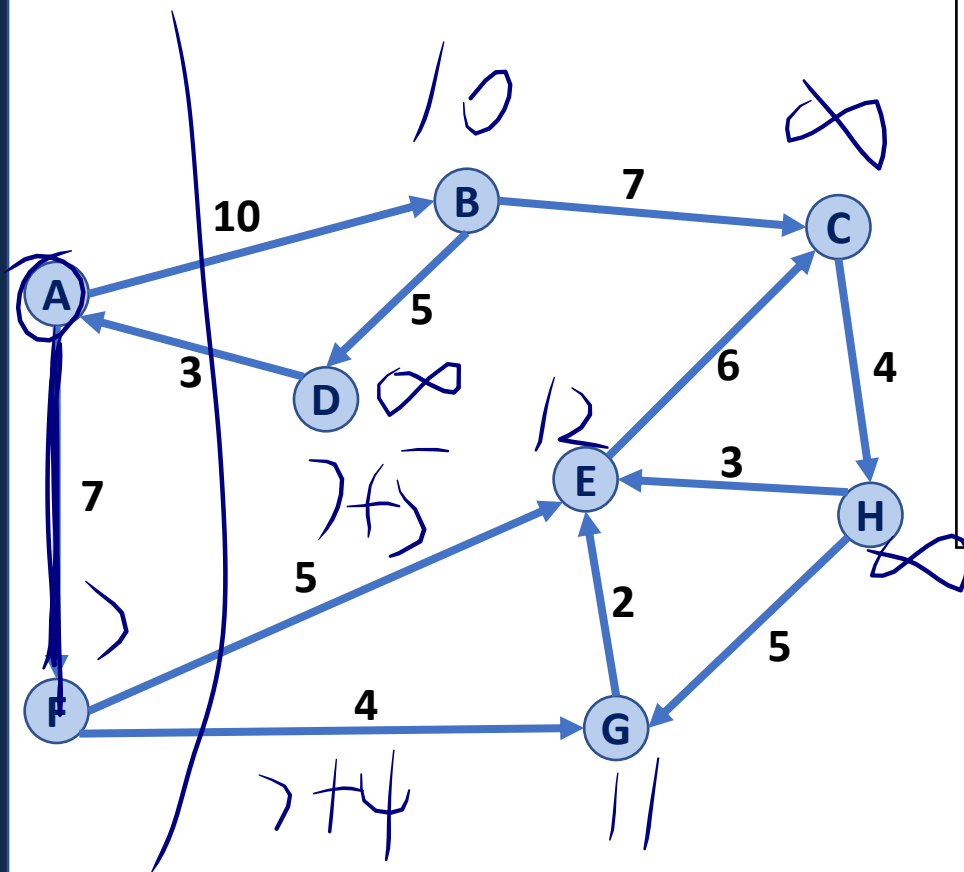
```
DijkstraSSSP(G, s):  
6  foreach (Vertex v : G):  
7      d[v] = +inf  
8      p[v] = NULL  
9  d[s] = 0  
10  
11  PriorityQueue Q // min distance, defined by d[v]  
12  Q.buildHeap(G.vertices())  
13  Graph T          // "labeled set"  
14  
15  repeat n times:  
16      Vertex u = Q.removeMin()  
17      T.add(u)  
18      foreach (Vertex v : neighbors of u not in T):  
19          if _____ < d[v]:  
20              d[v] = _____  
21              p[v] = m
```

Dijkstra's Algorithm (SSSP)



```
DijkstraSSSP(G, s):  
6  foreach (Vertex v : G):  
7      d[v] = +inf  
8      p[v] = NULL  
9  d[s] = 0  
10  
11  PriorityQueue Q // min distance, defined by d[v]  
12  Q.buildHeap(G.vertices())  
13  Graph T          // "labeled set"  
14  
15  repeat n times:  
16      Vertex u = Q.removeMin()  
17      T.add(u)  
18      foreach (Vertex v : neighbors of u not in T):  
19          if _____ < d[v]:  
20              d[v] = _____  
21              p[v] = m
```

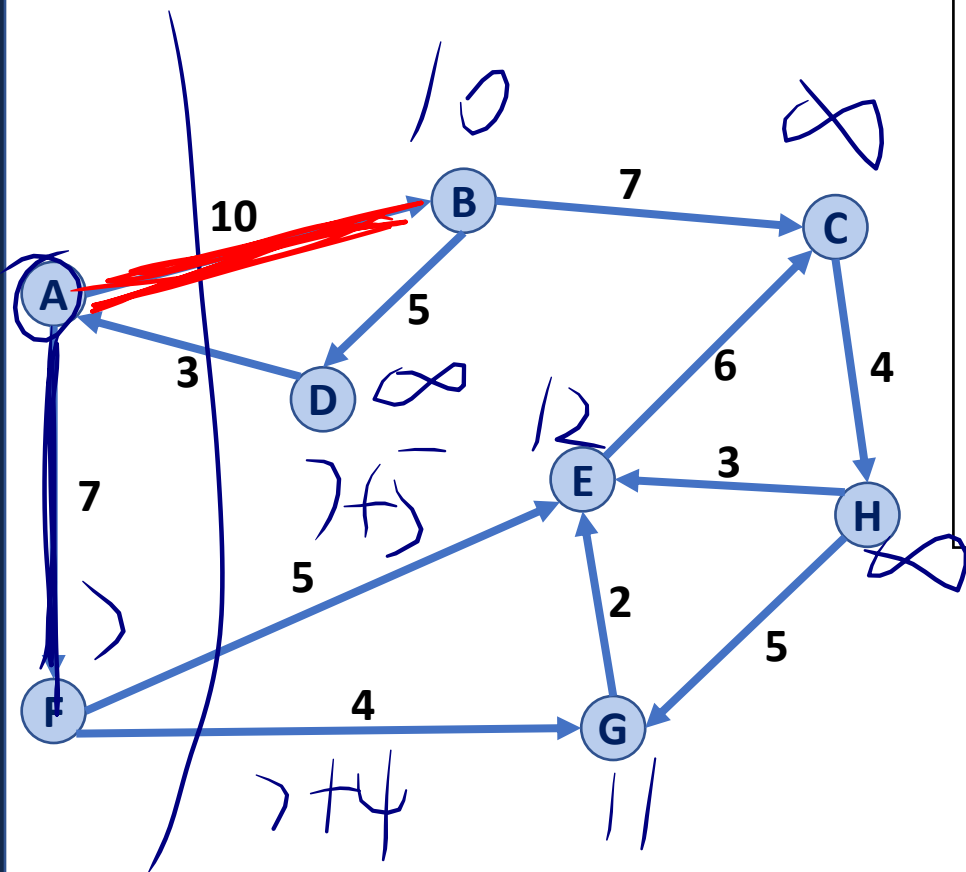
Dijkstra's Algorithm (SSSP)



```

DijkstraSSSP(G, s):
6   foreach (Vertex v : G):
7       d[v] = +inf
8       p[v] = NULL
9   d[s] = 0
10
11  PriorityQueue Q // min distance, defined by d[v]
12  Q.buildHeap(G.vertices())
13  Graph T          // "labeled set"
14
15  repeat n times:
16      Vertex u = Q.removeMin()
17      T.add(u)
18      foreach (Vertex v : neighbors of u not in T):
19          if  $\text{cost}(u, v) + d[u] < d[v]$ :
20              d[v] =  $\text{cost}(u, v) + d[u]$ 
21              p[v] = u
    
```

Dijkstra's Algorithm (SSSP)

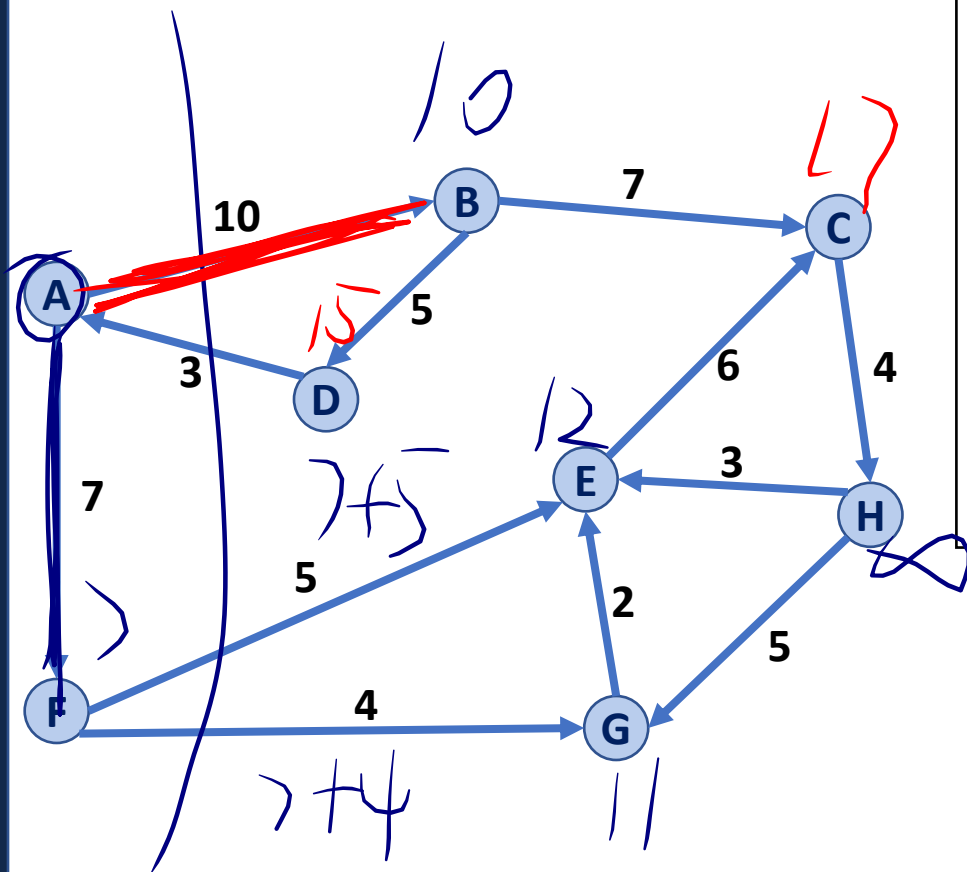


```

DijkstraSSSP(G, s):
6   foreach (Vertex v : G):
7       d[v] = +inf
8       p[v] = NULL
9   d[s] = 0
10
11  PriorityQueue Q // min distance, defined by d[v]
12  Q.buildHeap(G.vertices())
13  Graph T          // "labeled set"
14
15  repeat n times:
16      Vertex u = Q.removeMin()
17      T.add(u)
18      foreach (Vertex v : neighbors of u not in T):
19          if  $\text{cost}(u, v) + d[u] < d[v]$ :
20              d[v] =  $\text{cost}(u, v) + d[u]$ 
21              p[v] = u
    
```

$\text{cost}(u, v) + d[u]$

Dijkstra's Algorithm (SSSP)



DijkstraSSSP (G, s) :

```
6  foreach (Vertex v : G) :
```

```
7 |      d[v] = +inf
```

```
8 | p[v] = NULL
```

9	$d[s] = 0$
---	------------

10

```
11 | PriorityQueue Q // min distance, defined by d[v]
```

```
12 Q.buildHeap(G.vertices())
```

```
13 | Graph T // "labeled set"
```

14

```
repeat n times:
```

```
16 Vertex u = Q.removeMin()
```

17	T.add(u)
----	----------

```
18   foreach (Vertex v : neighbors of u not in T):
```

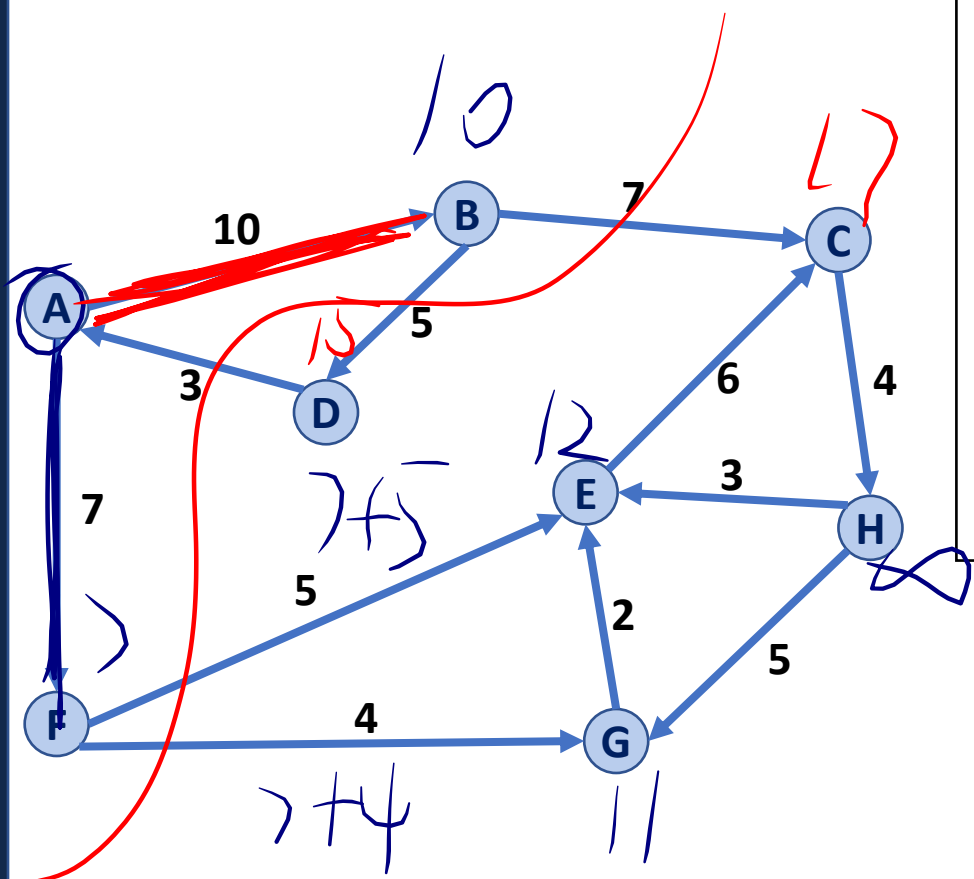
```
19 | if (cost[u, v] + d[u] < d[v] :
```

20 $d[\text{A}] =$

21 $p[v] = m$

$$\text{cost}(u, v) + d[u]$$

Dijkstra's Algorithm (SSSP)



```
DijkstraSSSP(G, s):
```

```
6  foreach (Vertex v : G):
```

```
7      d[v] = +inf
```

```
8      p[v] = NULL
```

```
9      d[s] = 0
```

```
10
```

```
11  PriorityQueue Q // min distance, defined by d[v]
```

```
12  Q.buildHeap(G.vertices())
```

```
13  Graph T          // "labeled set"
```

```
14
```

```
15  repeat n times:
```

```
16      Vertex u = Q.removeMin()
```

```
17      T.add(u)
```

```
18      foreach (Vertex v : neighbors of u not in T):
```

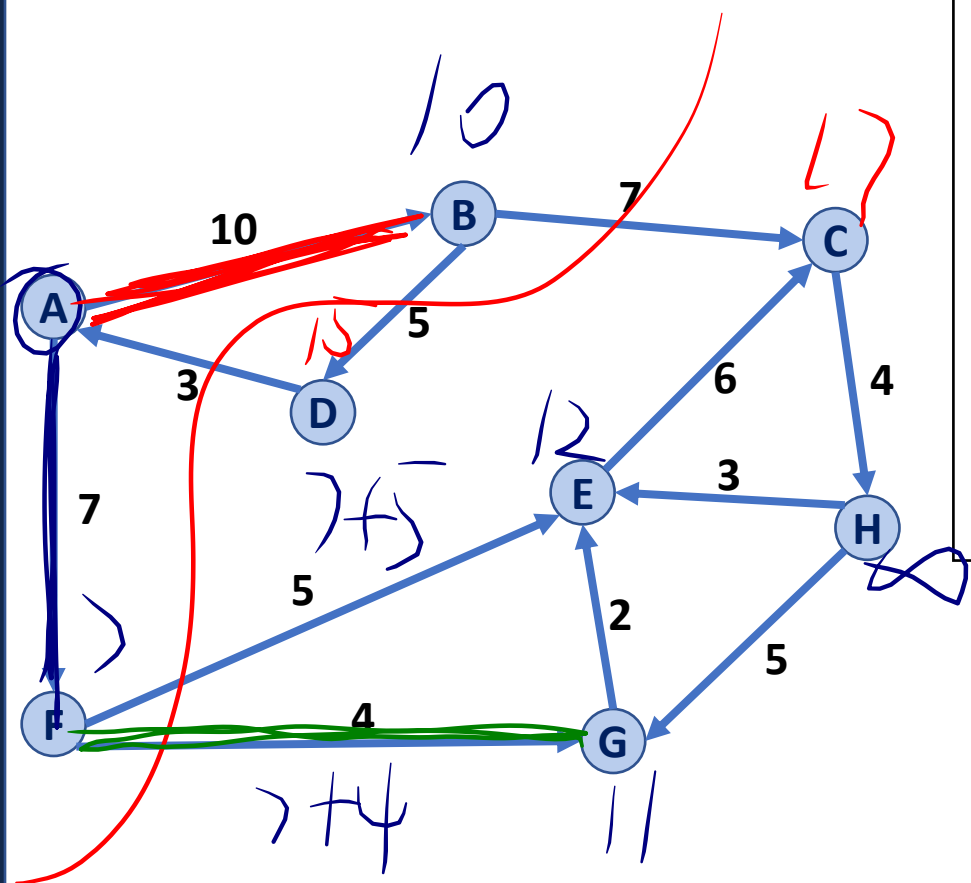
```
19          if  $\text{cost}(u, v) + d[u] < d[v]$ :
```

```
20              d[v] =  $\text{cost}(u, v) + d[u]$ 
```

```
21              p[v] = u
```

$\text{cost}(u, v) + d[u]$

Dijkstra's Algorithm (SSSP)



```
DijkstraSSSP(G, s):
```

```
6  foreach (Vertex v : G):
```

```
7      d[v] = +inf
```

```
8      p[v] = NULL
```

```
9      d[s] = 0
```

```
10
```

```
11  PriorityQueue Q // min distance, defined by d[v]
```

```
12  Q.buildHeap(G.vertices())
```

```
13  Graph T          // "labeled set"
```

```
14
```

```
15  repeat n times:
```

```
16      Vertex u = Q.removeMin()
```

```
17      T.add(u)
```

```
18      foreach (Vertex v : neighbors of u not in T):
```

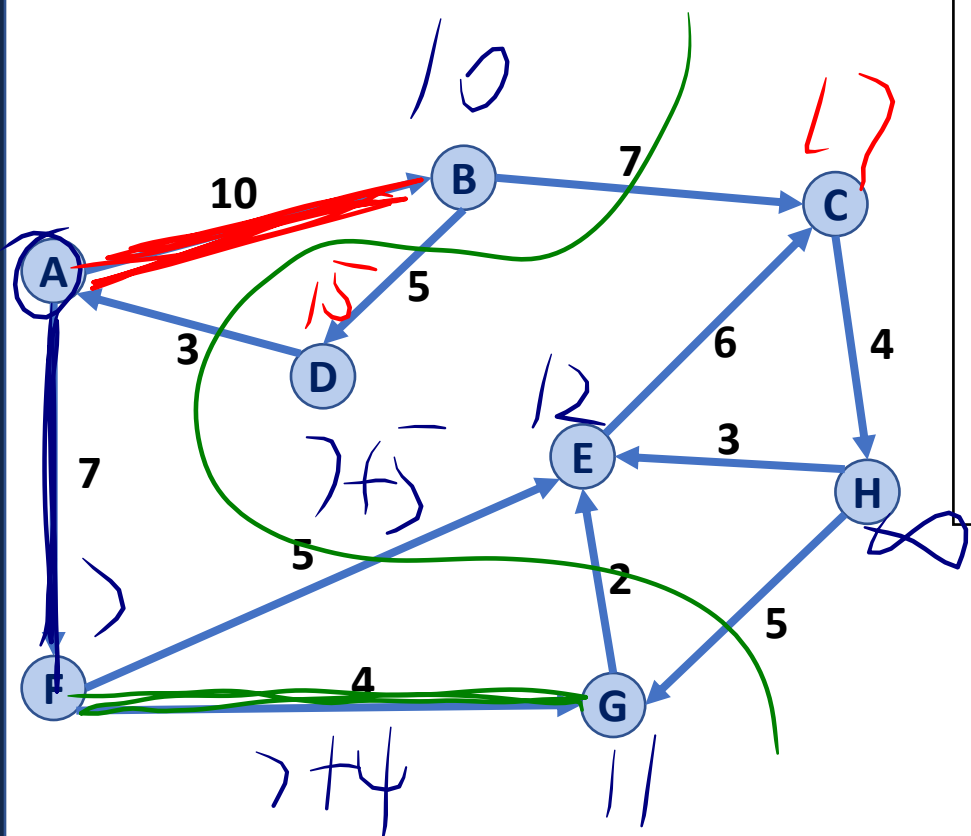
```
19          if  $d[u] + w(u,v) < d[v]$ :
```

```
20              d[v] =  $d[u] + w(u,v)$ 
```

```
21              p[v] = u
```

$d[u] + w(u,v)$

Dijkstra's Algorithm (SSSP)



```
DijkstraSSSP(G, s):
```

```
6  foreach (Vertex v : G):
```

```
7      d[v] = +inf
```

```
8      p[v] = NULL
```

```
9      d[s] = 0
```

```
10
```

```
11  PriorityQueue Q // min distance, defined by d[v]
```

```
12  Q.buildHeap(G.vertices())
```

```
13  Graph T          // "labeled set"
```

```
14
```

```
15  repeat n times:
```

```
16      Vertex u = Q.removeMin()
```

```
17      T.add(u)
```

```
18      foreach (Vertex v : neighbors of u not in T):
```

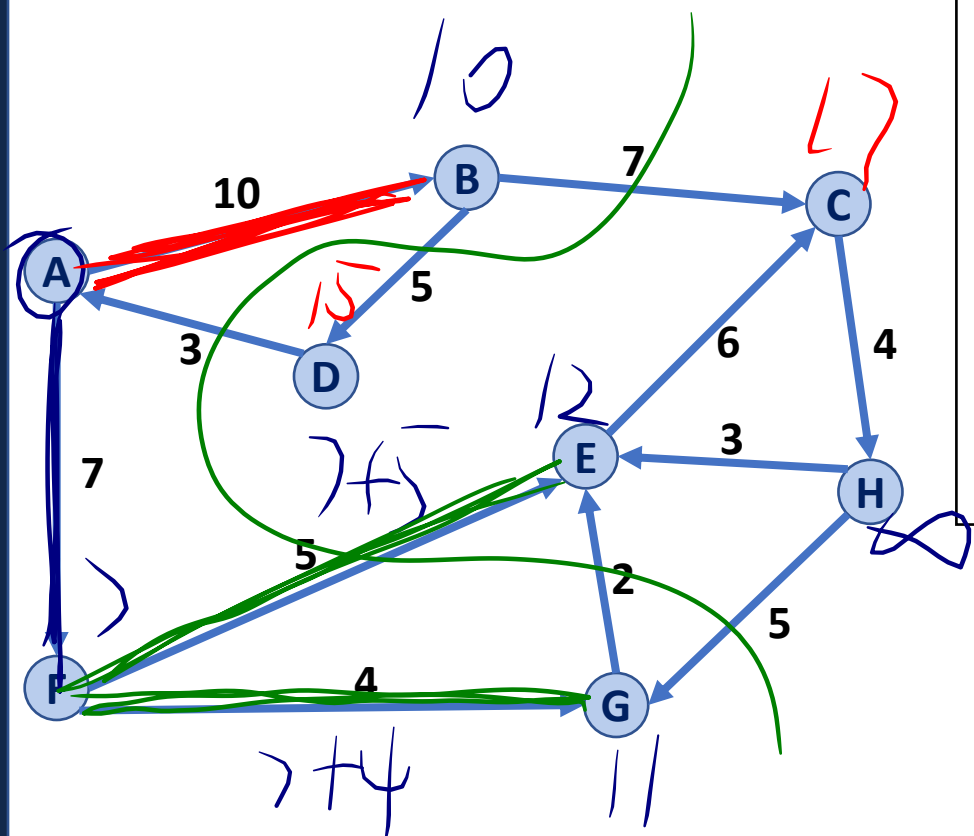
```
19          if  $d[u] + \text{cost}(u, v) < d[v]$ :
```

```
20              d[v] =  $d[u] + \text{cost}(u, v)$ 
```

```
21              p[v] = u
```

$\text{cost}(u, v) + d[u]$

Dijkstra's Algorithm (SSSP)



```
DijkstraSSSP(G, s):
```

```
6   foreach (Vertex v : G):
```

```
7       d[v] = +inf
```

```
8       p[v] = NULL
```

```
9       d[s] = 0
```

```
10
```

```
11   PriorityQueue Q // min distance, defined by d[v]
```

```
12   Q.buildHeap(G.vertices())
```

```
13   Graph T          // "labeled set"
```

```
14
```

```
15   repeat n times:
```

```
16       Vertex u = Q.removeMin()
```

```
17       T.add(u)
```

```
18       foreach (Vertex v : neighbors of u not in T):
```

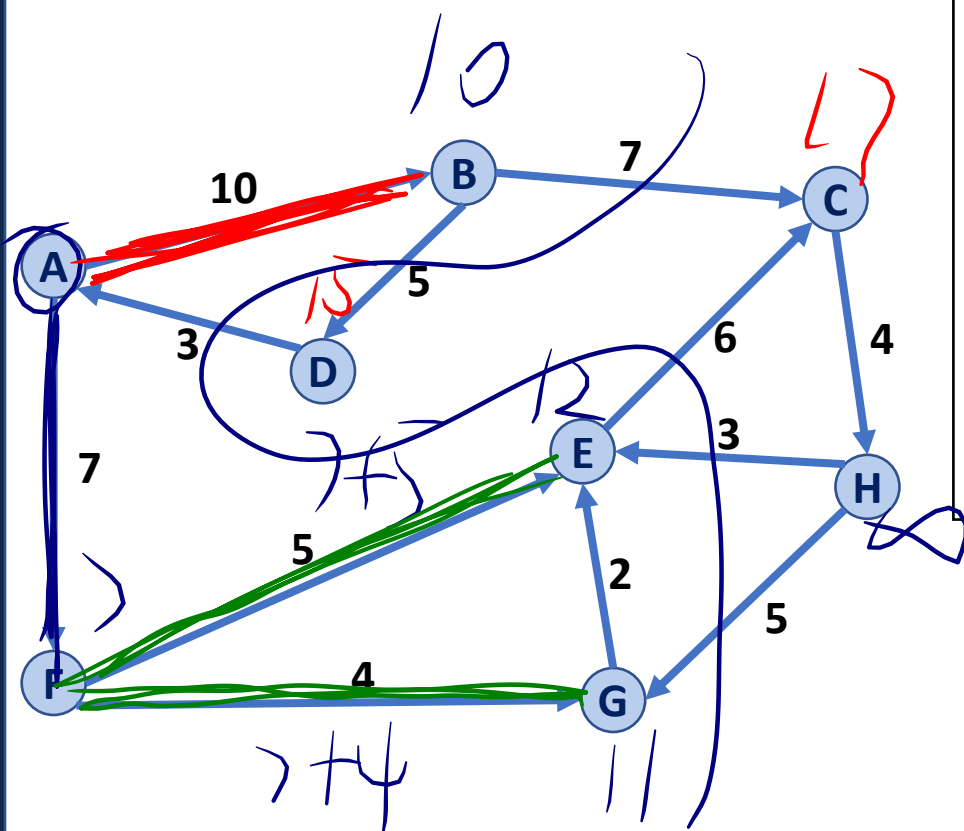
```
19           if  $\text{cost}(u, v) + d[u] < d[v]$ :
```

```
20               d[v] =  $\text{cost}(u, v) + d[u]$ 
```

```
21               p[v] = u
```

$\text{cost}(u, v) + d[u]$

Dijkstra's Algorithm (SSSP)



```
DijkstraSSSP(G, s):
```

```
6  foreach (Vertex v : G):
```

```
7      d[v] = +inf
```

```
8      p[v] = NULL
```

```
9      d[s] = 0
```

```
10
```

```
11  PriorityQueue Q // min distance, defined by d[v]
```

```
12  Q.buildHeap(G.vertices())
```

```
13  Graph T          // "labeled set"
```

```
14
```

```
15  repeat n times:
```

```
16      Vertex u = Q.removeMin()
```

```
17      T.add(u)
```

```
18      foreach (Vertex v : neighbors of u not in T):
```

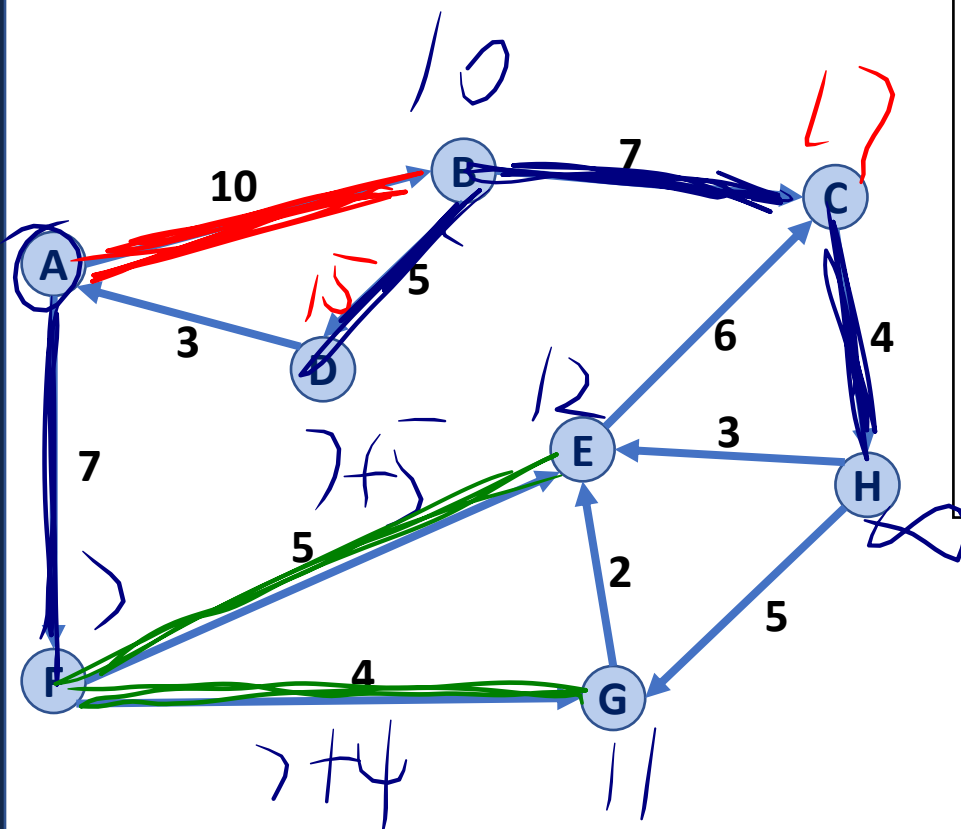
```
19          if  $\text{cost}(u, v) + d[u] < d[v]$ :
```

```
20              d[v] =  $\text{cost}(u, v) + d[u]$ 
```

```
21              p[v] = u
```

$\text{cost}(u, v) + d[u]$

Dijkstra's Algorithm (SSSP)



```
DijkstraSSSP(G, s):
```

```
6  foreach (Vertex v : G):
```

```
7      d[v] = +inf
```

```
8      p[v] = NULL
```

```
9      d[s] = 0
```

```
10
```

```
11  PriorityQueue Q // min distance, defined by d[v]
```

```
12  Q.buildHeap(G.vertices())
```

```
13  Graph T          // "labeled set"
```

```
14
```

```
15  repeat n times:
```

```
16      Vertex u = Q.removeMin()
```

```
17      T.add(u)
```

```
18      foreach (Vertex v : neighbors of u not in T):
```

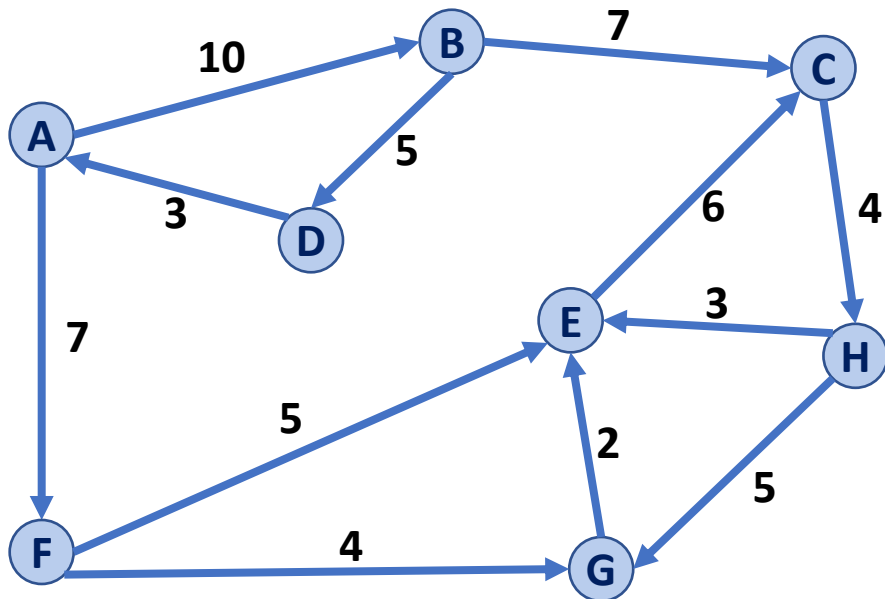
```
19          if  $\text{cost}(u, v) + d[u] < d[v]$ :
```

```
20              d[v] =  $\text{cost}(u, v) + d[u]$ 
```

```
21              p[v] = u
```

$\text{cost}(u, v) + d[u]$

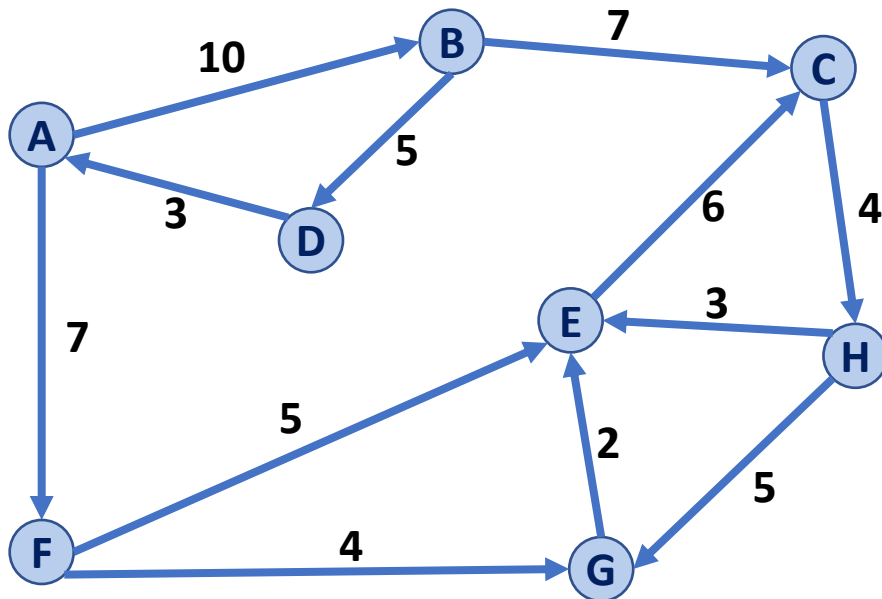
Dijkstra's Algorithm (SSSP)



```
DijkstraSSSP(G, s):
```

```
6   foreach (Vertex v : G):
7       d[v] = +inf
8       p[v] = NULL
9   d[s] = 0
10
11   PriorityQueue Q // min distance, defined by d[v]
12   Q.buildHeap(G.vertices())
13   Graph T          // "labeled set"
14
15   repeat n times:
16       Vertex u = Q.removeMin()
17       T.add(u)
18       foreach (Vertex v : neighbors of u not in T):
19           if cost(u, v) + d[u] < d[v]:
20               d[v] = cost(u, v) + d[u]
21               p[v] = u
```

Dijkstra's Algorithm (SSSP)

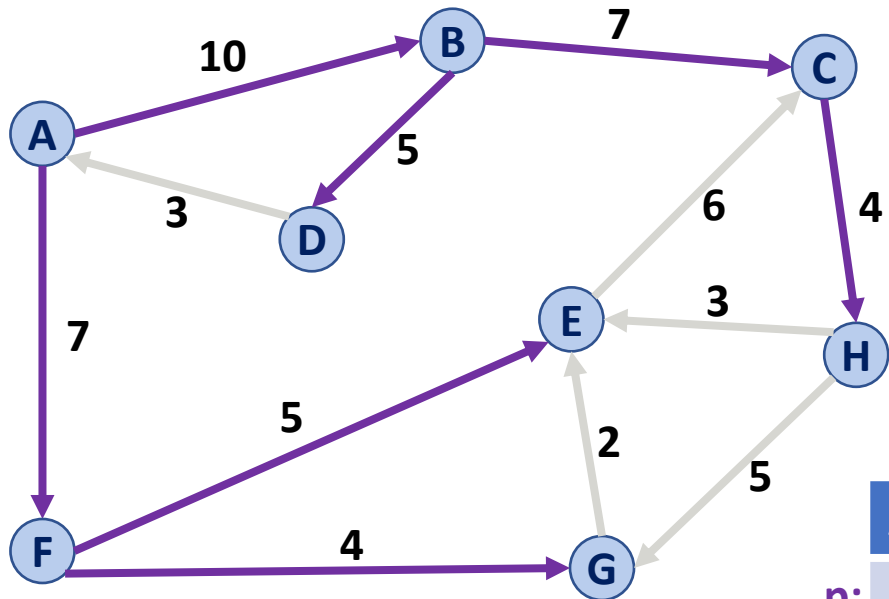


DijkstraSSSP(G, s):

```
6  foreach (Vertex v : G):
7      d[v] = +inf
8      p[v] = NULL
9  d[s] = 0
10
11  PriorityQueue Q // min distance, defined by d[v]
12  Q.buildHeap(G.vertices())
13  Graph T          // "labeled set"
14
15  repeat n times:
16      Vertex u = Q.removeMin()
17      T.add(u)
18      foreach (Vertex v : neighbors of u not in T):
19          if cost(u, v) + d[u] < d[v]:
20              d[v] = cost(u, v) + d[u]
21              p[v] = u
```

Dijkstra's Algorithm (SSSP)

Dijkstra gives us the shortest path from our path (single source) to **every** connected vertex!

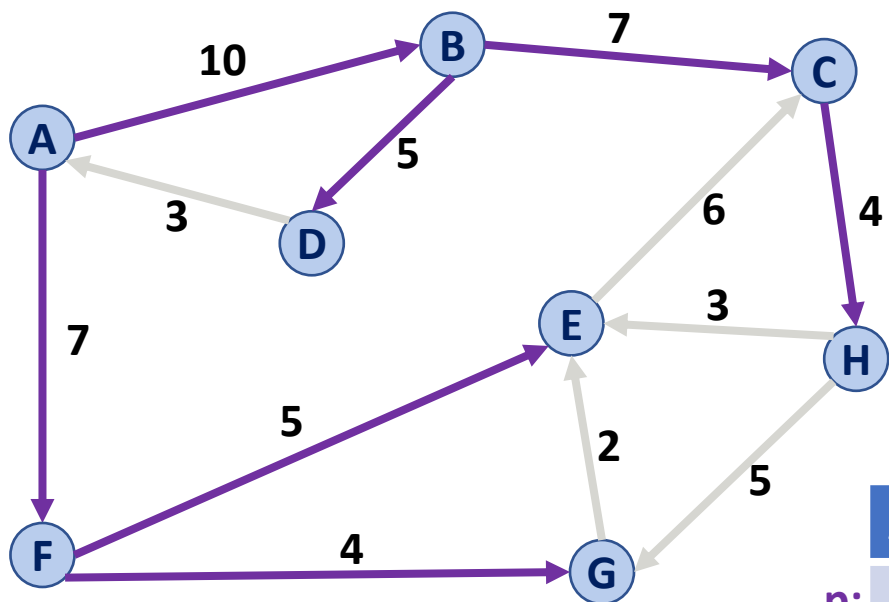


	A	B	C	D	E	F	G	H
p:	NULL	A	B	B	F	A	F	C
d:	0	10	17	15	12	7	11	21

Dijkstra's Algorithm (SSSP)

Dijkstra gives us the shortest path from our path (single source) to **every** connected vertex!

① Path from A to G

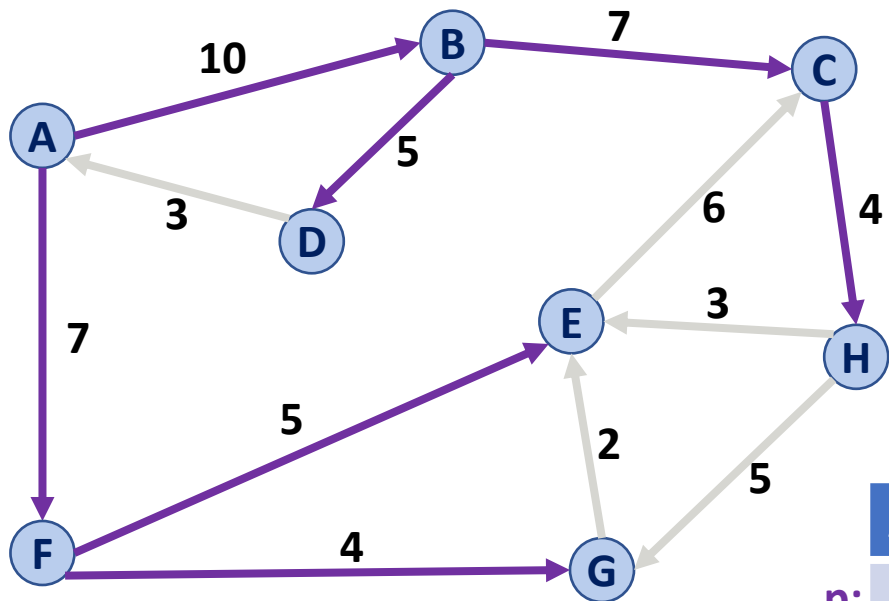


	A	B	C	D	E	F	G	H
p:	NULL	A	B	B	F	A	F	C
d:	0	10	17	15	12	7	11	21

Dijkstra's Algorithm (SSSP)

Dijkstra gives us the shortest path from our path (single source) to **every** connected vertex!

① Path from A to G

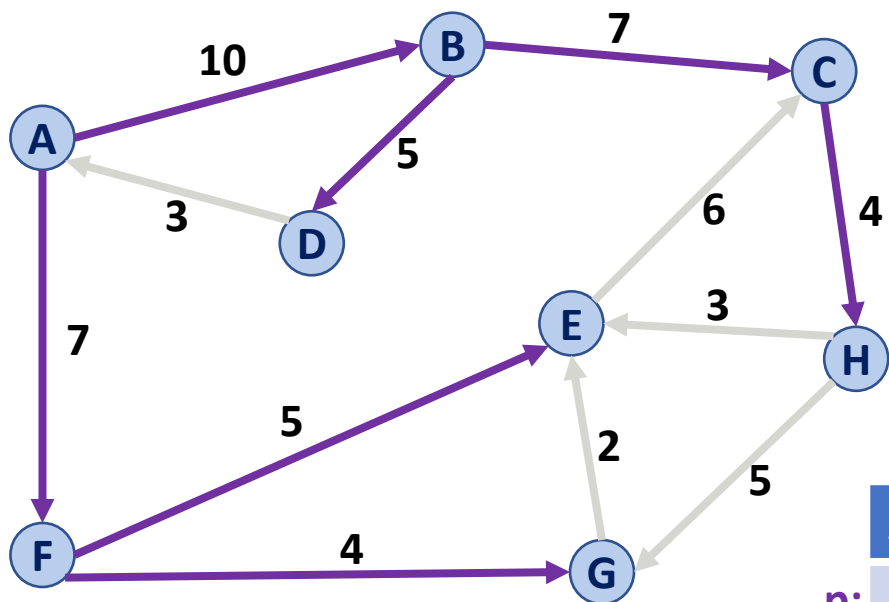


	A	B	C	D	E	F	G	H
p:	NULL	A	B	B	F	A	F	C
d:	0	10	17	15	12	7	11	21

Dijkstra's Algorithm (SSSP)

Dijkstra gives us the shortest path from our path (single source) to **every** connected vertex!

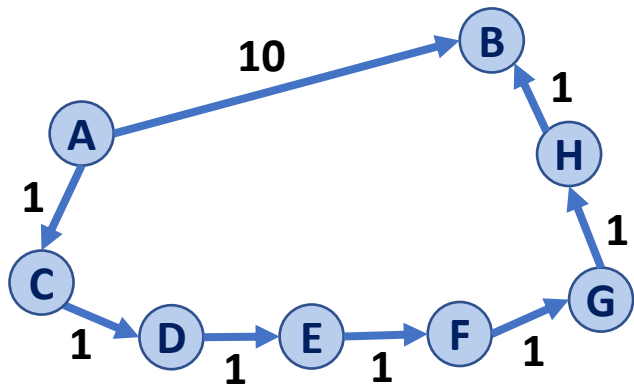
① Path from A to G



	A	B	C	D	E	F	G	H
p:	NULL	A	B	B	F	A	F	C
d:	0	10	17	15	12	7	11	21

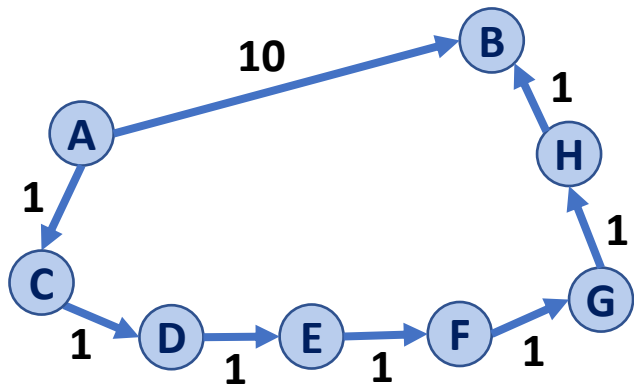
Dijkstra's Algorithm (SSSP)

Q: How does Dijkstra handle a single heavy-weight path vs. many light-weight paths?



Dijkstra's Algorithm (SSSP)

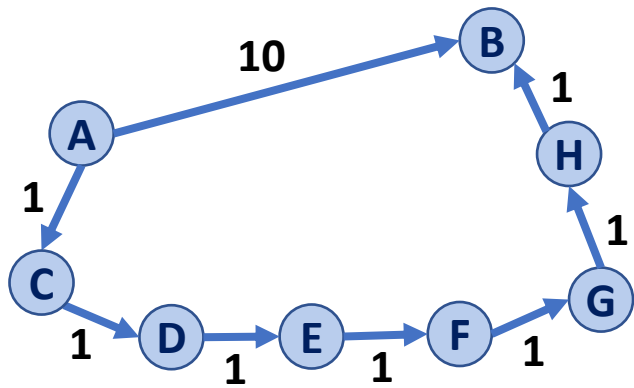
Q: How does Dijkstra handle a single heavy-weight path vs. many light-weight paths?



Shortest path $A \rightarrow B$?

Dijkstra's Algorithm (SSSP)

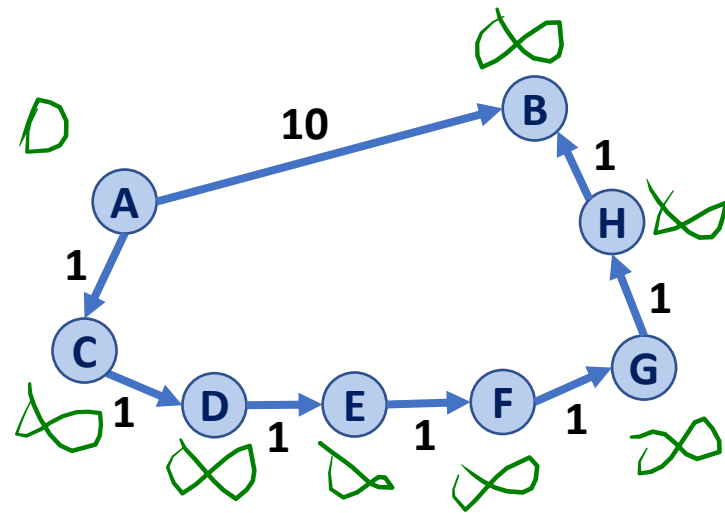
Q: How does Dijkstra handle a single heavy-weight path vs. many light-weight paths?



Shortest path $A \rightarrow B$?
• Run Dijkstra's on A.

Dijkstra's Algorithm (SSSP)

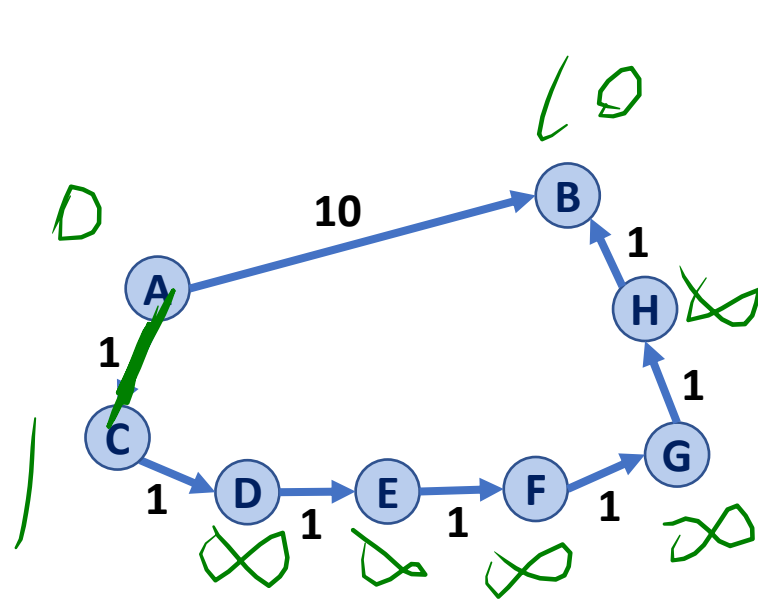
Q: How does Dijkstra handle a single heavy-weight path vs. many light-weight paths?



Shortest path $A \rightarrow B$?
• Run Dijkstra's on A.

Dijkstra's Algorithm (SSSP)

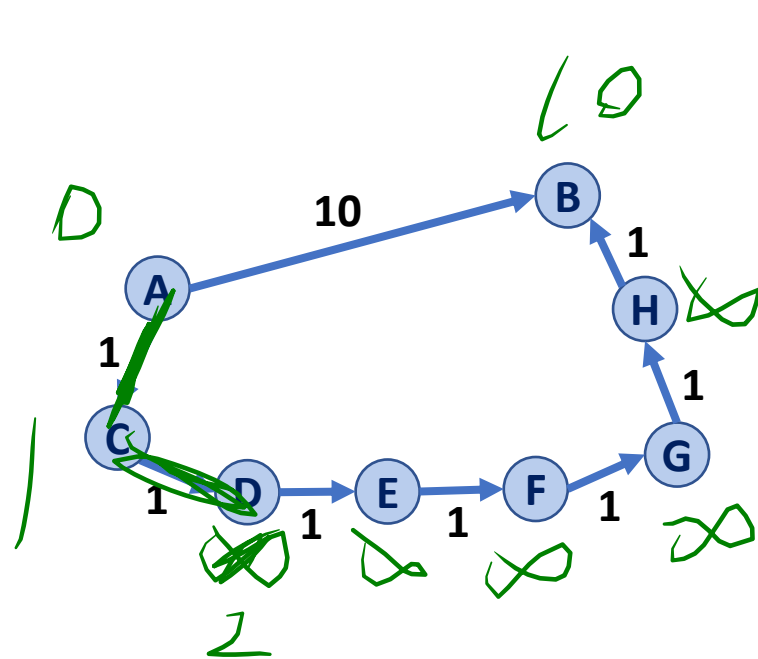
Q: How does Dijkstra handle a single heavy-weight path vs. many light-weight paths?



Shortest path $A \rightarrow B$?
• Run Dijkstra's on A.

Dijkstra's Algorithm (SSSP)

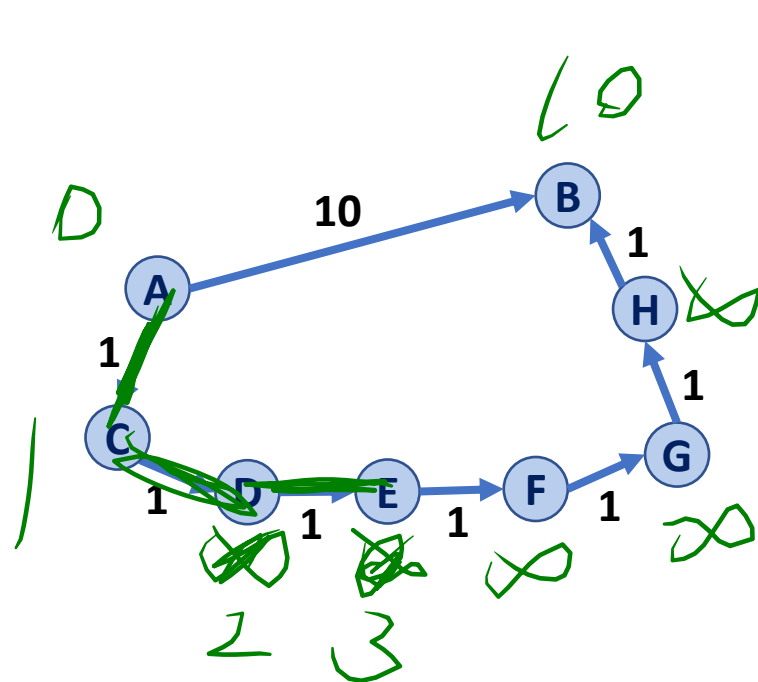
Q: How does Dijkstra handle a single heavy-weight path vs. many light-weight paths?



Shortest path $A \rightarrow B$?
• Run Dijkstra's on A.

Dijkstra's Algorithm (SSSP)

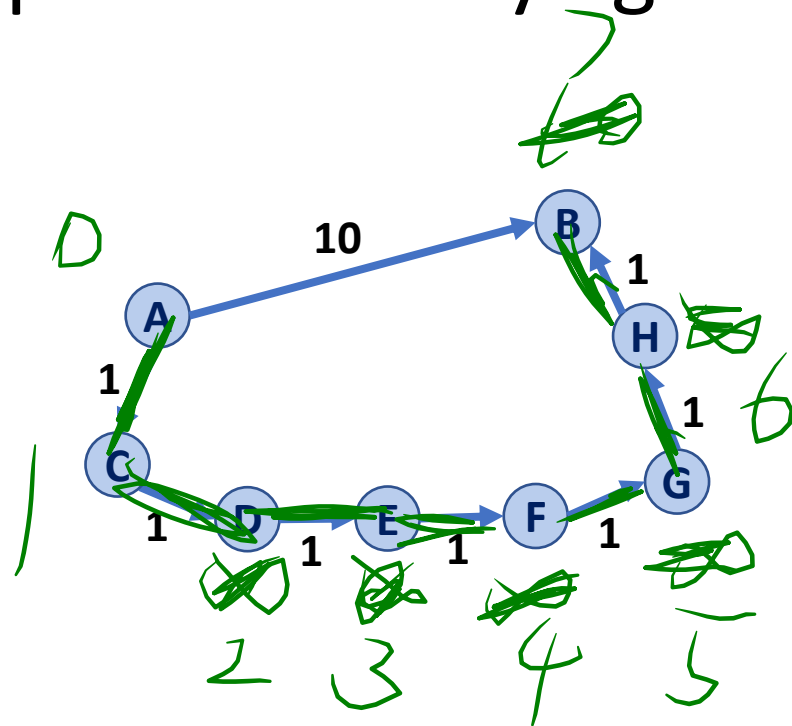
Q: How does Dijkstra handle a single heavy-weight path vs. many light-weight paths?



Shortest path $A \rightarrow B$?
• Run Dijkstra's on A.

Dijkstra's Algorithm (SSSP)

Q: How does Dijkstra handle a single heavy-weight path vs. many light-weight paths?

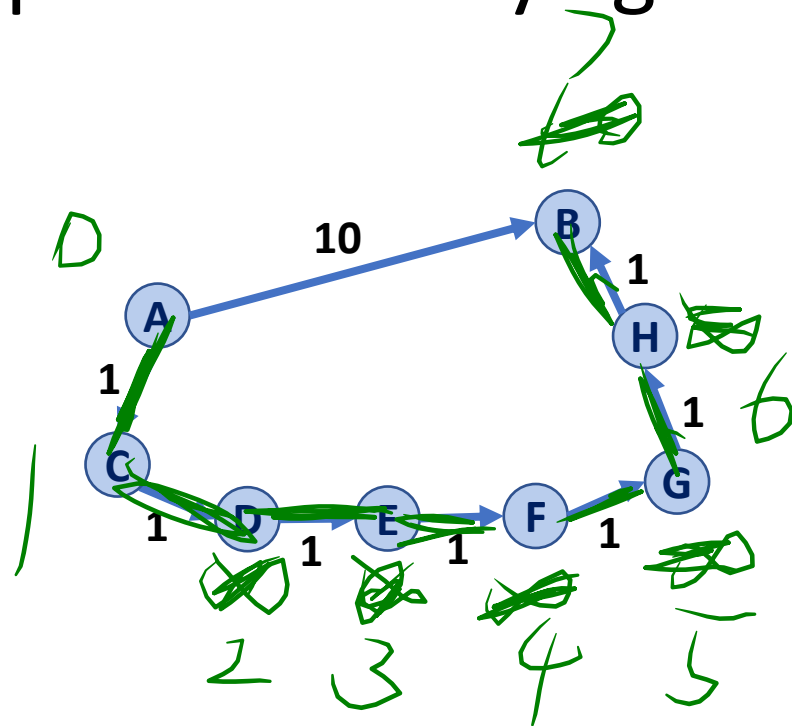


Shortest path $A \rightarrow B$?

- Run Dijkstra's on A.

Dijkstra's Algorithm (SSSP)

Q: How does Dijkstra handle a single heavy-weight path vs. many light-weight paths?



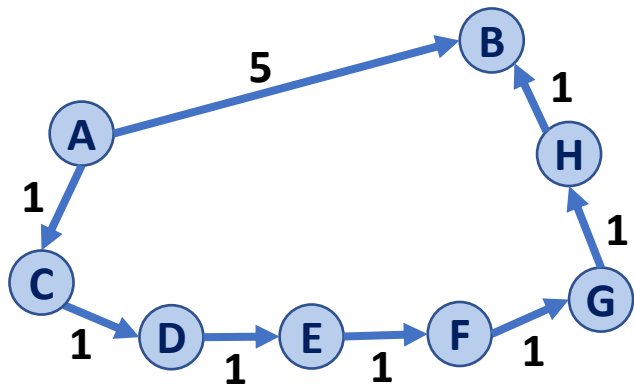
Shortest path $A \rightarrow B$?

- Run Dijkstra's on A.

ACDEFGHB

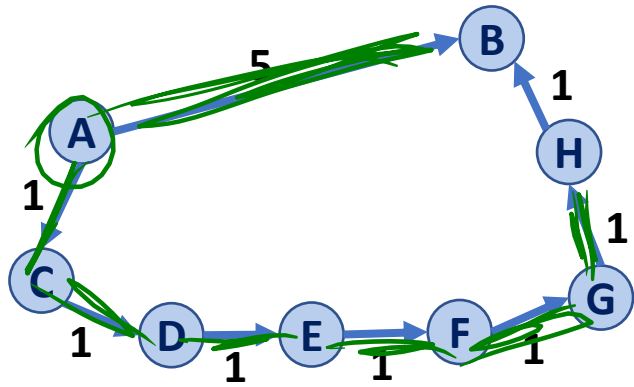
Dijkstra's Algorithm (SSSP)

Q: How does Dijkstra handle a single heavy-weight path vs. many light-weight paths?



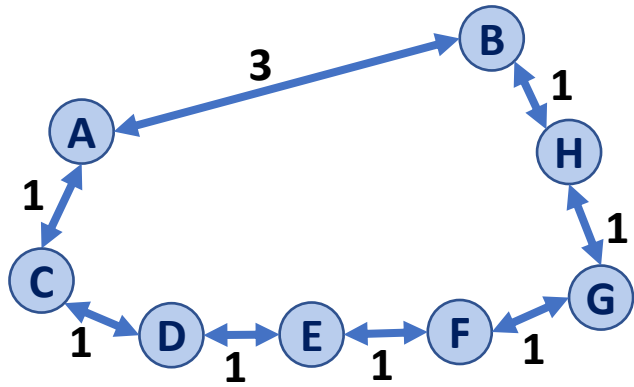
Dijkstra's Algorithm (SSSP)

Q: How does Dijkstra handle a single heavy-weight path vs. many light-weight paths?



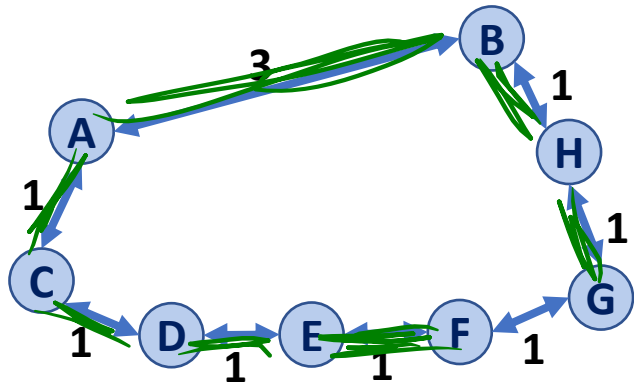
Dijkstra's Algorithm (SSSP)

Q: How does Dijkstra handle undirected graphs?



Dijkstra's Algorithm (SSSP)

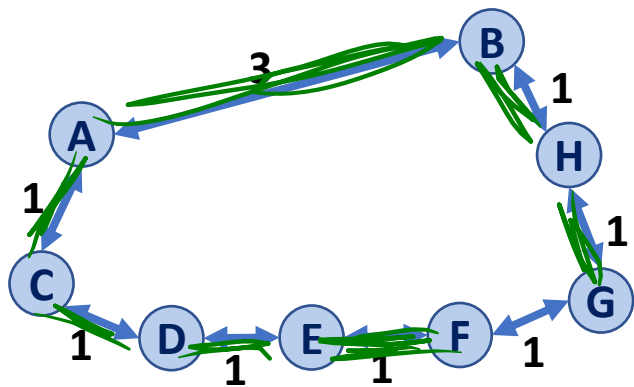
Q: How does Dijkstra handle undirected graphs?



Dijkstra's Algorithm (SSSP)

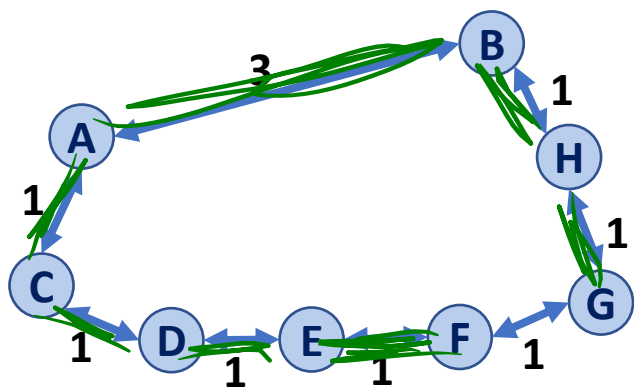
Q: How does Dijkstra handle undirected graphs?

Shortest path $A \rightarrow G$?



Dijkstra's Algorithm (SSSP)

Q: How does Dijkstra handle undirected graphs?

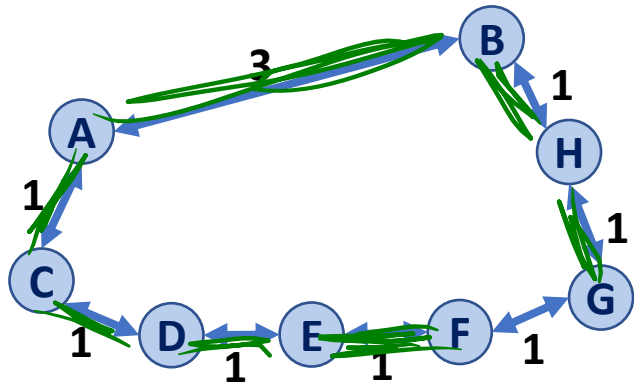


Shortest path $A \rightarrow G$?

A B H G

Dijkstra's Algorithm (SSSP)

Q: How does Dijkstra handle undirected graphs?



Shortest path $A \rightarrow G$?

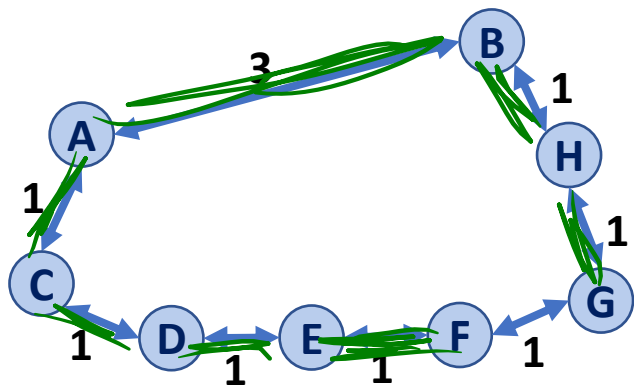
ABHG

or

ACDEFG

Dijkstra's Algorithm (SSSP)

Q: How does Dijkstra handle undirected graphs?



Shortest path $A \rightarrow G$?

ABHG

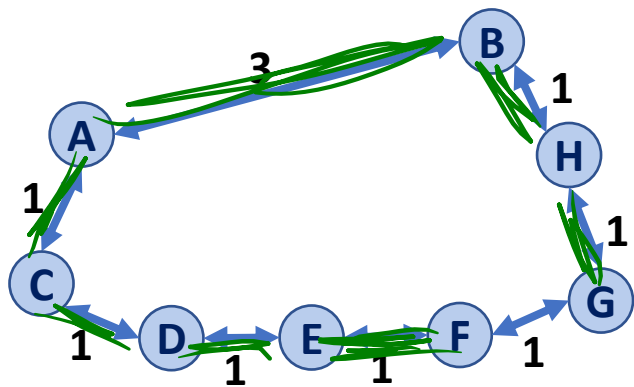
or

ACDEFG

★ A shortest path is not unique

Dijkstra's Algorithm (SSSP)

Q: How does Dijkstra handle undirected graphs?



Shortest path $A \rightarrow G$?

$A B H G$ $w=5$

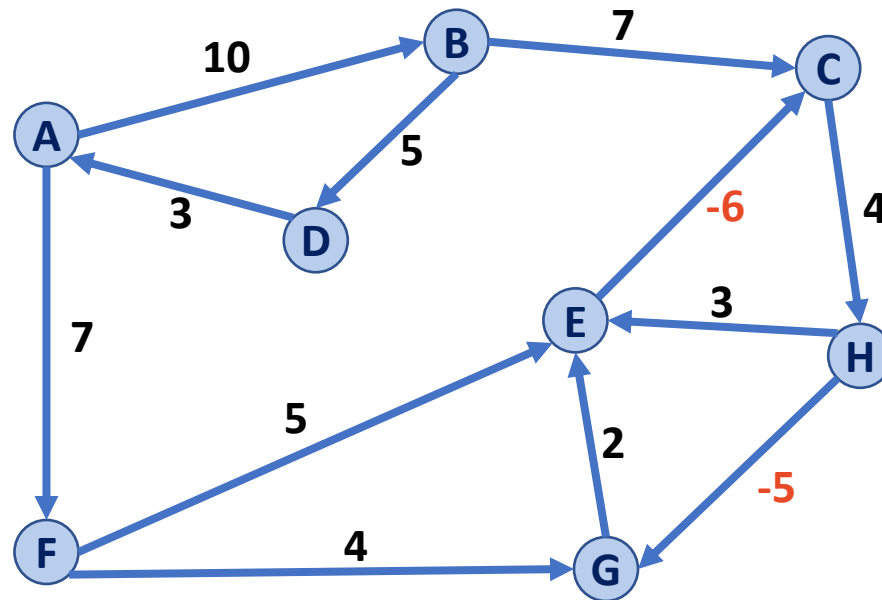
or

$A C D E F G$ $w=5$

★ A shortest path is not unique

Dijkstra's Algorithm (SSSP)

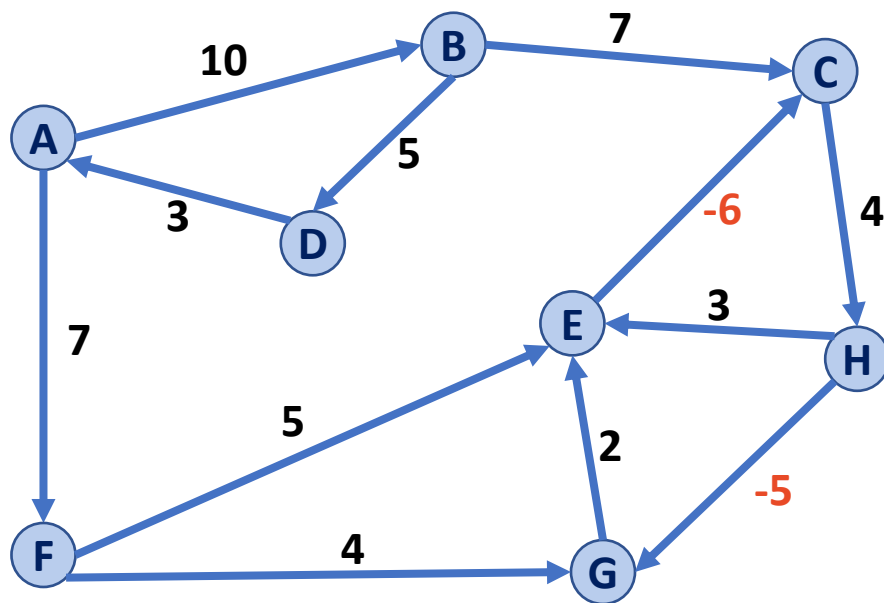
What about negative weight cycles?



Dijkstra's Algorithm (SSSP)

What about negative weight cycles?

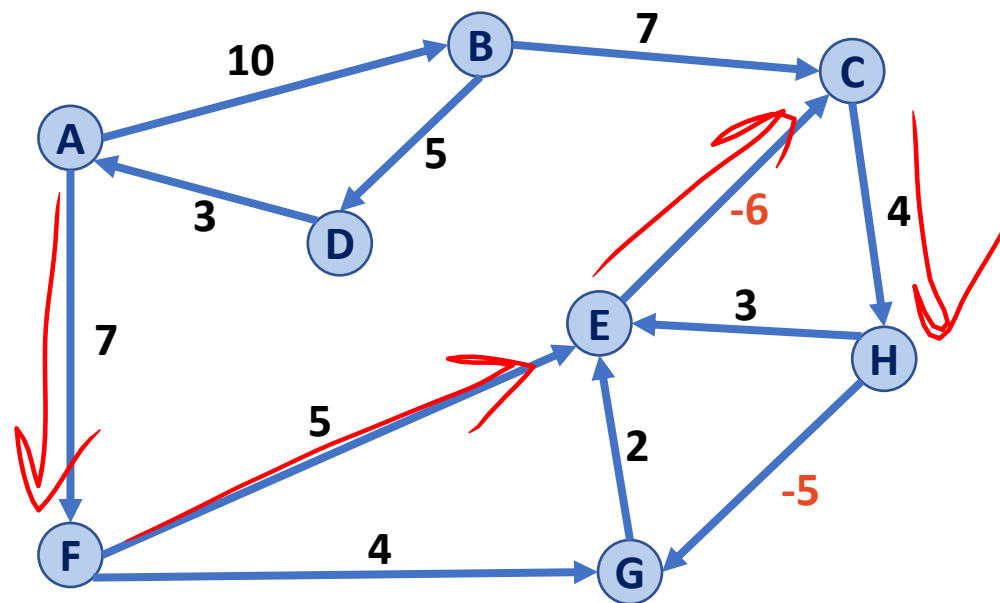
Shortest path A --- H



Dijkstra's Algorithm (SSSP)

What about negative weight cycles?

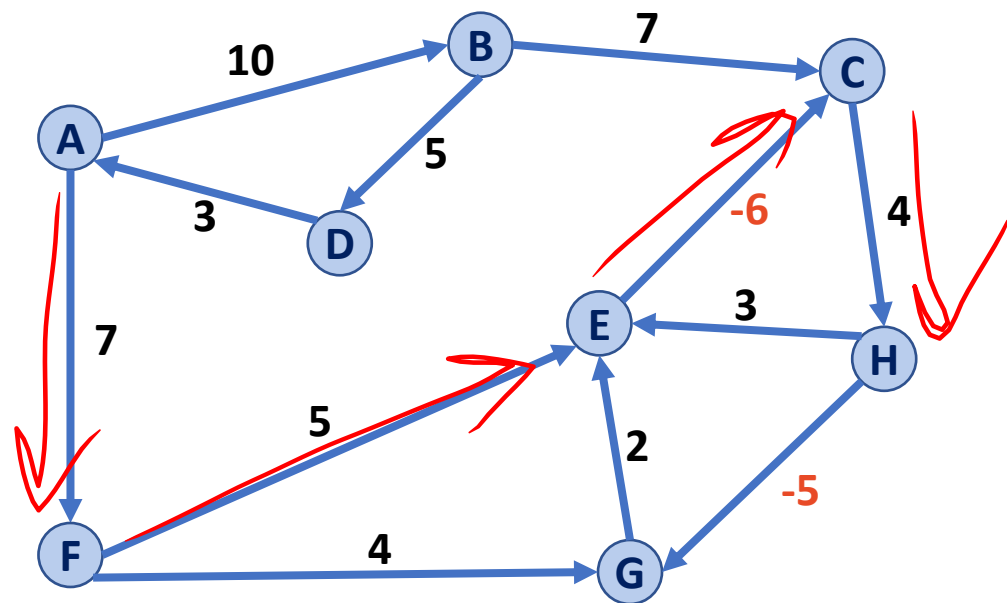
Shortest path A --- H



Dijkstra's Algorithm (SSSP)

What about negative weight cycles?

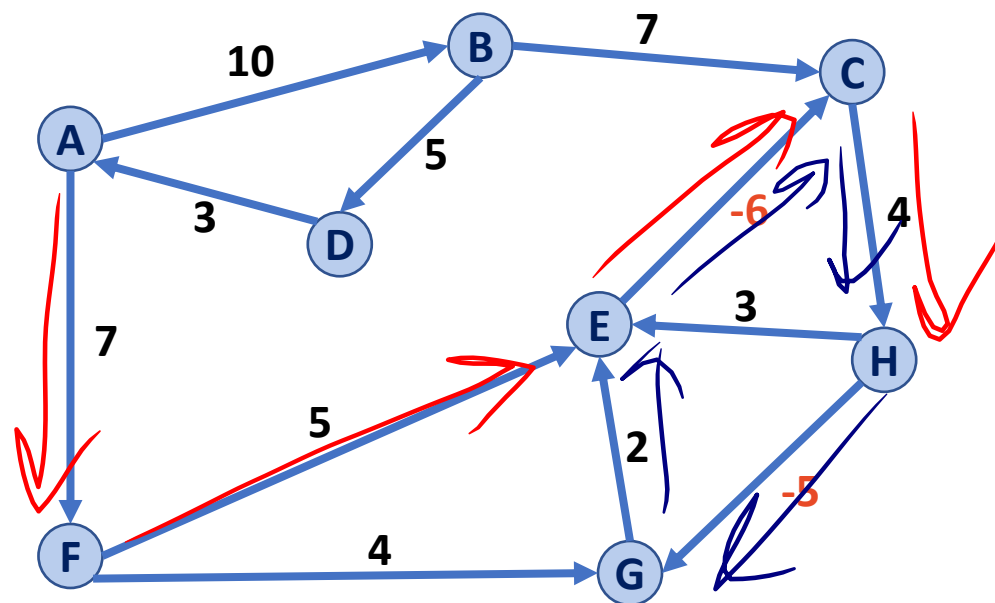
Shortest path A...H



A F E C H
w=10

Dijkstra's Algorithm (SSSP)

What about negative weight cycles?



Shortest path A...H

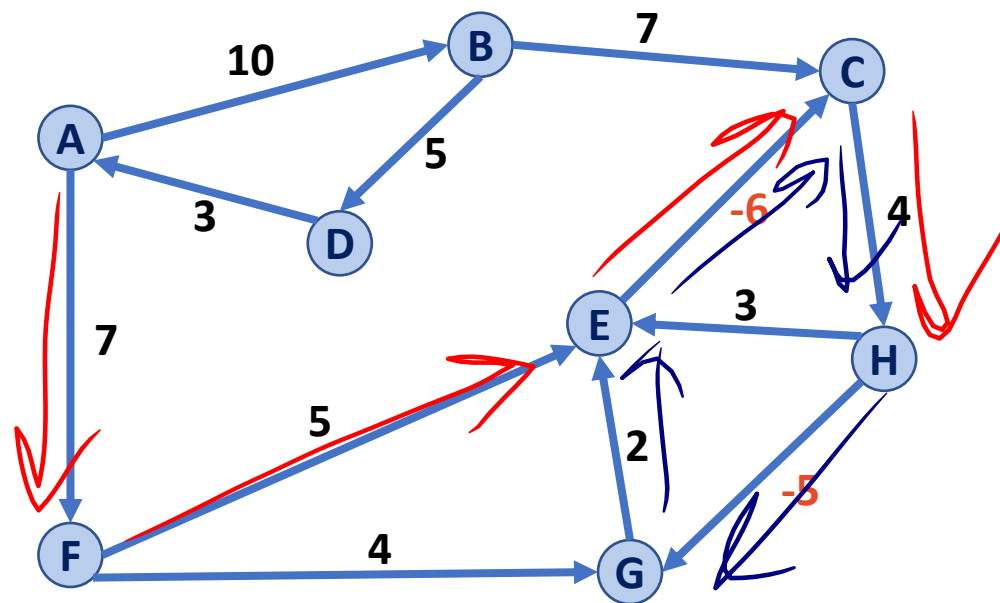
A F E C H $w=10$

H G E C H $w=-5$

Dijkstra's Algorithm (SSSP)

What about negative weight cycles?

Shortest path A...H



A F E C H $w=10$

H G E C H $w=-5$

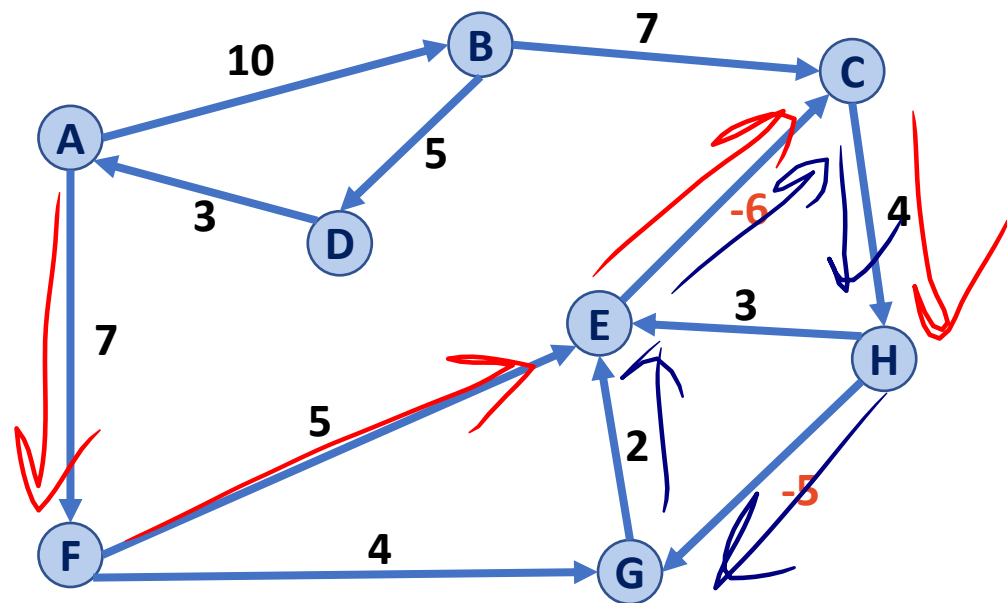
$A F E C (H G E C H)^x$

Dijkstra's Algorithm (SSSP)

What about negative weight cycles?

No shortest path exists w/

Shortest path A...H



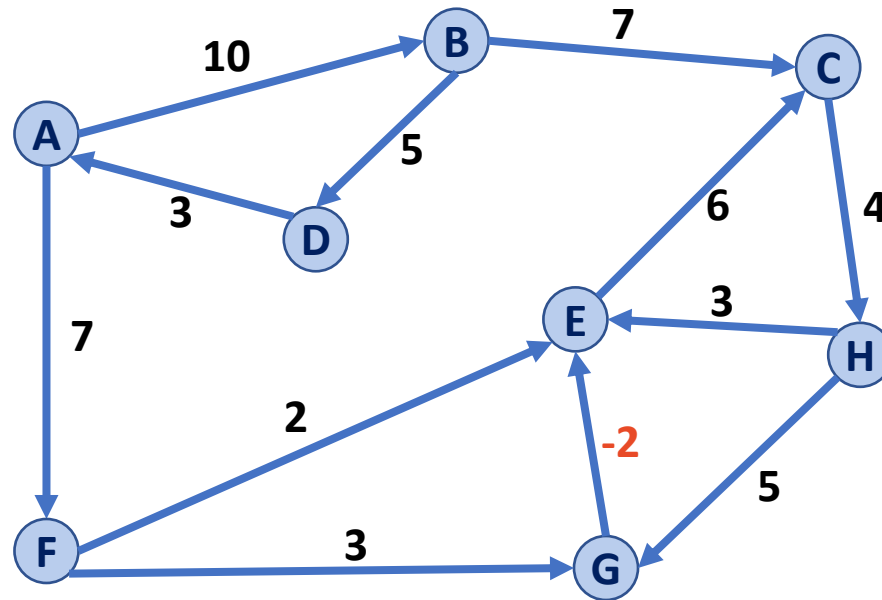
A F E C H $w=10$

H G E C H $w=-5$

$A F E C (H G E C H)^x$

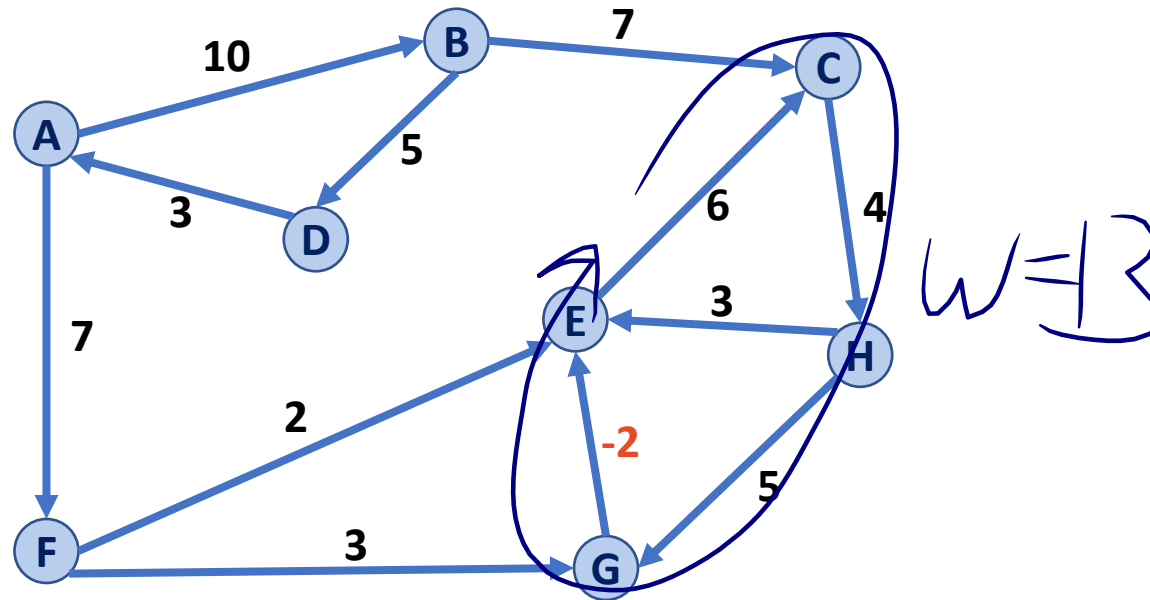
Dijkstra's Algorithm (SSSP)

What about negative weight edges, without negative weight cycles?



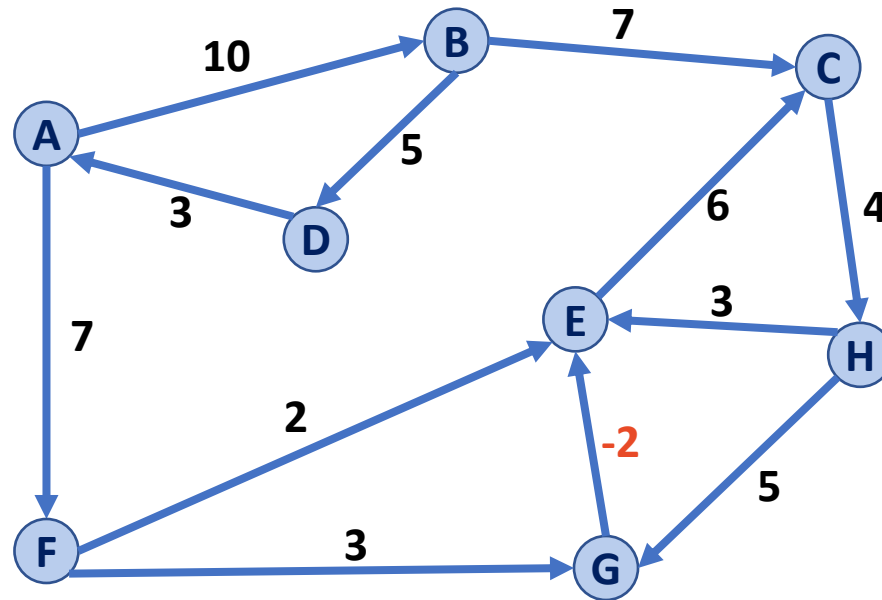
Dijkstra's Algorithm (SSSP)

What about negative weight edges, without negative weight cycles?



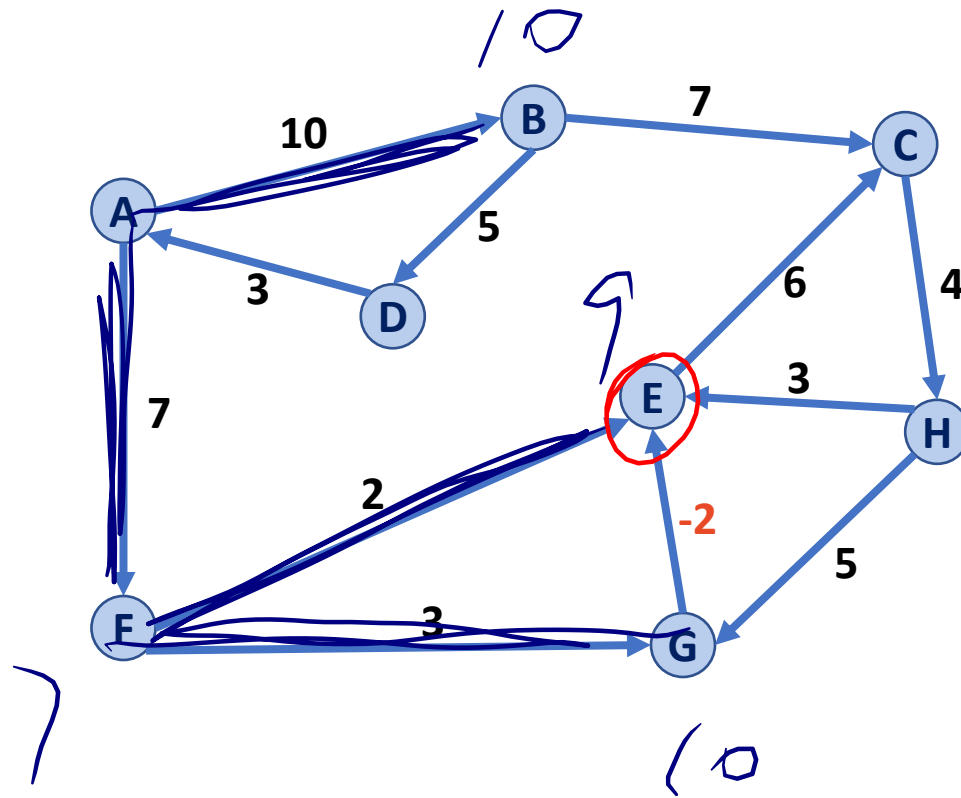
Dijkstra's Algorithm (SSSP)

What about negative weight edges, without negative weight cycles?



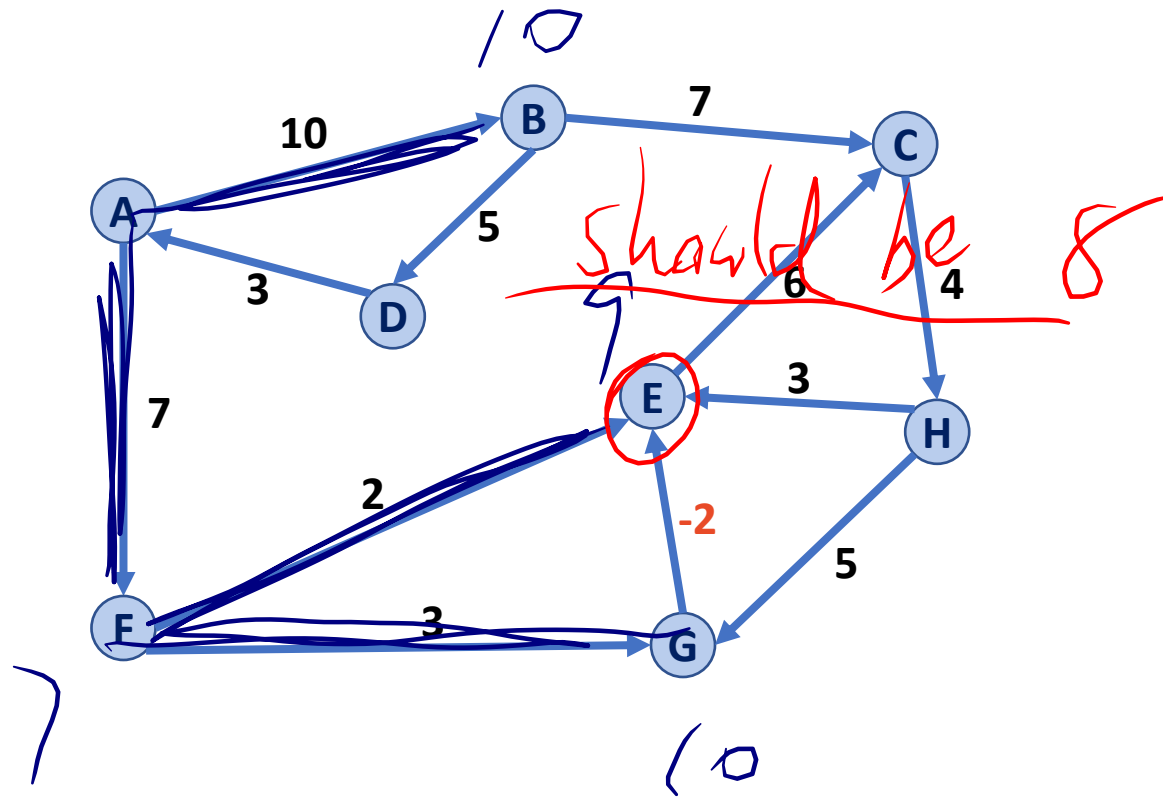
Dijkstra's Algorithm (SSSP)

What about negative weight edges, without negative weight cycles?

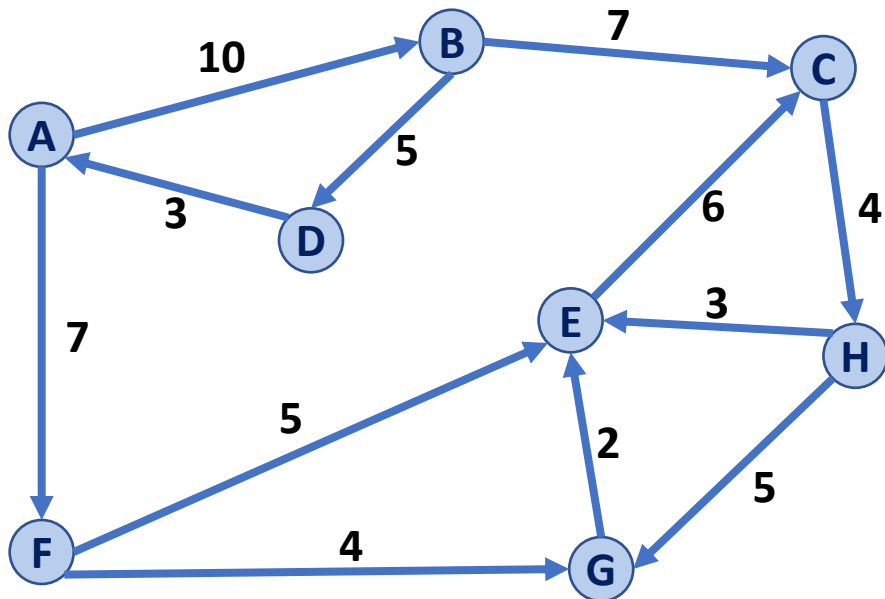


Dijkstra's Algorithm (SSSP)

What about negative weight edges, without negative weight cycles?



Dijkstra's Algorithm (SSSP)



```
DijkstraSSSP(G, s):
```

```
6   foreach (Vertex v : G):
```

```
7       d[v] = +inf
```

```
8       p[v] = NULL
```

```
9   d[s] = 0
```

```
10
```

```
11   PriorityQueue Q // min distance, defined by d[v]
```

```
12   Q.buildHeap(G.vertices())
```

```
13   Graph T           // "labeled set"
```

```
14
```

```
15   repeat n times:
```

```
16       Vertex u = Q.removeMin()
```

```
17       T.add(u)
```

```
18       foreach (Vertex v : neighbors of u not in T):
```

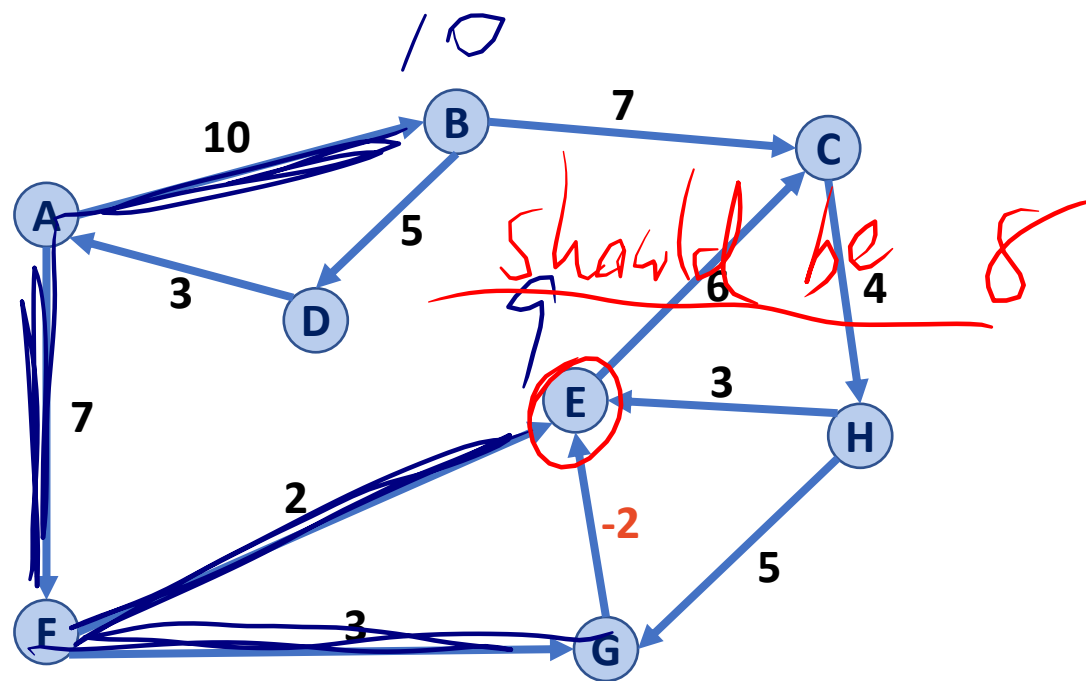
```
19           if cost(u, v) + d[u] < d[v]:
```

```
20               d[v] = cost(u, v) + d[u]
```

```
21               p[v] = u
```

Dijkstra's Algorithm (SSSP)

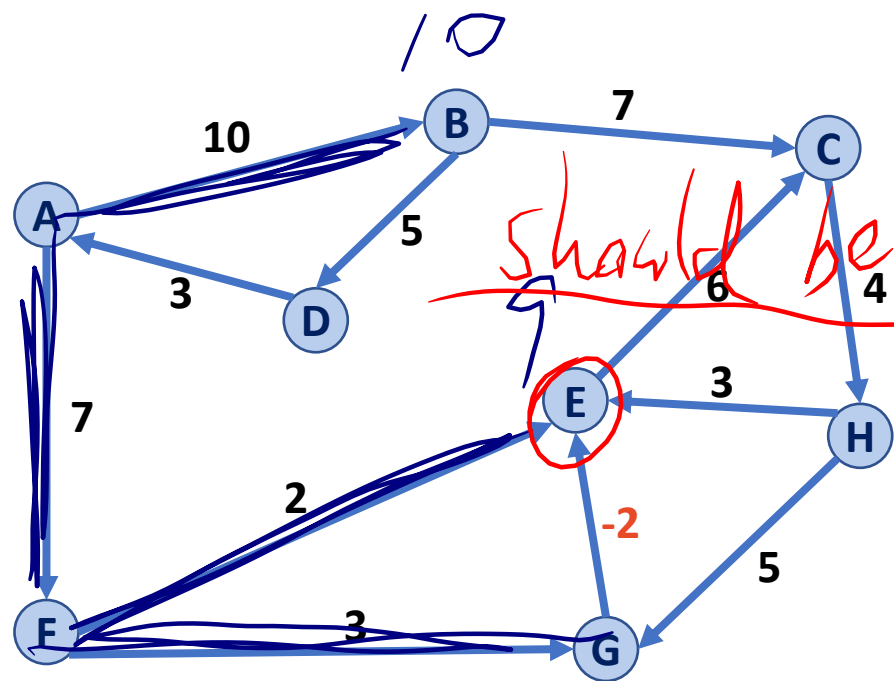
What about negative weight edges, without negative weight cycles?



Assumes: Once vertex is in the labeled set its optimal path is found

Dijkstra's Algorithm (SSSP)

What about negative weight edges, without negative weight cycles?



Does not work
w/ neg. weight
edges

Assumes: once vertex is in the labeled set
its optimal path is found



CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

Dijkstra's Algorithm (SSSP)

What is Dijkstra's running time?

```

DijkstraSSSP(G, s):
6   foreach (Vertex v : G):
7       d[v] = +inf
8       p[v] = NULL
9   d[s] = 0
10
11   PriorityQueue Q // min distance, defined by d[v]
12   Q.buildHeap(G.vertices())
13   Graph T          // "labeled set"
14
15   repeat n times:
16       Vertex u = Q.removeMin()
17       T.add(u)
18       foreach (Vertex v : neighbors of u not in T):
19           if cost(u, v) + d[u] < d[v]:
20               d[v] = cost(u, v) + d[u]
21               p[v] = u
22
23   return T
```

Dijkstra's Algorithm (SSSP)

What is Dijkstra's running time?

$\equiv$ Prim's

```
DijkstraSSSP(G, s):
6   foreach (Vertex v : G):
7       d[v] = +inf
8       p[v] = NULL
9   d[s] = 0
10
11   PriorityQueue Q // min distance, defined by d[v]
12   Q.buildHeap(G.vertices())
13   Graph T          // "labeled set"
14
15   repeat n times:
16       Vertex u = Q.removeMin()
17       T.add(u)
18       foreach (Vertex v : neighbors of u not in T):
19           if cost(u, v) + d[u] < d[v]:
20               d[v] = cost(u, v) + d[u]
21               p[v] = u
22
23   return T
```

Dijkstra's Algorithm (SSSP)

What is Dijkstra's running time?

$\equiv$ Prim's

Best

$O(m + n \lg(n))$

```

DijkstraSSSP(G, s):
6   foreach (Vertex v : G):
7       d[v] = +inf
8       p[v] = NULL
9   d[s] = 0
10
11   PriorityQueue Q // min distance, defined by d[v]
12   Q.buildHeap(G.vertices())
13   Graph T          // "labeled set"
14
15   repeat n times:
16       Vertex u = Q.removeMin()
17       T.add(u)
18       foreach (Vertex v : neighbors of u not in T):
19           if cost(u, v) + d[u] < d[v]:
20               d[v] = cost(u, v) + d[u]
21               p[v] = u
22
23   return T
```

Dijkstra's Algorithm (SSSP)

What is Dijkstra's running time?

$\equiv$ Prim's

Best

$O(m + n \lg(n))$

with Fib. heap

```
DijkstraSSSP(G, s):
6   foreach (Vertex v : G):
7       d[v] = +inf
8       p[v] = NULL
9   d[s] = 0
10
11   PriorityQueue Q // min distance, defined by d[v]
12   Q.buildHeap(G.vertices())
13   Graph T          // "labeled set"
14
15   repeat n times:
16       Vertex u = Q.removeMin()
17       T.add(u)
18       foreach (Vertex v : neighbors of u not in T):
19           if cost(u, v) + d[u] < d[v]:
20               d[v] = cost(u, v) + d[u]
21               p[v] = u
22
23   return T
```

Floyd-Warshall Algorithm

Floyd-Warshall's Algorithm is an alternative to Dijkstra in the presence of **negative-weight edges** (not **negative weight cycles**).

```
FloydWarshall(G):  
6   Let d be a adj. matrix initialized to +inf  
7   foreach (Vertex v : G):  
8       d[v, v] = 0  
9   foreach (Edge (u, v) : G):  
10      d[u, v] = cost(u, v)  
11  
12  foreach (Vertex w : G):  
13      foreach (Vertex u : G):  
14          foreach (Vertex v : G):  
15              if d[u, v] > d[u, w] + d[w, v]:  
16                  d[u, v] = d[u, w] + d[w, v]
```

Floyd-Warshall Algorithm

Floyd-Warshall's Algorithm is an alternative to Dijkstra in the presence of **negative-weight edges** (not **negative weight cycles**).

```
FloydWarshall(G):  
6   Let d be a adj. matrix initialized to +inf  
7   foreach (Vertex v : G):  
8       d[v, v] = 0  
9   foreach (Edge (u, v) : G):  
10      d[u, v] = cost(u, v)  
11  
12  foreach (Vertex w : G):  
13      foreach (Vertex u : G):  
14          foreach (Vertex v : G):  
15              if d[u, v] > d[u, w] + d[w, v]:  
16                  d[u, v] = d[u, w] + d[w, v]
```

Floyd-Warshall Algorithm

Floyd-Warshall's Algorithm is an alternative to Dijkstra in the presence of **negative-weight edges** (not **negative weight cycles**).

	A	B	C	D
A	0	6	∞	∞
B	∞	0	∞	∞
C	∞	∞	0	∞
D	∞	∞	∞	0

```
FloydWarshall(G):  
6   Let d be a adj. matrix initialized to +inf  
7   foreach (Vertex v : G):  
8       d[v, v] = 0  
9   foreach (Edge (u, v) : G):  
10      d[u, v] = cost(u, v)  
11  
12  foreach (Vertex w : G):  
13      foreach (Vertex u : G):  
14          foreach (Vertex v : G):  
15              if d[u, v] > d[u, w] + d[w, v]:  
16                  d[u, v] = d[u, w] + d[w, v]
```

Floyd-Warshall Algorithm

Floyd-Warshall's Algorithm is an alternative to Dijkstra in the presence of **negative-weight edges** (not **negative weight cycles**).

	A	B	C	D
A	0	6	∞	7
B	∞	0	∞	∞
C	∞	∞	0	∞
D	∞	∞	∞	0

$O(n^3)$

```
FloydWarshall(G):  
6   Let d be a adj. matrix initialized to +inf  
7   foreach (Vertex v : G):  
8       d[v, v] = 0  
9   foreach (Edge (u, v) : G):  
10      d[u, v] = cost(u, v)  
11  
12  foreach (Vertex w : G):  
13      foreach (Vertex u : G):  
14          foreach (Vertex v : G):  
15              if d[u, v] > d[u, w] + d[w, v]:  
16                  d[u, v] = d[u, w] + d[w, v]
```

Floyd-Warshall Algorithm

Let $d_w[u,v]$ be the weight of the shortest path $u \rightarrow v$ that goes only through nodes $0, 1, \dots, w-1$.

```
FloydWarshall(G):  
6   Let d be a adj. matrix initialized to +inf  
7   foreach (Vertex v : G):  
8       d[v, v] = 0  
9   foreach (Edge (u, v) : G):  
10      d[u, v] = cost(u, v)  
11  
12  foreach (Vertex w : G):  
13      foreach (Vertex u : G):  
14          foreach (Vertex v : G):  
15              if d[u, v] > d[u, w] + d[w, v]:  
16                  d[u, v] = d[u, w] + d[w, v]
```

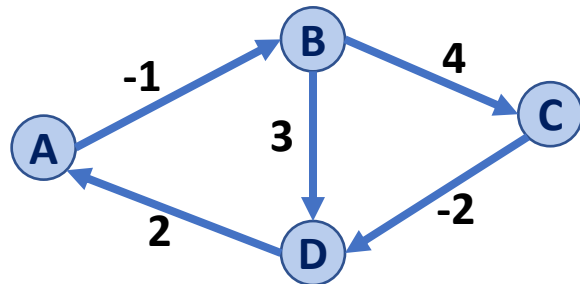
$$d_{w+1}[u,v] = \min(d_w[u,v], d_w[u,w] + d_w[w,v])$$

Floyd-Warshall Algorithm

```
FloydWarshall(G) :
```

```
6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G) :
8       d[v, v] = 0
9   foreach (Edge (u, v) : G) :
10      d[u, v] = cost(u, v)
11
12   foreach (Vertex w : G) :
13       foreach (Vertex u : G) :
14           foreach (Vertex v : G) :
15               if d[u, v] > d[u, w] + d[w, v]:
16                   d[u, v] = d[u, w] + d[w, v]
```

	A	B	C	D
A				
B				
C				
D				

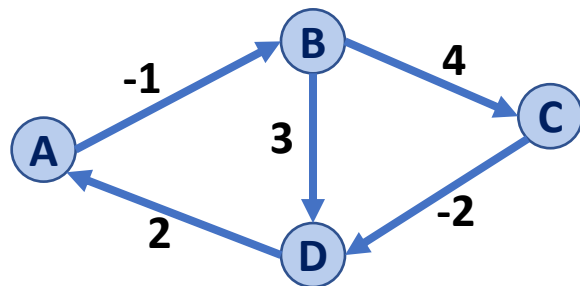


Floyd-Warshall Algorithm

```
FloydWarshall(G):
```

```
6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12   foreach (Vertex w : G):
13       foreach (Vertex u : G):
14           foreach (Vertex v : G):
15               if d[u, v] > d[u, w] + d[w, v]:
16                   d[u, v] = d[u, w] + d[w, v]
```

	A	B	C	D
A	0	-1		
B		0	4	3
C			0	-2
D	2			0

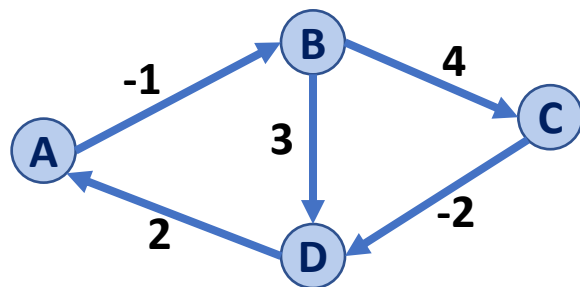


Floyd-Warshall Algorithm

FloydWarshall(G) :

```
6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```

	A	B	C	D
A	0	-1	∞	∞
B	∞	0	4	3
C	∞	∞	0	-2
D	2	∞	∞	0



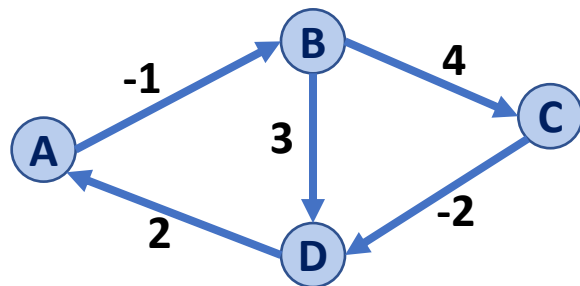
Floyd-Warshall Algorithm

FloydWarshall(G) :

```
6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12   foreach (Vertex w : G):
13       foreach (Vertex u : G):
14           foreach (Vertex v : G):
15               if d[u, v] > d[u, w] + d[w, v]:
16                   d[u, v] = d[u, w] + d[w, v]
```

	A	B	C	D
A	0	-1	∞	∞
B	∞	0	4	3
C	∞	∞	0	-2
D	2	∞	∞	0

Set $w = A$



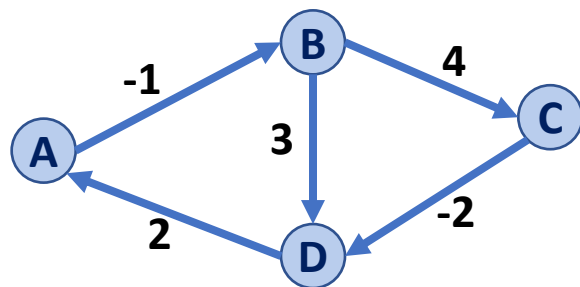
Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```

	A	B	C	D
A	0	-1	∞	∞
B	∞	0	4	3
C	∞	∞	0	-2
D	2	∞	∞	0



Set $w = A$

B-A-C

C-A-B

D-A-B

B-A-D

C-A-D

D-A-C

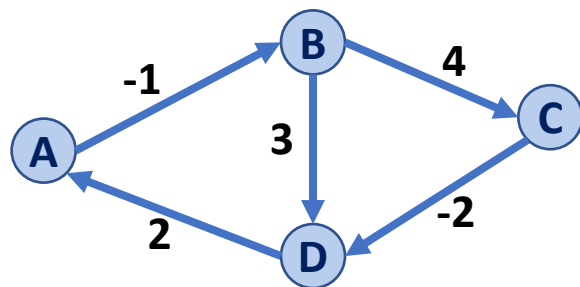
Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```

	A	B	C	D
A	0	-1	∞	∞
B	∞	0	4	3
C	∞	∞	0	-2
D	2	∞	∞	0



Set $w = A$

B-A-C ∞

C-A-B

D-A-B

B-A-D ∞

C-A-D

D-A-C

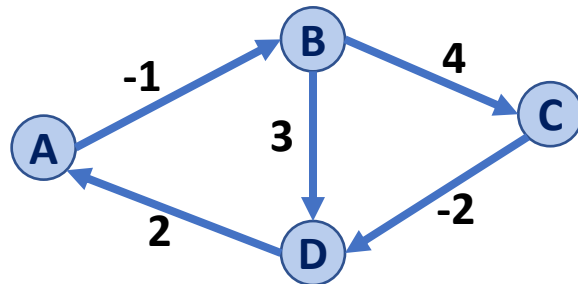
Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```

	A	B	C	D
A	0	-1	∞	∞
B	∞	0	4	3
C	∞	∞	0	-2
D	2	∞	∞	0



Set $w = A$

B-A-C ∞

C-A-B ∞

D-A-B

B-A-D ∞

C-A-D ∞

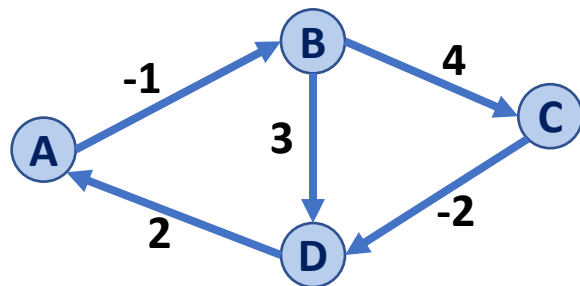
D-A-C

Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	∞	∞
B	∞	0	4	3
C	∞	∞	0	-2
D	2	∞	∞	0

Set $w = A$

B-A-C ∞

C-A-B ∞

D-A-B 1

B-A-D ∞

C-A-D ∞

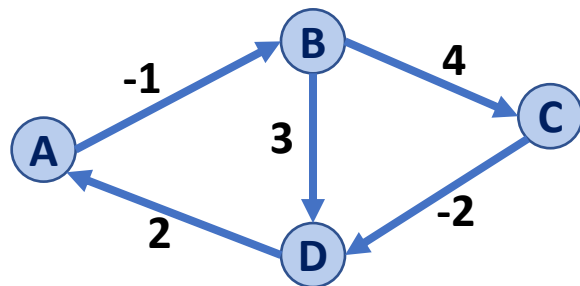
D-A-C

Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	∞	∞
B	∞	0	4	3
C	∞	∞	0	-2
D	2	1	∞	0

Set $w = A$

B-A-C ∞

C-A-B ∞

D-A-B 1

B-A-D ∞

C-A-D ∞

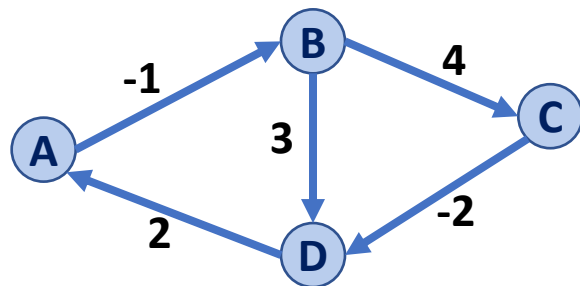
D-A-C

Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	∞	∞
B	∞	0	4	3
C	∞	∞	0	-2
D	2	1	∞	0

Set $w = A$

B-A-C ∞

C-A-B ∞

D-A-B 1

B-A-D ∞

C-A-D ∞

D-A-C ∞

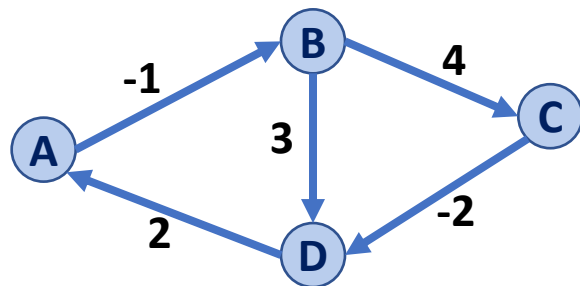
Floyd-Warshall Algorithm

FloydWarshall(G) :

```
6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```

	A	B	C	D
A	0	-1	∞	∞
B	∞	0	4	3
C	∞	∞	0	-2
D	2	1	∞	0

Set $w = B$



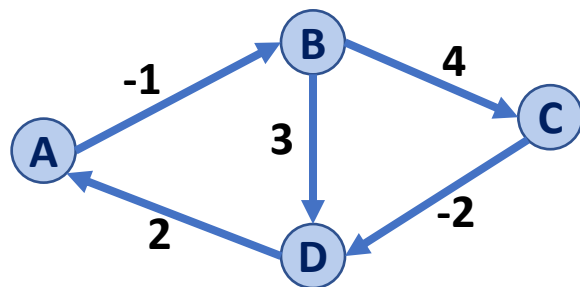
Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G) :
8       d[v, v] = 0
9   foreach (Edge (u, v) : G) :
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G) :
13      foreach (Vertex u : G) :
14          foreach (Vertex v : G) :
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```

	A	B	C	D
A	0	-1	∞	∞
B	∞	0	4	3
C	∞	∞	0	-2
D	2	/	∞	0



Set $w = B$

A-B-C

A-B-D

C-B-A

C-B-D

D-B-A

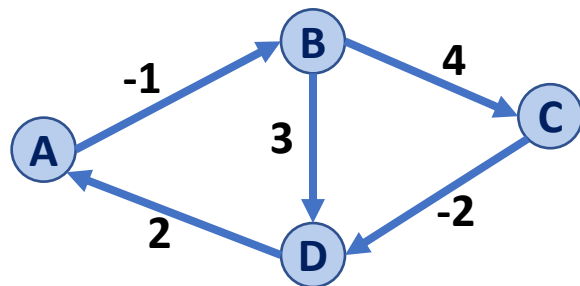
D-B-C

Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	∞	∞
B	∞	0	4	3
C	∞	∞	0	-2
D	2	1	∞	0

Set $w = B$

A-B-C 3

A-B-D

C-B-A

C-B-D

D-B-A

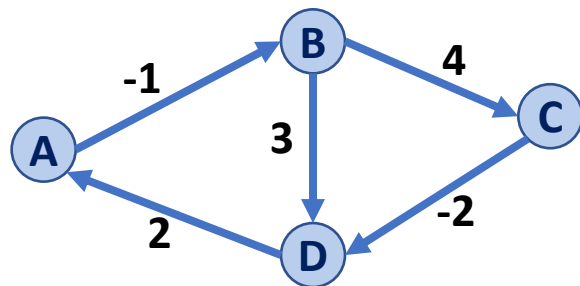
D-B-C

Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	3	∞
B	∞	0	4	3
C	∞	∞	0	-2
D	2	1	∞	0

Set $w = B$

A-B-C 3

A-B-D

C-B-A

C-B-D

D-B-A

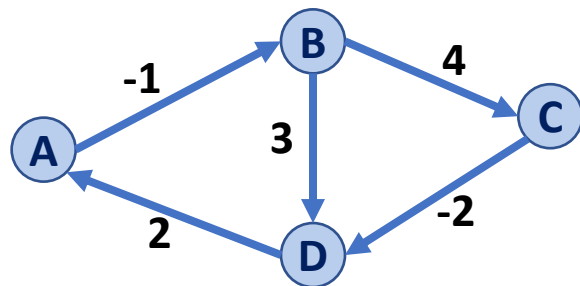
D-B-C

Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	3	∞
B	∞	0	4	3
C	∞	∞	0	-2
D	2	1	∞	0

Set $w = B$

A-B-C 3
A-B-D 2

C-B-A
C-B-D

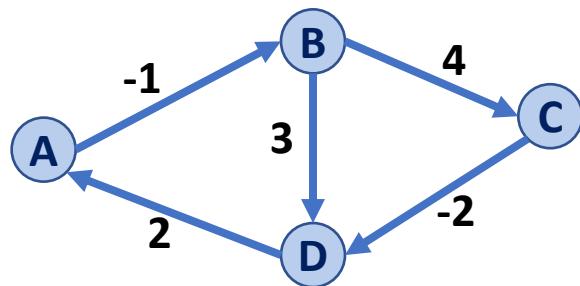
D-B-A
D-B-C

Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	3	2
B	∞	0	4	3
C	∞	∞	0	-2
D	2	1	∞	0

Set $w = B$

A-B-C 3
A-B-D 2

C-B-A

C-B-D

D-B-A

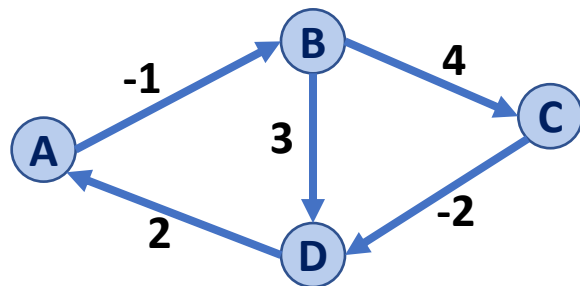
D-B-C

Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	3	2
B	∞	0	4	3
C	∞	∞	0	-2
D	2	1	∞	0

Set $w = B$

A-B-C 3
A-B-D 2

C-B-A ∞

C-B-D ∞

D-B-A

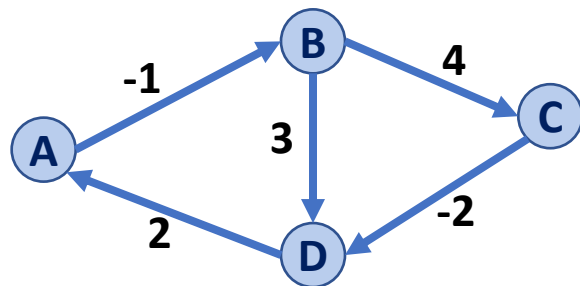
D-B-C

Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	3	2
B	∞	0	4	3
C	∞	∞	0	-2
D	2	1	∞	0

Set $w = B$

$A \xrightarrow{-1} B \xrightarrow{4} C$ 3
 $A \xrightarrow{-1} B \xrightarrow{3} D$ 2

$C \xrightarrow{1} B \xrightarrow{\infty} A$ ∞

$C \xrightarrow{1} B \xrightarrow{\infty} D$ ∞

$D \xrightarrow{1} B \xrightarrow{\infty} A$ ∞

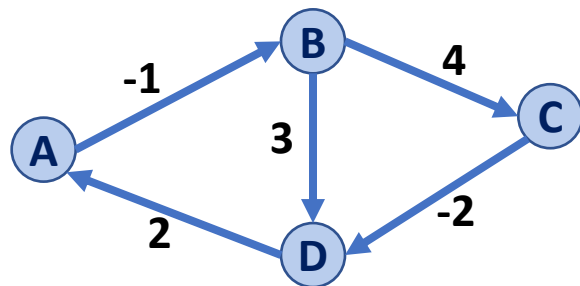
$D \xrightarrow{1} B \xrightarrow{\infty} C$

Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	3	2
B	∞	0	4	3
C	∞	∞	0	-2
D	2	1	∞	0

Set $w = B$

A-B-C 3
A-B-D 2

C-B-A ∞
C-B-D ∞

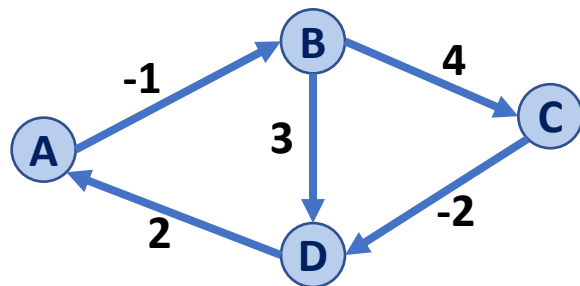
D-B-A ∞
D-B-C 5

Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	3	2
B	∞	0	4	3
C	∞	∞	0	-2
D	2	1	5	0

Set $w = B$

$A \xrightarrow{-1} B \xrightarrow{4} C \quad 3$
 $A \xrightarrow{-1} B \xrightarrow{3} D \quad 2$

$C \xrightarrow{1} B \xrightarrow{\infty} A \quad \infty$
 $C \xrightarrow{1} B \xrightarrow{4} D \quad \infty$

$D \xrightarrow{1} B \xrightarrow{4} A \quad \infty$
 $D \xrightarrow{1} B \xrightarrow{4} C \quad 5$

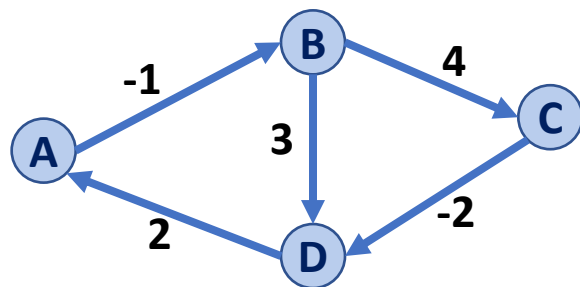
Floyd-Warshall Algorithm

FloydWarshall(G) :

```
6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```

	A	B	C	D
A	0	-1	3	2
B	∞	0	4	3
C	∞	∞	0	-2
D	2	1	5	0

Set $w = C$



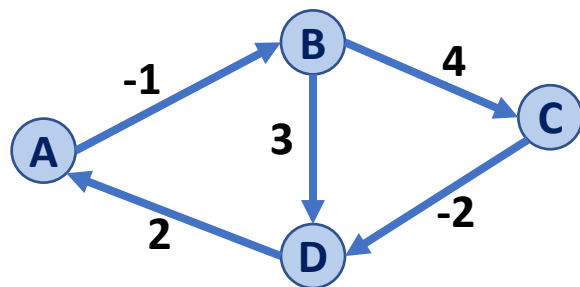
Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```

	A	B	C	D
A	0	-1	3	2
B	∞	0	4	3
C	∞	∞	0	-2
D	2	1	5	0



Set $w = C$

~~A~~ - C - B

B - C - A

~~D~~ - C - A

A - C - D

B - C - D

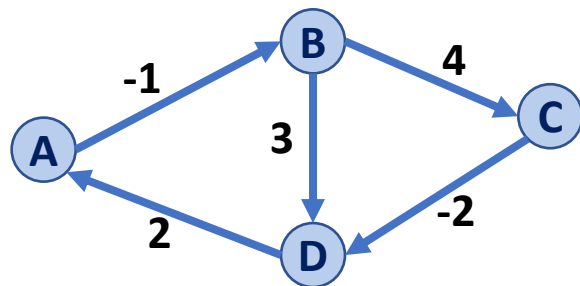
D - C - B

Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	3	2
B	∞	0	4	3
C	∞	∞	0	-2
D	2	1	5	0

Set $w = C$

$A \xrightarrow{3} C \xrightarrow{\infty} B$

$A \xrightarrow{3} C \xrightarrow{-2} D$

$B \rightarrow C \rightarrow A$

$B \rightarrow C \rightarrow D$

$D \rightarrow C \rightarrow A$

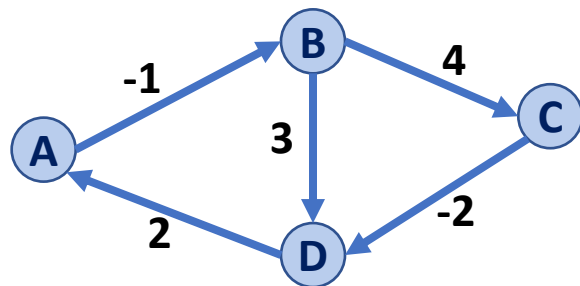
$D \rightarrow C \rightarrow B$

Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	3	1
B	∞	0	4	3
C	∞	∞	0	-2
D	2	1	5	0

Set $w = C$

$A \xrightarrow{3} C \xrightarrow{\infty} B$

$A \xrightarrow{3} C \xrightarrow{-2} D$

$B \rightarrow C \rightarrow A$

$B \rightarrow C \rightarrow D$

$D \rightarrow C \rightarrow A$

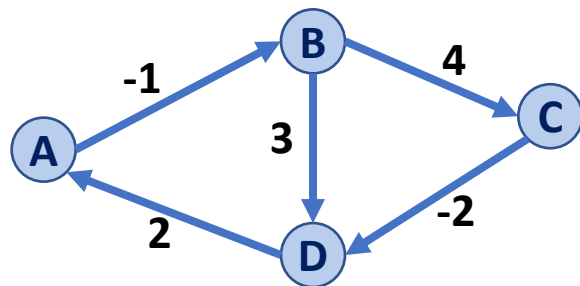
$D \rightarrow C \rightarrow B$

Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	3	1
B	∞	0	4	3
C	∞	∞	0	-2
D	2	1	5	0

Set $w = C$

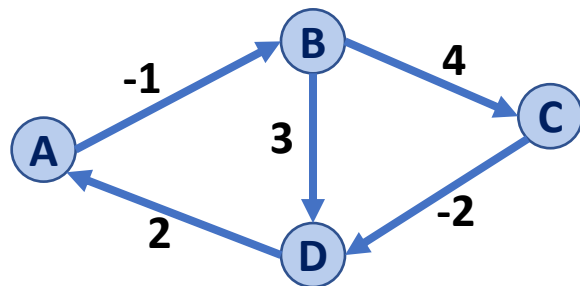
$A \xrightarrow{3} C \xrightarrow{\infty} B \xrightarrow{\infty}$ $B \xrightarrow{4} C \xrightarrow{\infty} A \xrightarrow{\infty}$ $D \xrightarrow{-2} C \xrightarrow{\infty} A \xrightarrow{\infty}$
 $A \xrightarrow{3} C \xrightarrow{-2} D \xrightarrow{1}$ $B \xrightarrow{4} C \xrightarrow{-2} D \xrightarrow{2}$ $D \xrightarrow{-2} C \xrightarrow{-2} B \xrightarrow{2}$

Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	3	1
B	∞	0	4	2
C	∞	∞	0	-2
D	2	1	5	0

Set $w = C$

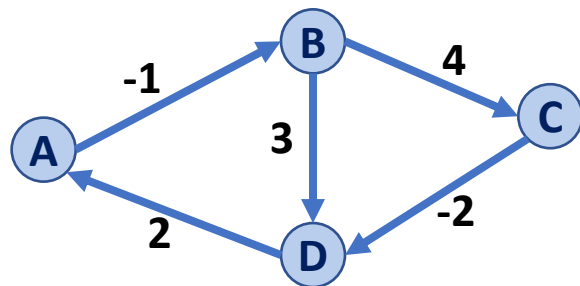
$A \xrightarrow{3} C \xrightarrow{\infty} B \xrightarrow{\infty}$
 $A \xrightarrow{3} C \xrightarrow{-2} D \xrightarrow{1}$
 $B \xrightarrow{4} C \xrightarrow{\infty} A \xrightarrow{\infty}$
 $B \xrightarrow{4} C \xrightarrow{-2} D \xrightarrow{2}$
 $D \xrightarrow{-2} C \xrightarrow{\infty} A \xrightarrow{\infty}$
 $D \xrightarrow{-2} C \xrightarrow{2} B \xrightarrow{2}$

Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	3	1
B	∞	0	4	2
C	∞	∞	0	-2
D	2	1	5	0

Set $w = C$

$A \xrightarrow{3} C \xrightarrow{\infty} B \xrightarrow{\infty}$
 $A \xrightarrow{3} C \xrightarrow{-2} D \xrightarrow{1}$
 $B \xrightarrow{4} C \xrightarrow{\infty} A \xrightarrow{\infty}$
 $B \xrightarrow{4} C \xrightarrow{-2} D \xrightarrow{2}$
 $D \xrightarrow{5} C \xrightarrow{\infty} A \xrightarrow{\infty}$
 $D \xrightarrow{5} C \xrightarrow{\infty} B \xrightarrow{\infty}$

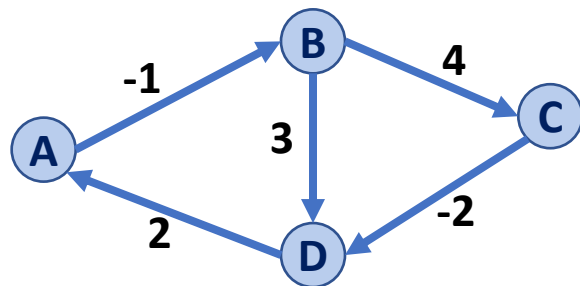
Floyd-Warshall Algorithm

FloydWarshall(G) :

```
6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```

	A	B	C	D
A	0	-1	3	1
B	∞	0	4	2
C	∞	∞	0	-2
D	2	1	5	0

Set $w = D$



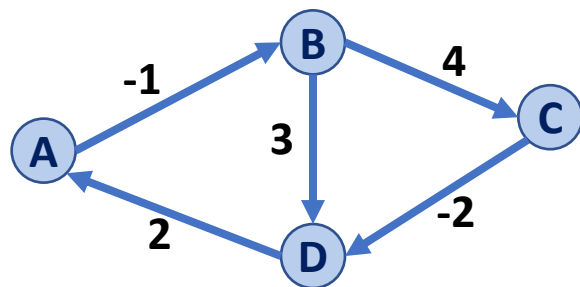
Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```

	A	B	C	D
A	0	-1	3	1
B	∞	0	4	2
C	∞	∞	0	-2
D	2	1	5	0



Set $w = D$

A-D-B

B-D-A

C-D-A

A-D-C

B-D-C

C-D-B

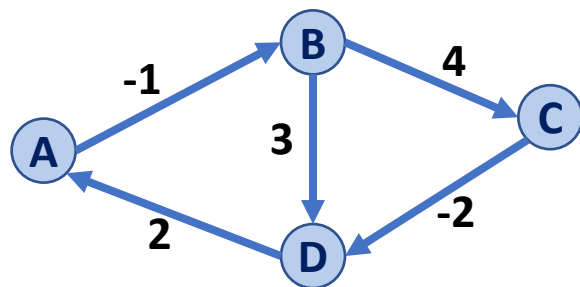
Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```

	A	B	C	D
A	0	-1	3	1
B	∞	0	4	2
C	∞	∞	0	-2
D	2	1	5	0



Set $w = D$

A-D-B 2

B-D-A

C-D-A

A-D-C

B-D-C

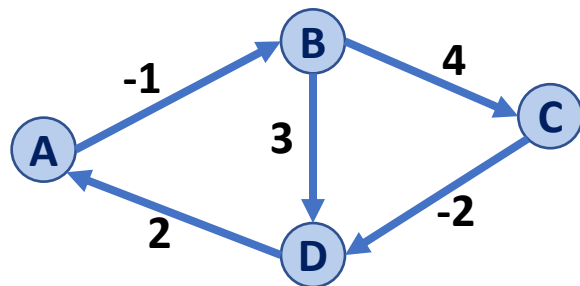
C-D-B

Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	3	1
B	∞	0	4	2
C	∞	∞	0	-2
D	2	1	5	0

Set $w = D$

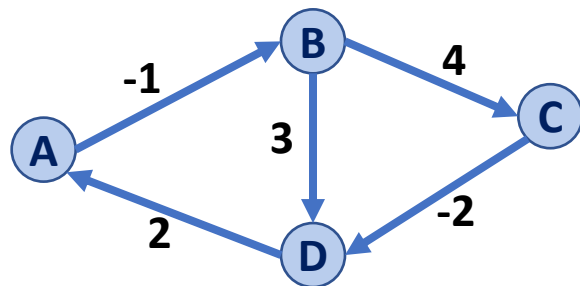
$A \xrightarrow{-1} D \xrightarrow{1} B$ $B \xrightarrow{4} D \xrightarrow{-2} A$ $C \xrightarrow{-2} D \xrightarrow{5} A$
 $A \xrightarrow{2} D \xrightarrow{5} C$ $B \xrightarrow{3} D \xrightarrow{4} C$ $C \xrightarrow{-2} D \xrightarrow{1} B$

Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	3	1
B	∞	0	4	2
C	∞	∞	0	-2
D	2	1	5	0

Set $w = D$

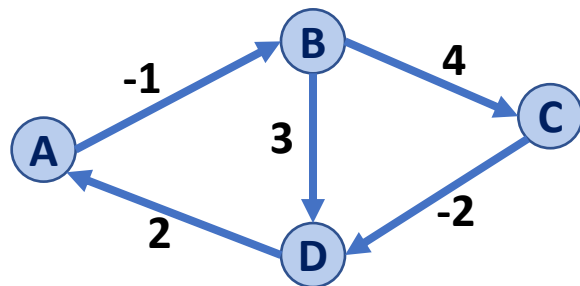
$A \xrightarrow{-1} D \xrightarrow{1} B \quad 2$ $B \xrightarrow{2} D \xrightarrow{-2} A \quad 4$ $C \xrightarrow{-2} D \xrightarrow{5} A$
 $A \xrightarrow{2} D \xrightarrow{5} C \quad 4$ $B \xrightarrow{3} D \xrightarrow{-2} C$ $C \xrightarrow{-2} D \xrightarrow{1} B$

Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	3	1
B	4	0	4	2
C	∞	∞	0	-2
D	2	1	5	0

Set $w = D$

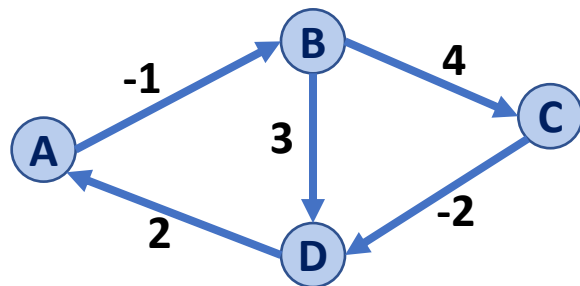
$A \xrightarrow{-1} D \xrightarrow{1} B \quad 2$ $B \xrightarrow{2} D \xrightarrow{-2} A \quad 4$ $C \xrightarrow{-2} D \xrightarrow{5} A$
 $A \xrightarrow{2} D \xrightarrow{5} C \quad 4$ $B \xrightarrow{3} D \xrightarrow{-2} C$ $C \xrightarrow{-2} D \xrightarrow{1} B$

Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	3	1
B	4	0	4	2
C	∞	∞	0	-2
D	2	1	5	0

Set $w = D$

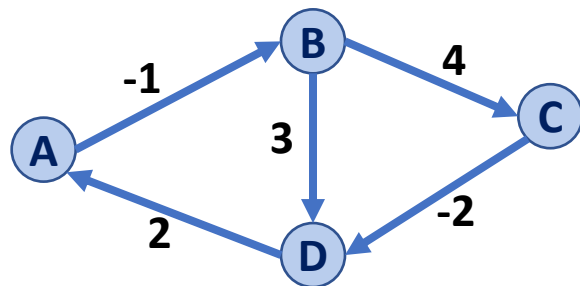
$A \xrightarrow{-1} D \xrightarrow{1} B \quad 2$ $B \xrightarrow{2} D \xrightarrow{-2} A \quad 4$ $C \xrightarrow{-2} D \xrightarrow{5} A$
 $A \xrightarrow{2} D \xrightarrow{5} C \quad 4$ $B \xrightarrow{3} D \xrightarrow{5} C \quad 7$ $C \xrightarrow{-2} D \xrightarrow{1} B$

Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	3	1
B	4	0	4	2
C	∞	∞	0	-2
D	2	1	5	0

Set $w = D$

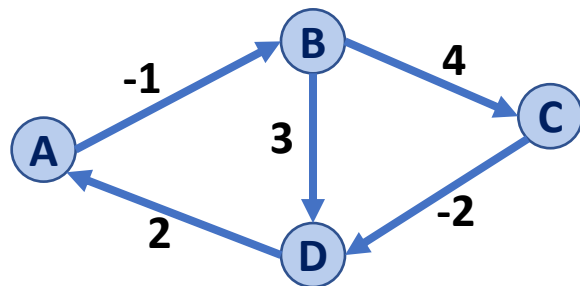
$A \xrightarrow{-1} D \xrightarrow{1} B \quad 2$ $B \xrightarrow{2} D \xrightarrow{-2} A \quad 4$ $C \xrightarrow{2} D \xrightarrow{-2} A \quad 0$
 $A \xrightarrow{2} D \xrightarrow{5} C \quad 4$ $B \xrightarrow{2} D \xrightarrow{5} C \quad 7$ $C \xrightarrow{-2} D \xrightarrow{1} B$

Floyd-Warshall Algorithm

FloydWarshall(G) :

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	3	1
B	4	0	4	2
C	0	∞	0	-2
D	2	1	5	0

Set $w = D$

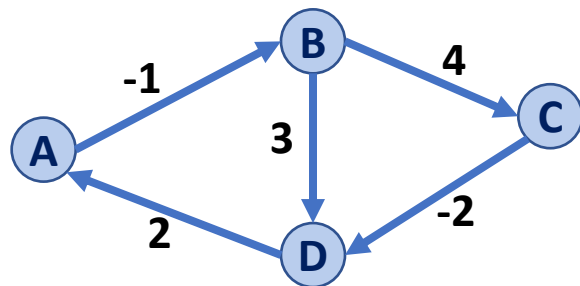
$A \xrightarrow{-1} D \xrightarrow{1} B \quad 2$ $B \xrightarrow{2} D \xrightarrow{-2} A \quad 4$ $C \xrightarrow{2} D \xrightarrow{-2} A \quad 0$
 $A \xrightarrow{2} D \xrightarrow{5} C \quad 4$ $B \xrightarrow{2} D \xrightarrow{5} C \quad 7$ $C \xrightarrow{-2} D \xrightarrow{0} B$

Floyd-Warshall Algorithm

FloydWarshall(G):

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	3	1
B	4	0	4	2
C	0	∞	0	-2
D	2	1	5	0

Set $w = D$

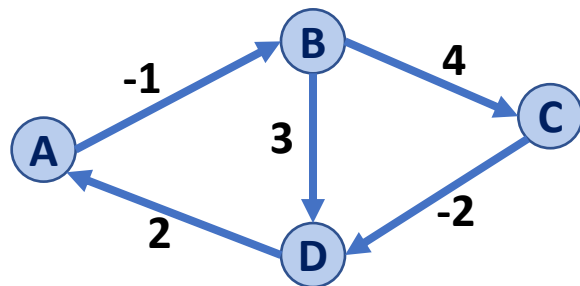
$A \xrightarrow{-1} D \xrightarrow{1} B \quad 2$ $B \xrightarrow{2} D \xrightarrow{-2} A \quad 4$ $C \xrightarrow{2} D \xrightarrow{-2} A \quad 0$
 $A \xrightarrow{1} D \xrightarrow{5} C \quad 4$ $B \xrightarrow{2} D \xrightarrow{5} C \quad 7$ $C \xrightarrow{-2} D \xrightarrow{1} B \quad -1$

Floyd-Warshall Algorithm

FloydWarshall(G):

```

6   Let d be a adj. matrix initialized to +inf
7   foreach (Vertex v : G):
8       d[v, v] = 0
9   foreach (Edge (u, v) : G):
10      d[u, v] = cost(u, v)
11
12  foreach (Vertex w : G):
13      foreach (Vertex u : G):
14          foreach (Vertex v : G):
15              if d[u, v] > d[u, w] + d[w, v]:
16                  d[u, v] = d[u, w] + d[w, v]
```



	A	B	C	D
A	0	-1	3	1
B	4	0	4	2
C	0	-1	0	-2
D	2	1	5	0

Set $w = D$

$A \xrightarrow{-1} D \xrightarrow{1} B \quad 2$ $B \xrightarrow{2} D \xrightarrow{-2} A \quad 4$ $C \xrightarrow{2} D \xrightarrow{-2} A \quad 0$
 $A \xrightarrow{2} D \xrightarrow{5} C \quad 4$ $B \xrightarrow{2} D \xrightarrow{5} C \quad 7$ $C \xrightarrow{-2} D \xrightarrow{1} B \quad -1$

Shortest Path Algorithms Runtime:

- Dijkstra's Algorithm:
 $O(m + n \lg(n))$

- Floyd-Warshall:
 $O(n^3)$

All Pairs Shortest Path:

Dense Graphs:

Sparse Graphs:

Shortest Path Algorithms Runtime:

- Dijkstra's Algorithm:
 $O(m + n \lg(n))$

- Floyd-Warshall:
 $O(n^3)$

All Pairs Shortest Path:

$$n \times (m + n \lg n) = mn + n^2 \lg n$$

Dense Graphs:

Sparse Graphs:

Shortest Path Algorithms Runtime:

- Dijkstra's Algorithm:
 $O(m + n \lg(n))$

- Floyd-Warshall:
 $O(n^3)$

All Pairs Shortest Path:

$$n \times (m + n \lg n) = mn + n^2 \lg n$$

Dense Graphs:

$$m \sim n^2$$

Sparse Graphs:

Shortest Path Algorithms Runtime:

- Dijkstra's Algorithm:
 $O(m + n \lg(n))$

- Floyd-Warshall:
 $O(n^3)$

All Pairs Shortest Path:

$$n \times (m + n \lg n) = mn + n^2 \lg n$$

Dense Graphs:

$$m \sim n^2 \rightarrow n^3 + n^2 \lg n$$

Sparse Graphs:

Shortest Path Algorithms Runtime:

- Dijkstra's Algorithm:
 $O(m + n \lg(n))$

- Floyd-Warshall:
 $O(n^3)$

All Pairs Shortest Path:

$$n \times (m + n \lg n) = mn + n^2 \lg n$$

Dense Graphs:

$$m \sim n^2 \rightarrow n^3 + n^2 \lg n$$

Sparse Graphs:

$$m \sim n$$

Shortest Path Algorithms Runtime:

- Dijkstra's Algorithm:
 $O(m + n \lg(n))$

- Floyd-Warshall:
 $O(n^3)$

All Pairs Shortest Path:

$$n \times (m + n \lg n) = mn + n^2 \lg n$$

Dense Graphs:

$$m \sim n^2 \rightarrow n^3 + n^2 \lg n$$

Sparse Graphs:

$$m \sim n \rightarrow n^2 + n^2 \lg n$$

Graphs

Graph Implementations:

- Edge List
- Adjacency Matrix
- Adjacency List

Graph Traversals:

- Breadth First
- Depth First

Minimum Spanning Trees:

- Kruskal's Algorithm
- Prim's Algorithm

Shortest Path:

- Dijkstra's Algorithm
- Floyd-Warshall's Algorithm



CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

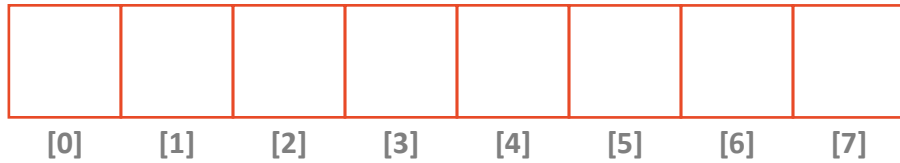
In Review: Data Structures

Array

- Sorted Array
- Unsorted Array
- Stacks
- Queues
- Hashing
- Heaps
 - Priority Queues
- UpTrees
 - Disjoint Sets

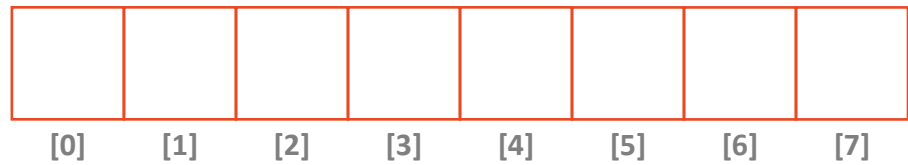
List

- Doubly Linked List
- Skip List
- Trees
 - BTree
 - Binary Tree
 - Huffman Encoding
 - kd-Tree
 - AVL Tree

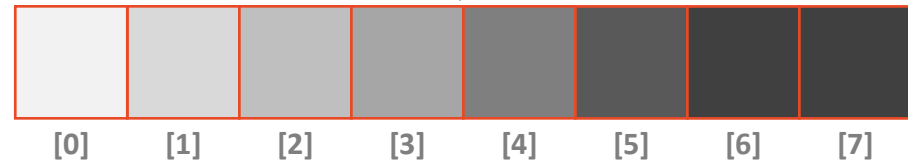


Array

-  Constant time access to any element, given an index $a[k]$ is accessed in $O(1)$ time, no matter how large the array grows
- Cache-optimized
Many modern systems cache or pre-fetch nearby memory values due the “Principle of Locality”. Therefore, arrays often perform faster than lists in identical operations.

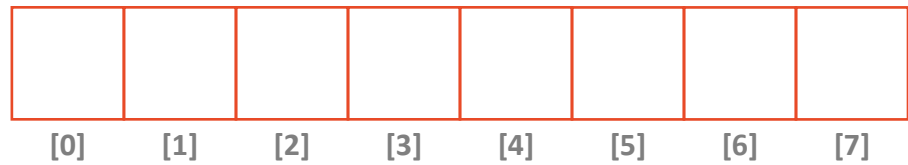


Array

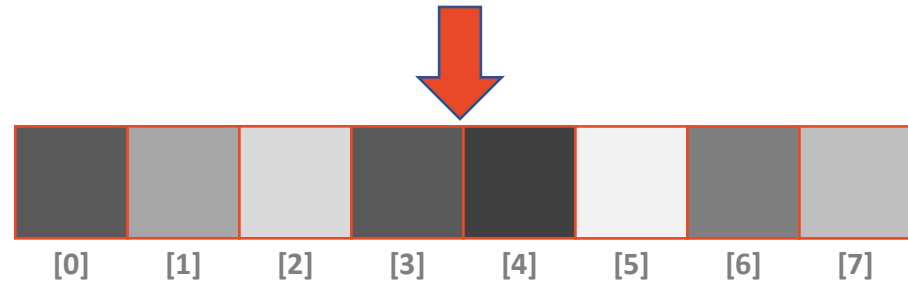


Sorted Array

- Efficient general search structure
Searches on the sort property run in $O(\log(n))$ with Binary Search
- Inefficient insert/remove
Elements must be inserted and removed at the location dictated by the sort property, resulting shifting the array in memory – an $O(n)$ operation

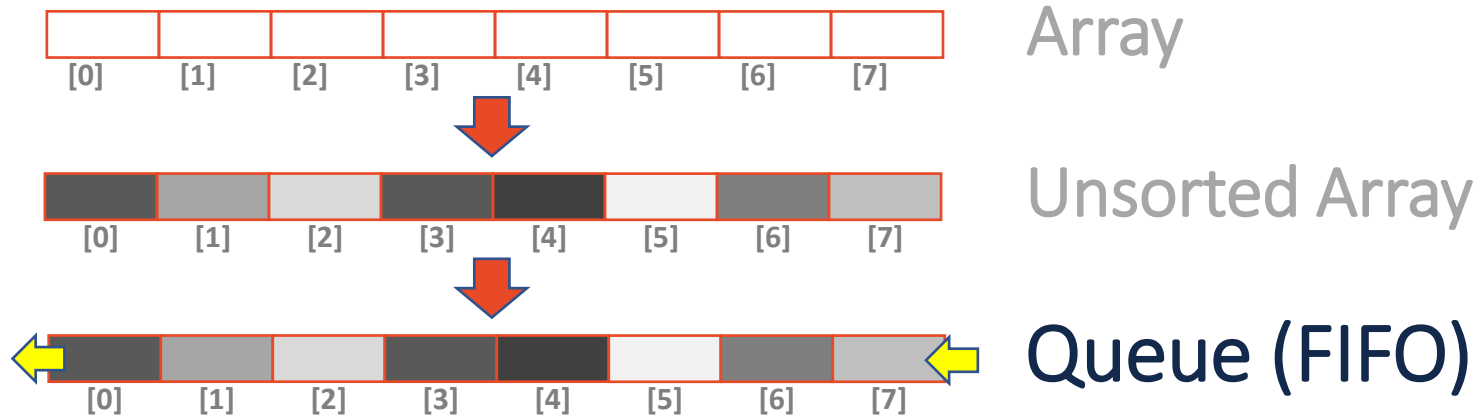


Array



Unsorted Array

- Constant time add/remove at the beginning/end
Amortized $O(1)$ insert and remove from the front and of the array
Idea: Double on resize
- Inefficient search structure
With no sort property, all searches must iterate the entire array; $O(n)$ time



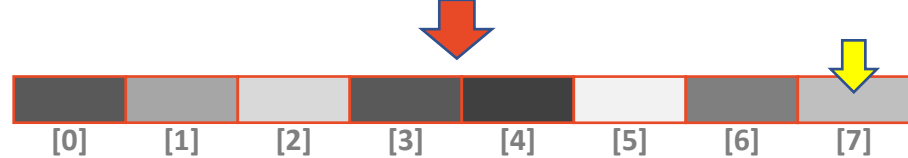
- First In First Out (FIFO) ordering of data
Maintains an arrival ordering of tasks, jobs, or data
- All ADT operations are constant time operations
enqueue() and dequeue() both run in $O(1)$ time



Array



Unsorted Array

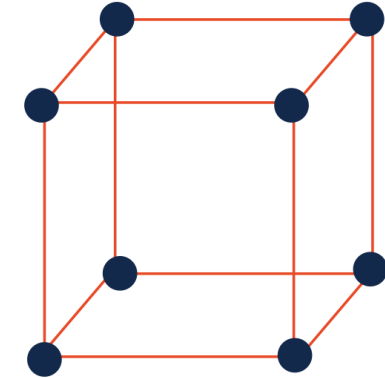


Stack (LIFO)

- Last In First Out (LIFO) ordering of data
Maintains a “most recently added” list of data
- All ADT operations are constant time operations
`push()` and `pop()` both run in $O(1)$ time

Iterators

Iterators give client code access to traverse the data!



Operators:

operator++
operator==
operator!=
operator=
operator*

Types of iterators:

Forward
Backward
Bidirectional

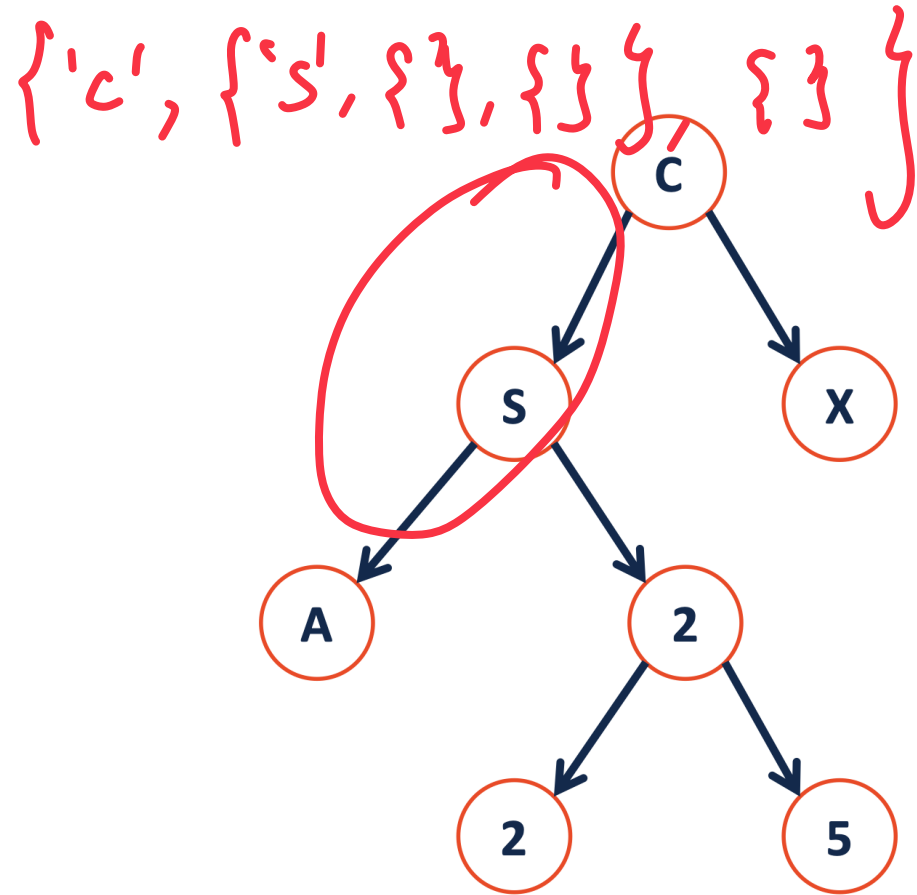
Binary Tree – Defined

A binary tree T is either:

- $T = \{\}$

OR

- $T = \{r, T_L, T_R\}$

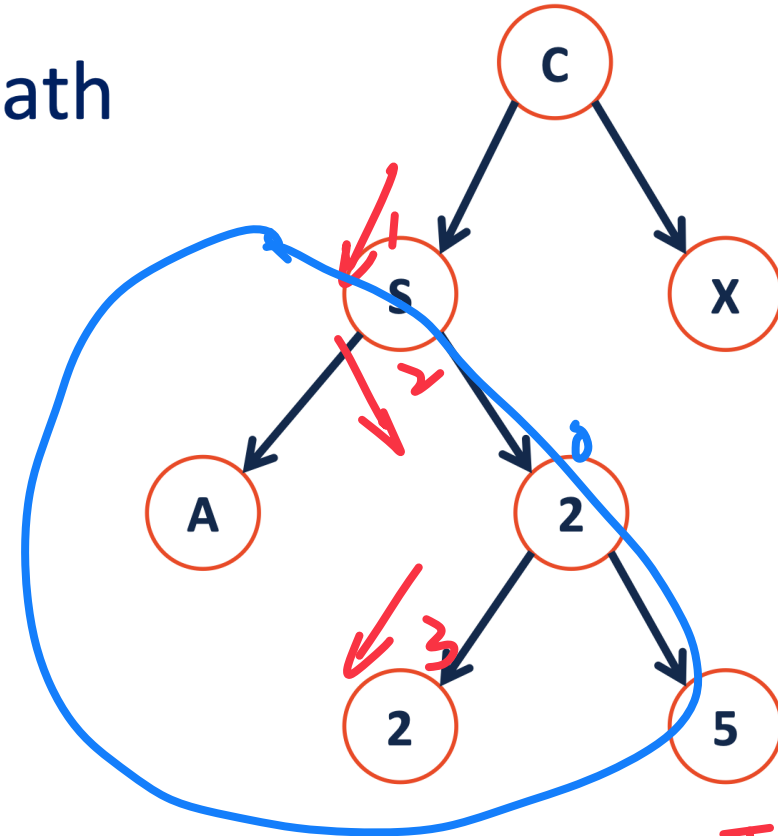


Tree Property: height

height(T): length of the longest path from the root to a leaf

↳ Count Edges

Given a binary tree T:



$$\text{height}(T) = \begin{cases} \text{if } \{r, T_L, T_R\}, \max(\text{height}(T_L), \text{height}(T_R)) + 1 \\ \text{if } \{\}, -1 \end{cases}$$

$T = 2 + 1$

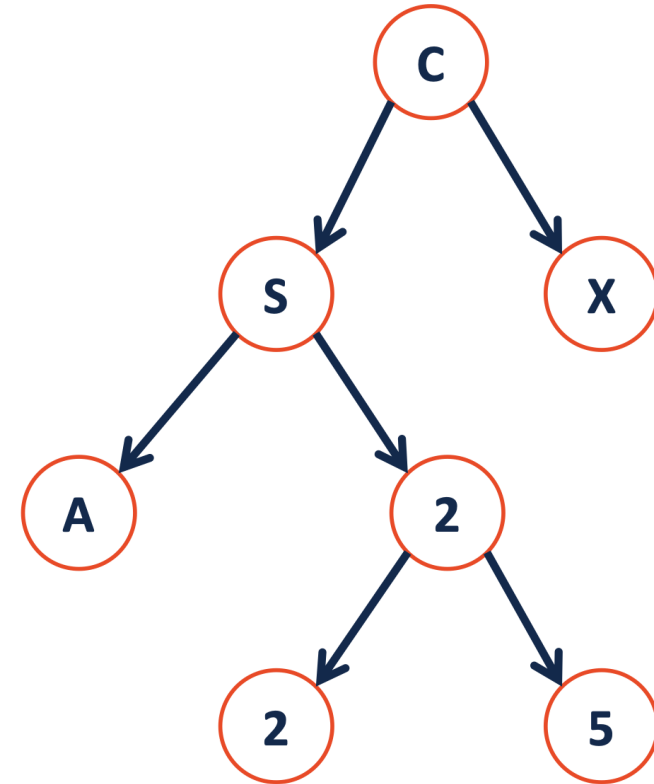
Tree Property: full

A tree F is **full** if and only if:

1. $F = \{\}$

OR

2. $F = \{r, T_L, T_R\}$ where T_L and T_R are both empty or both not empty



Tree Property: perfect

A perfect tree P is:

$$P_h = 2^{h+1} - 1$$

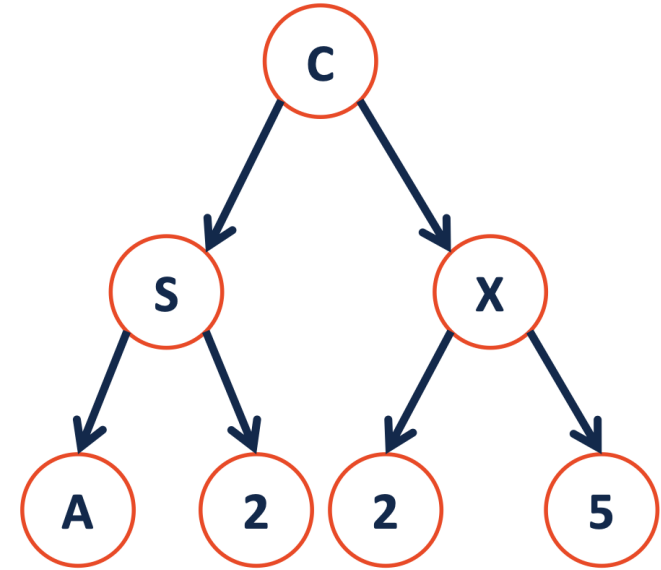
1. $P = \{\}$

OR

2. $P = \{r, T_L, T_R\}$ where T_L and T_R are of height T_{h-1}

$$P_0: \{ 'c', \{\}, \{\} \}$$

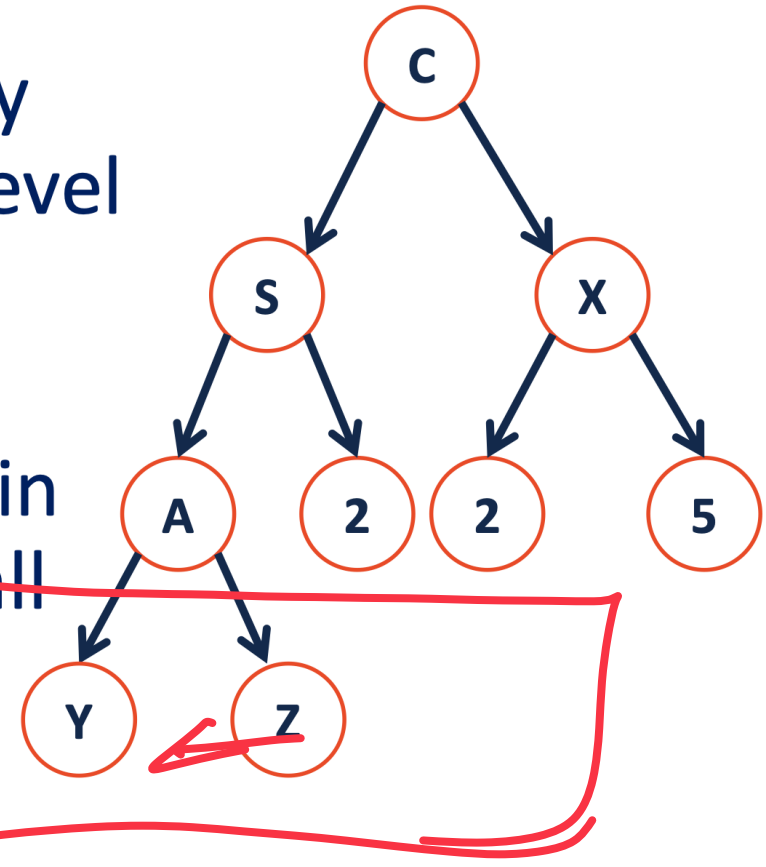
$$P_1: \{ 'c', \{ 's', \{\}, \{\} \}, \{ 'x', \{\}, \{\} \} \}$$



Tree Property: complete

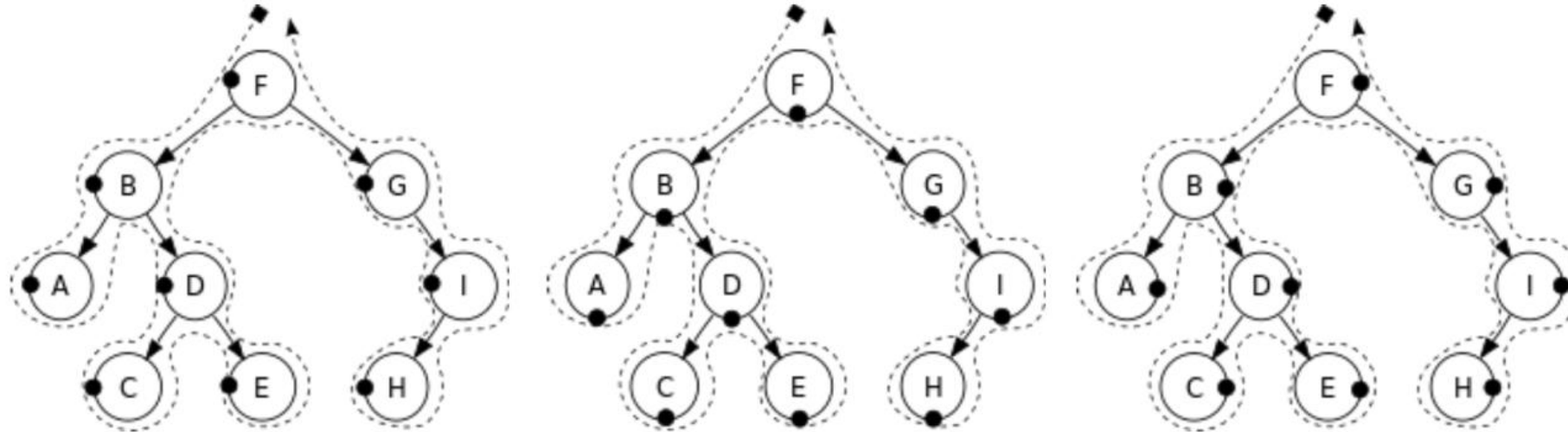
Conceptually: A perfect tree for every level except the last, where the last level is “pushed to the left”.

Slightly more formal: For any level k in $[0, h-1]$, k has 2^k nodes. For level h , all nodes are “pushed to the left”.

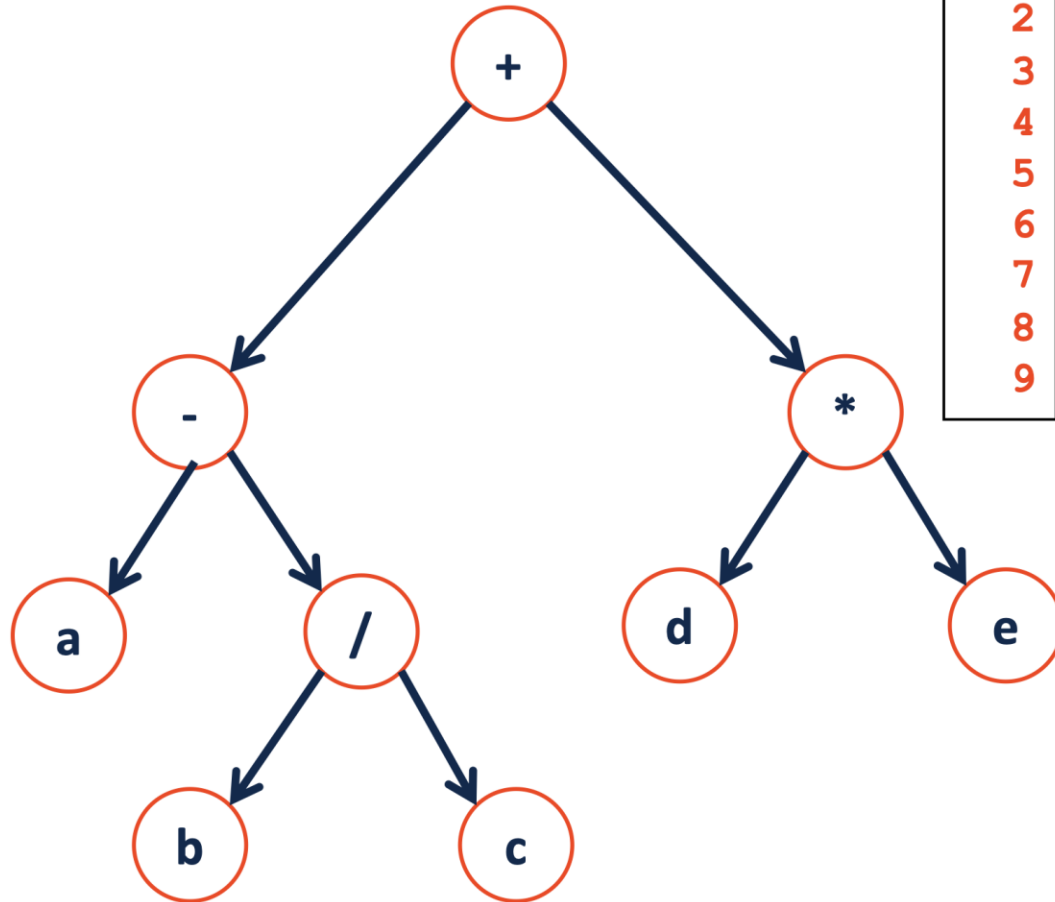


Tree traversals

- Depth-first: pre-order, in-order, post-order



Traversals

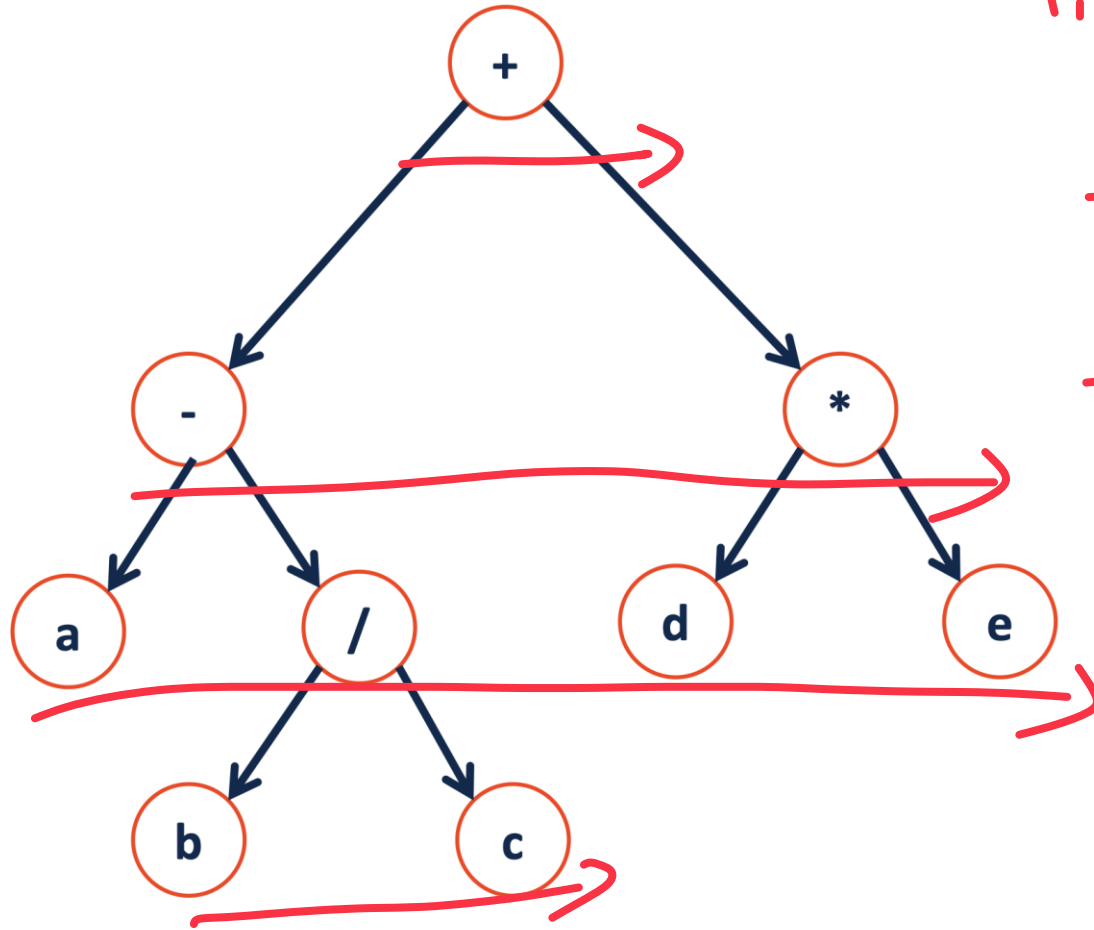


```
1 template<class T>
2 void BinaryTree<T>::preOrder(TreeNode * root)
3 {
4     if (root != NULL) {
5         yell( root->data );
6         preOrder(root->left);
7         preOrder(root->right);
8     }
9 }
```

pre : Data , L , R
In : L , Self , R
Post : L , R , Self

A Different Type of Traversal $\rightarrow$ level-by-level

$+, -, *, a, /, d, e$ How??



- put the first node to a queue

- while the queue is not empty:

① remove element from queue

② if not null:

- print value

- enqueue its children

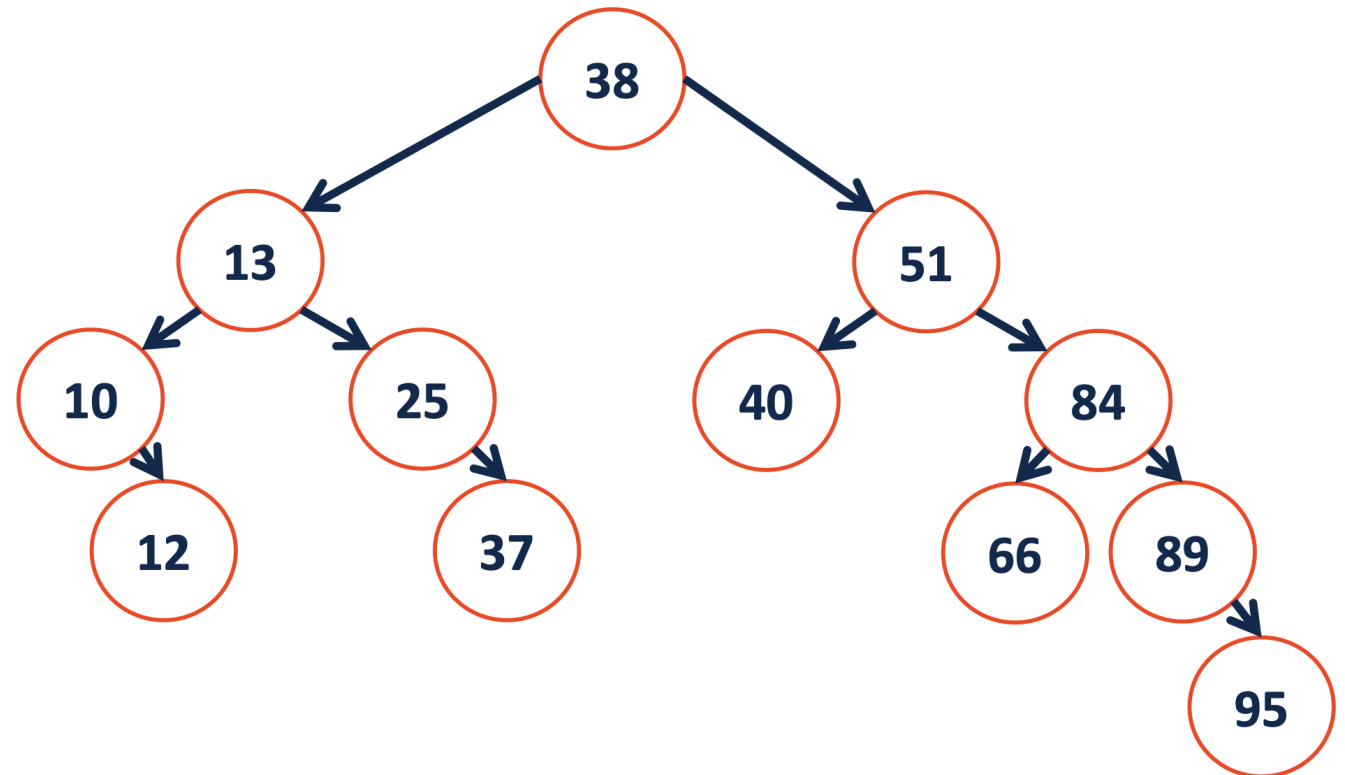
Recursion?? $\rightarrow$ No

Binary Search Tree (BST)

A **BST** is a binary tree **T** such that:

find(x)

$T = \{ \}$ or
 $T = \{ r, T_L, T_R \}$ and
- x in T_L if $x < r$
- x in T_R if $x > r$



```
1 template<class K, class V>
```

```
2 TreeNode * & BSF:: find(TreeNode *& root, const K & key) const {
```

```
3  
4 // Base case root == Null
```

```
5  
6 if (root == Null)  
7     return root ;
```

```
8  
9  
10 // Equal to
```

```
11 if (key == root->key)  
12     return root ;
```

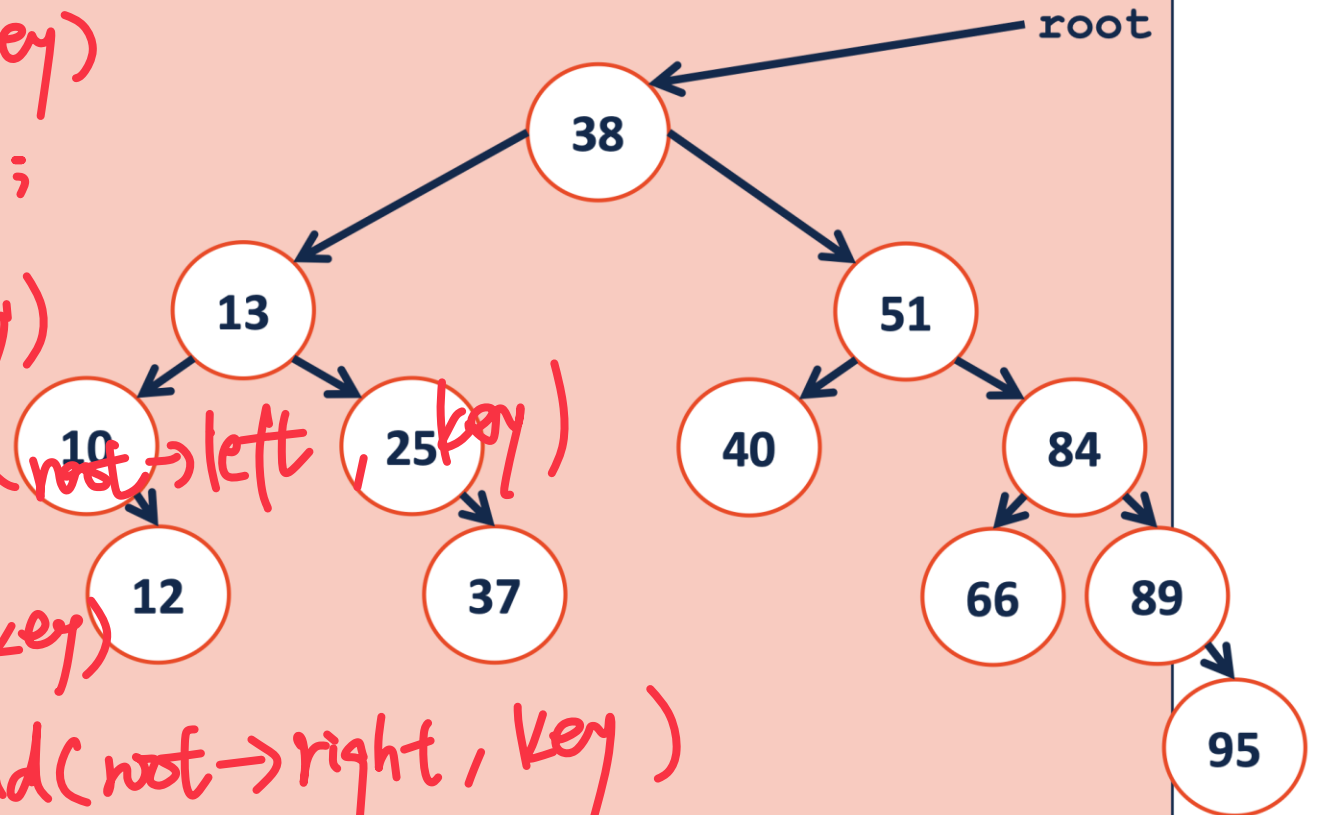
```
13  
14  
15 // Less than
```

```
16 if (key < root->key)  
17  
18     return -find(root->left, key)
```

```
19  
20  
21 // Equal than
```

```
22 if (key > root->key)  
23  
24     return -find(root->right, key)
```

```
25  
26 }
```



```
1 template<class K, class V>
```

```
2 void :: BST insert(TreeNode *& root, K key, V value) {
```

```
3
```

```
4
```

```
5
```

```
6
```

```
7
```

```
8
```

```
9
```

```
10
```

```
11
```

```
12
```

```
13
```

```
14
```

```
15
```

```
16
```

```
17
```

```
18
```

```
19
```

```
20
```

```
21
```

```
22
```

```
23
```

```
24
```

```
25
```

```
26 }
```

~~void :: BST~~

insert(TreeNode *& root, K key, V value) {

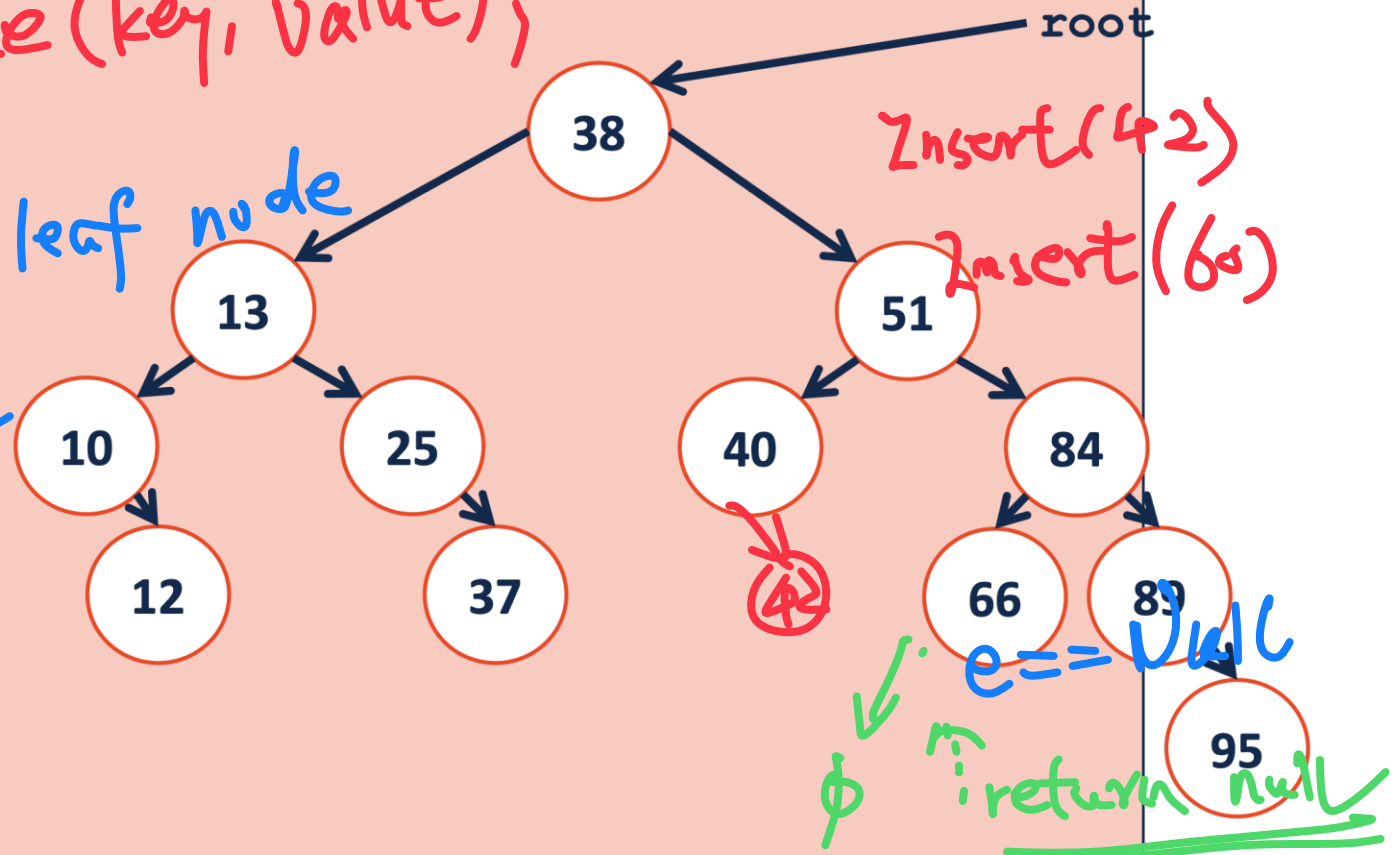
TreeNode *&e = find-(root, key);

// check if e is null

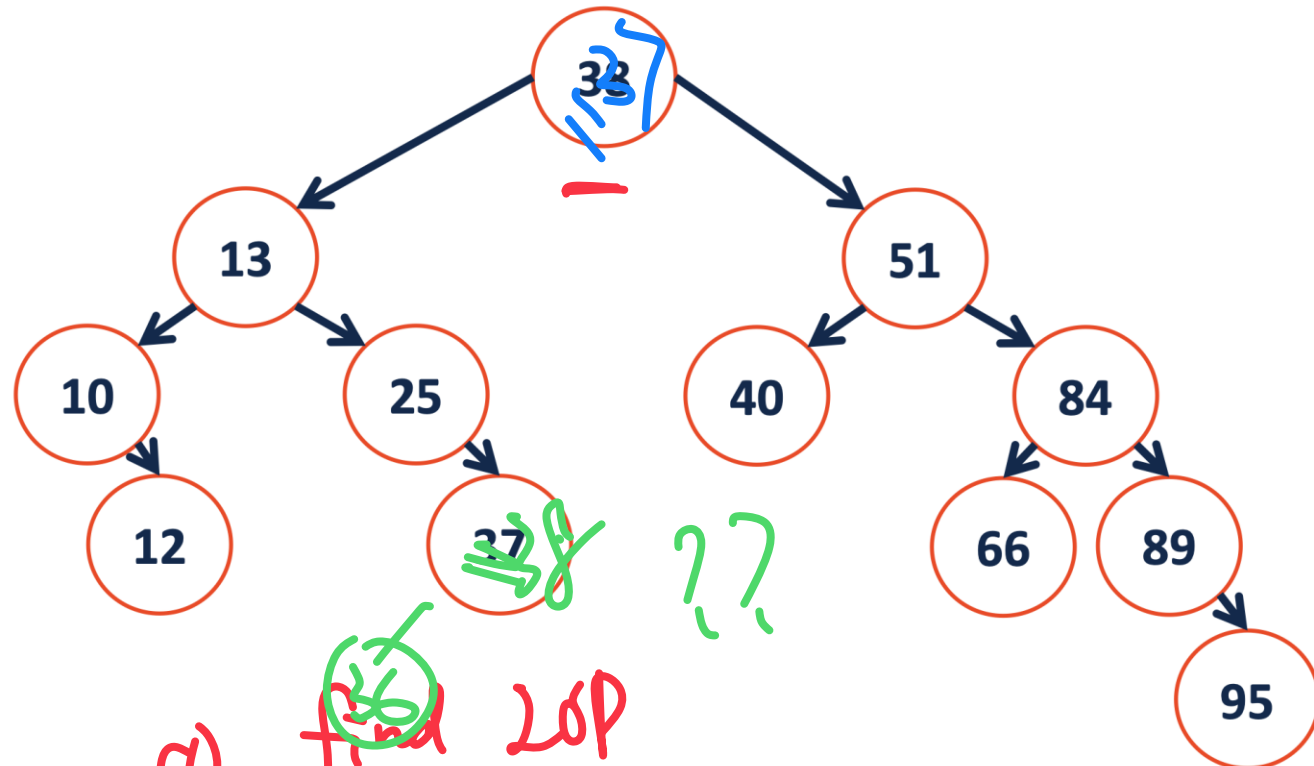
e = new TreeNode(key, value);

- Insert will always at leaf node

- Insert might increase
tree height



remove(38)



① find LOP

② Swap current removal with LOP

③ remove again → 0/1 child

```
1 template<class K, class V>
```

```
2         remove(TreeNode *& root, const K & key) {
```

```
3  
4 // Case 1: 0 child
```

```
5  
6  
7 // Case 2: 1 child
```

```
8  
9  
10  
11  
12 // Case 3: 2 children
```

```
13  
14 ① Find LOP
```

```
15  
16 ② Swap
```

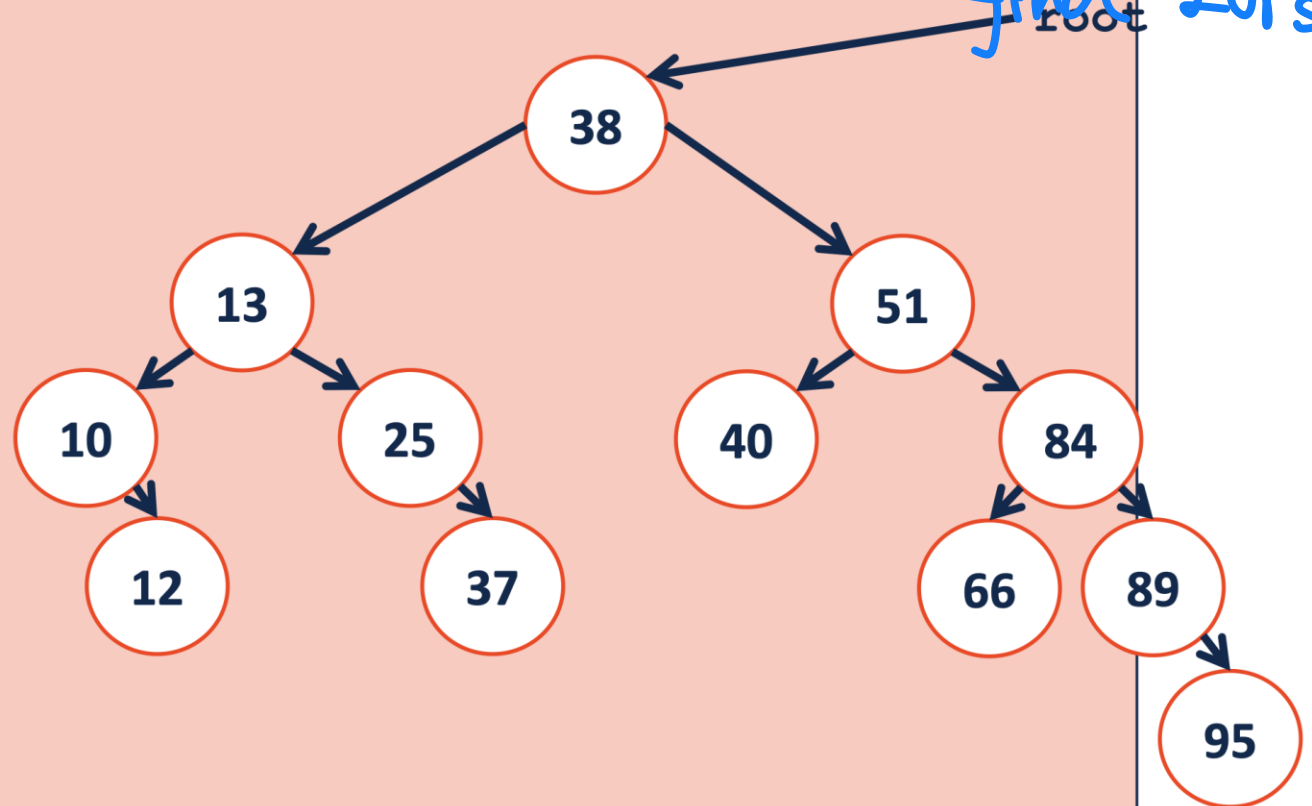
```
17  
18  
19  
20  
21 ③ _remove again
```

```
22  
23  
24  
25  
26 }
```

- rearrange with LOPs, : Two children

running time :

$O(h)$ $\Rightarrow$ find node/
find LOPs



BST Analysis – Running Time

Operation	BST Average Case	BST Worst Case	Sorted Array	Sorted List
find	$O(\lg n)$	$O(n)$	$O(\lg n)$	$O(n)$
insert	$O(\lg n)$	$O(n)$	$O(n)$	Given location: $O(1)$ Arbitrary: $O(n)$
delete	$O(\lg n)$	$O(n)$	$O(n)$	↓ the same.
traverse	$O(n)$			

BST Reflection

We know the **height** of a tree.

We know if a tree is **full**, **complete**, and/or **perfect**.

We know that every binary tree has $n+1$ NULL pointers.

We know many **traversals** of trees.

We know that a **BST's height is bound by n** such that:

$$\underline{\lg(n)} \leq h \leq \underline{n}$$

We know all key **BST operations run in $O(h)$** time.

We know a BST can be used to **implement a Dictionary**.

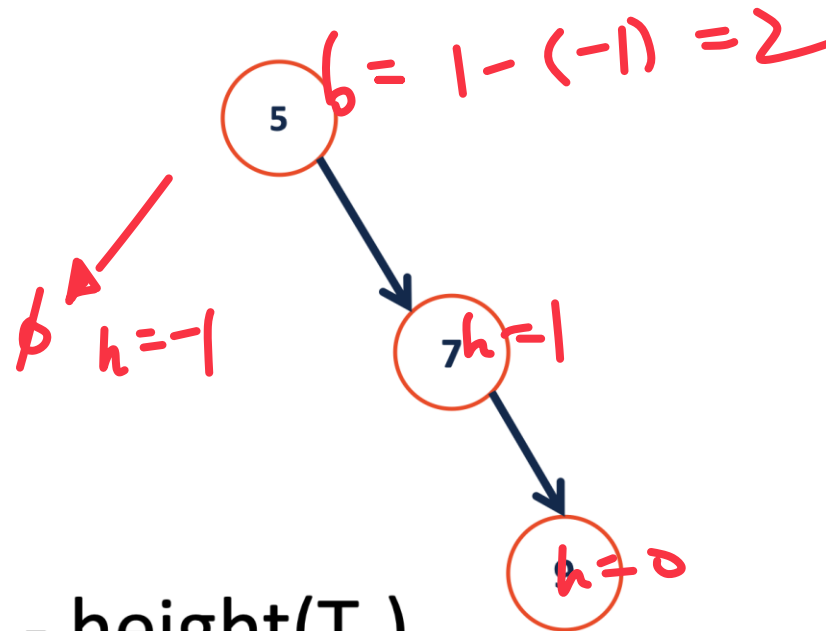
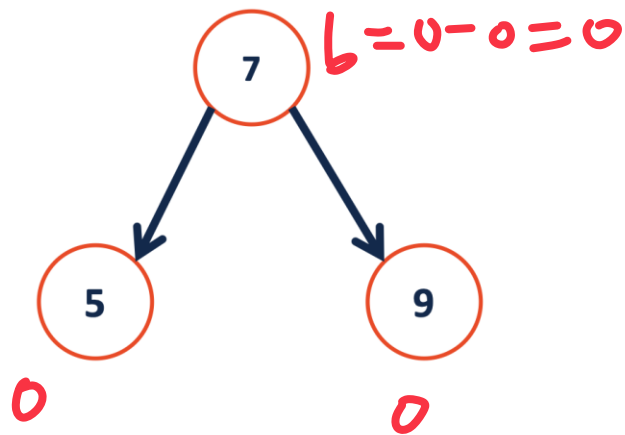
We know that a **random BST** has an average height of $\lg(n)$.

We know that an **inorder traversal** of a BST is a Ordered List.

We know **how to implement a BST** in C++.

Height-Balanced Tree

What tree makes you happier?



Height balance: $b = \text{height}(T_R) - \text{height}(T_L)$

A tree is height balanced if:

$T = \{\}$ or $T = \{r, T_R, T_L\}$

and $|b| \leq 1$

BST Rotation Summary

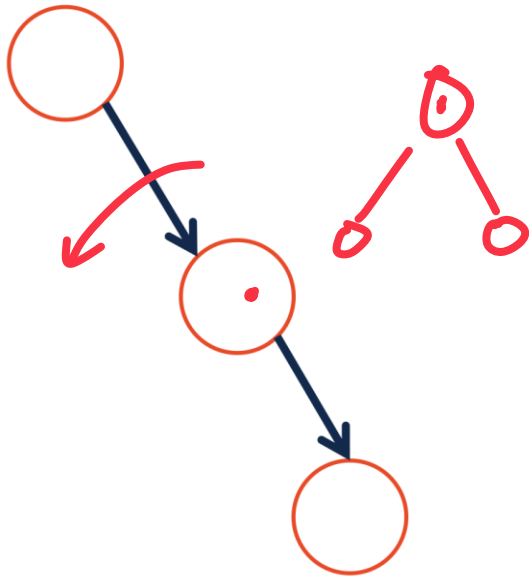
- Four kinds of rotations (L, R, LR, RL)
- All rotations are local (subtrees are not impacted)
- All rotations are constant time: $O(1)$
- BST property maintained

GOAL: Create a data structure with
height balanced BSTs

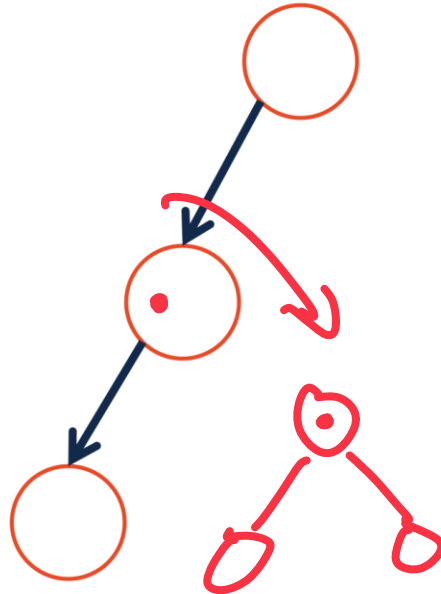
We call these trees:
Adelson-Velskii and Landis $\Rightarrow$ AVL Trees
1962

AVL Tree Rotations

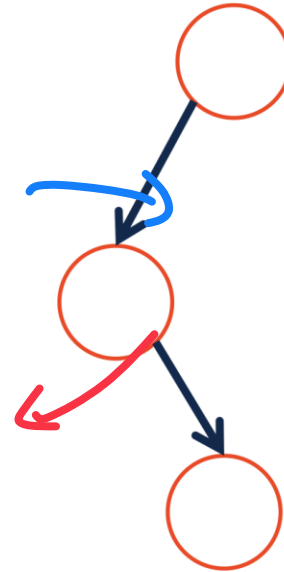
Four templates for rotations:



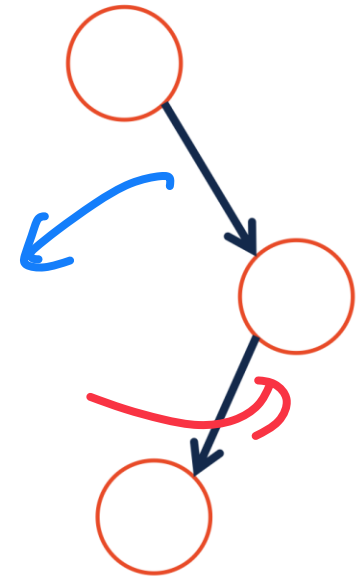
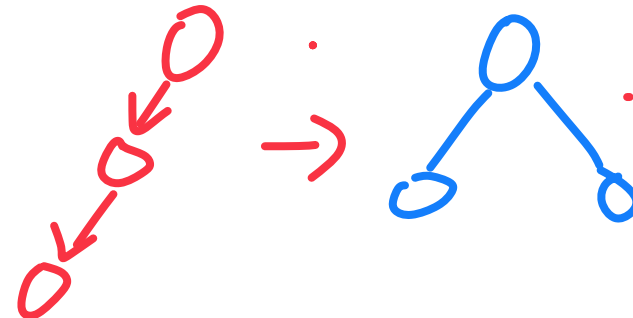
L



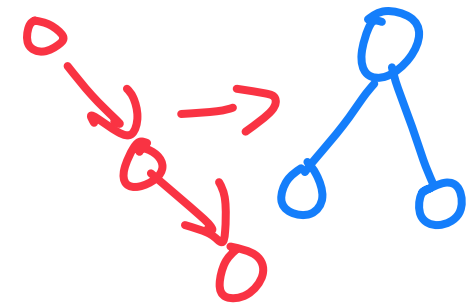
R



L R



R L



```

1  template <class T> void AVLTree<T>::insert_(const T & x, treeNode<T> * & t ) {
2      if( t == NULL ) {
3          t = new TreeNode<T>( x, 0, NULL, NULL);
4      }
5      base case
6      else if( x < t->key ) {
7          insert_( x, t->left );
8          int balance = height(t->right) - height(t->left);
9          int leftBalance = height(t->left->right) - height(t->left->left);
10         if ( balance == -2 ) {
11             if ( leftBalance == -1 ) { rotate Right ( t ); }
12             else { rotate Left Right ( t ); }
13         }
14     }
15
16     else if( x > t->key ) {
17         insert_( x, t->right );
18         int balance = height(t->right) - height(t->left);
19         int rightBalance = height(t->right->right) - height(t->right->left);
20         if( balance == 2 ) {
21             if( rightBalance == 1 ) { rotate Left ( t ); }
22             else { rotate Right Left ( t ); }
23         }
24     }
25
26     t->height = 1 + max(height(t->left), height(t->right));
27 }

```

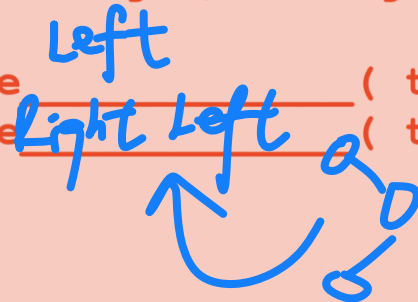
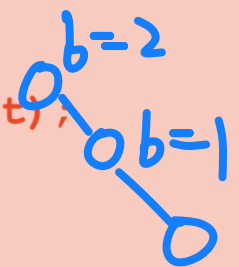
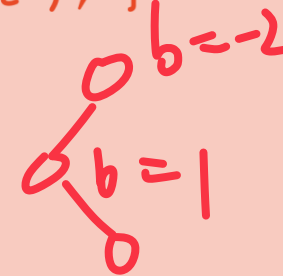
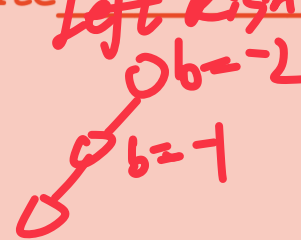
key

current node

insertion

check imbalance

rotation



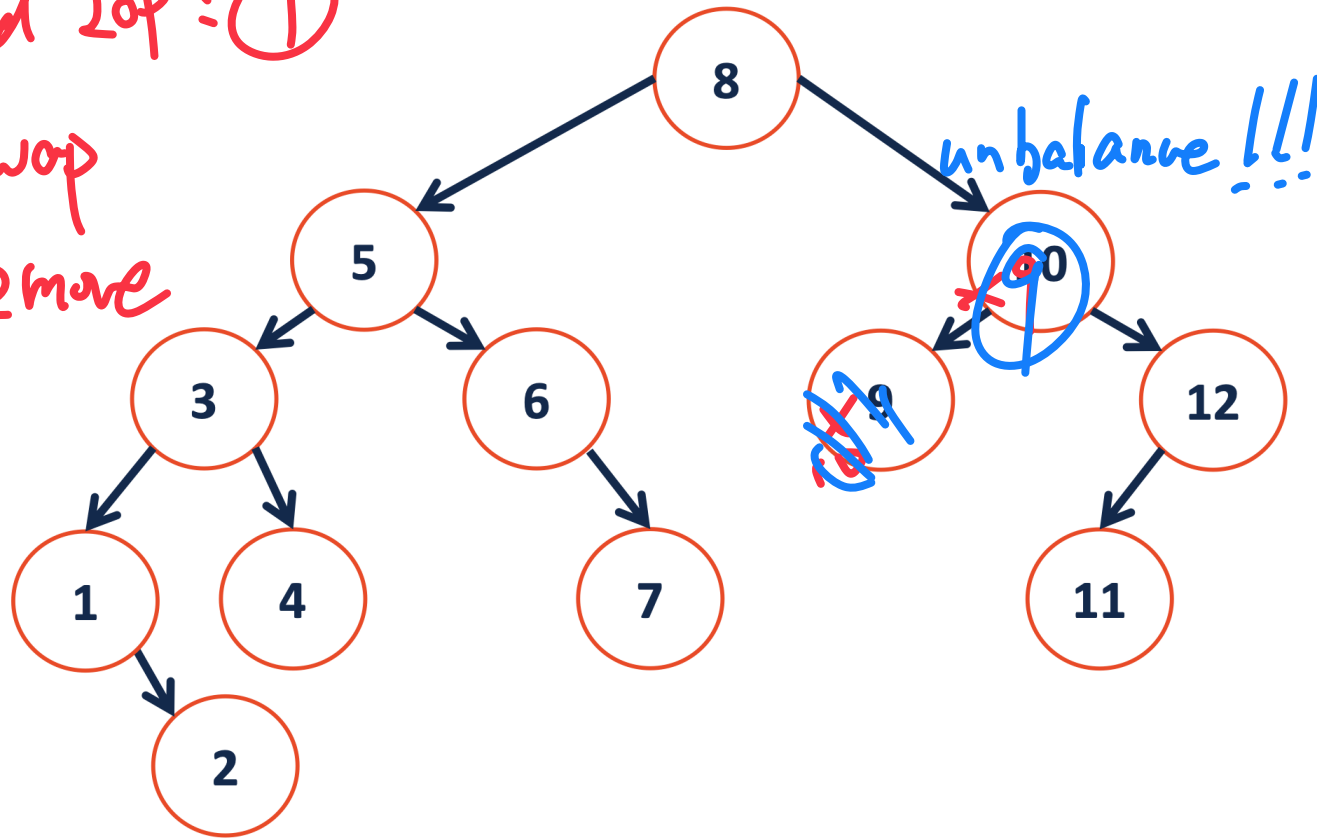
update height

Remove 10

① Find IOP: 9

② Swap

③ Remove



Rotate

Every Data Structure So Far

	Unsorted Array	Sorted Array	Unsorted List	Sorted List	Binary Tree	BST	AVL
Find	$O(n)$	$O(\lg(n))$	$O(n)$	$O(n)$	$O(n)$	$O(h)$	$O(\lg n)$
Insert	$O(1)$	find + $O(n)$	$O(1)$	find + $O(1)$	$O(1)$	find + $O(1)$	find + $O(1)$
Remove	find + $O(1)$	find + $O(n)$	find + $O(1)$	find + $O(1)$	find + $O(1)$	find + $O(1)$	find + $O(\lg n)$
Traverse	$O(n)$						

BTree

A **BTree**s of order **m** is an m-way tree:

- All keys within a node are ordered
- All leaves hold no more than **m-1** nodes.



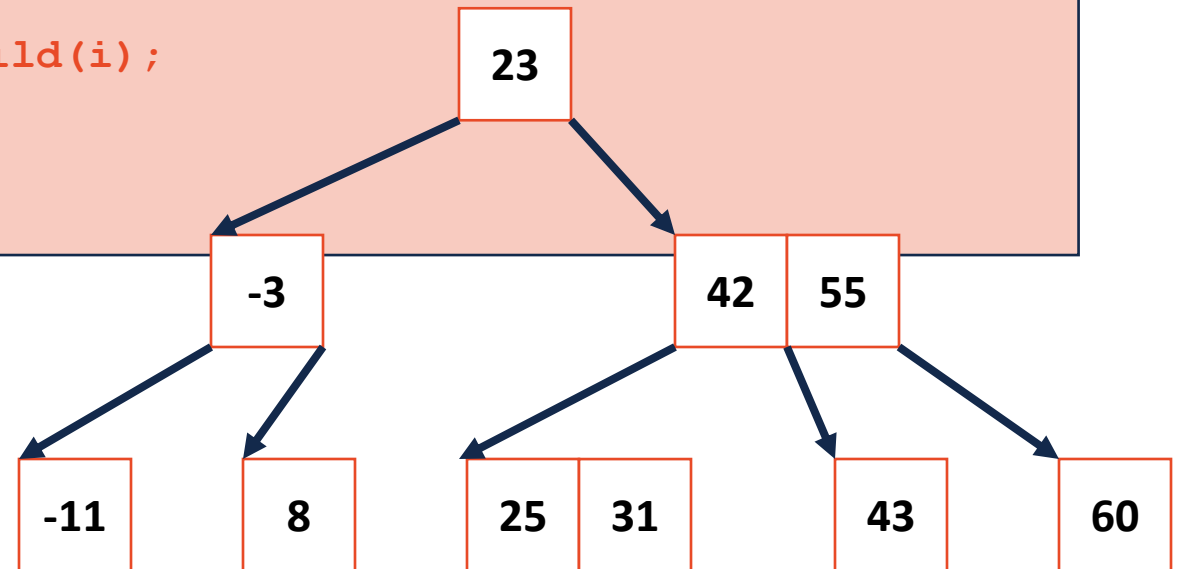
BTree Properties

A **BTree** of order **m** is an m-way tree:

- All keys within a node are ordered
- All leaves hold no more than **m-1** nodes.
- All internal nodes have exactly **one more child than key**.
- Root nodes can be a leaf or have **[2, m]** children.
 - [1, m-1] keys
- All non-root, internal nodes have **[ceil(m/2), m]** children.
 - [ceil(m/2)-1, m-1] keys
- All leaves are on the same level

BTree Search

```
1 bool Btree::_exists(BTreeNode & node, const K & key) {  
2  
3     unsigned i;  
4     for ( i = 0; i < node.keys_ct_ && key > node.keys_[i]; i++) { }  
5  
6     if ( i < node.keys_ct_ && key == node.keys_[i] ) {  
7         return true;  
8     }  
9  
10    if ( node.isLeaf() ) {  
11        return false;  
12    } else {  
13        BTreeNode nextChild = node._fetchChild(i);  
14        return _exists(nextChild, key);  
15    }  
16 }
```



BTree Analysis

The height of the BTree determines maximum number of fetches possible in data search.

Disk reads, demand

...and the height of the structure is: $\log_m n$.

$$O(\log_m n) = O(\lg n)$$

Therefore: The number of seeks is no more than $\log_m n$.

...suppose we want to prove this!

Hashing

Goals:

We want to define a **keyspace**, a (mathematical) description of the keys for a set of data.

...use a function to map the **keyspace** into a small set of integers.

$$h(\text{key}) : \text{key} \rightarrow \mathbb{N}$$

A Hash Table based Dictionary

Client Code:

```
1 Dictionary<KeyType, ValueType> d;  
2 d[k] = v;
```

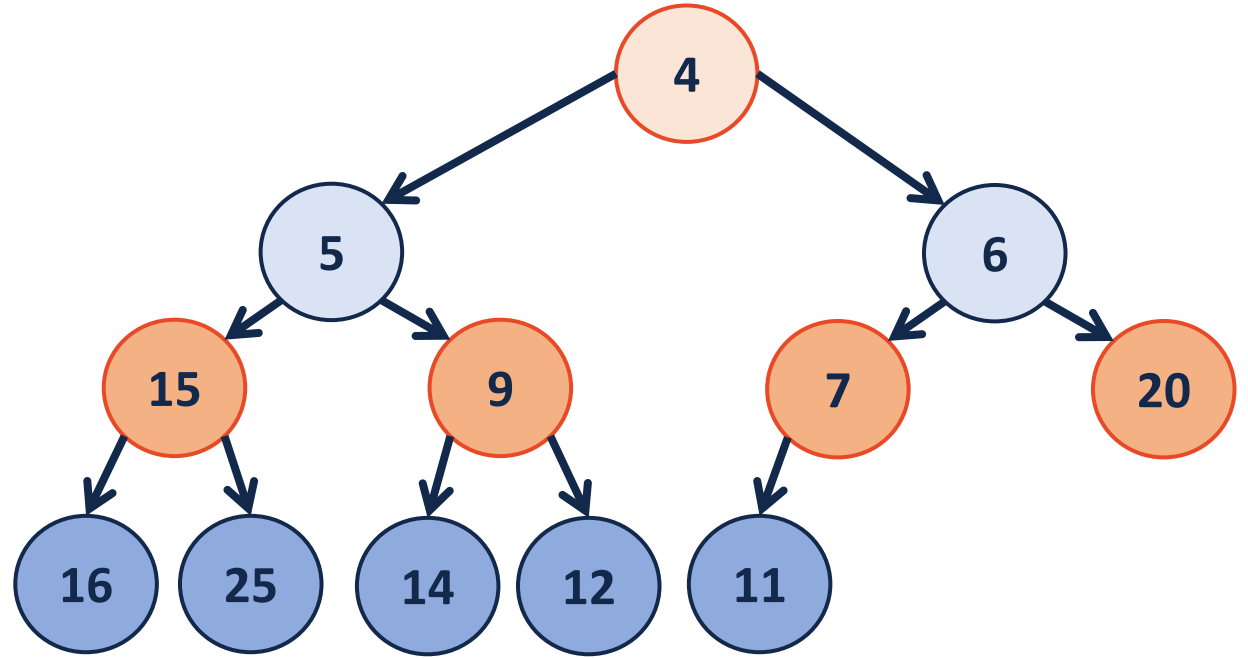
A **Hash Table** consists of three things:

1. hash function $h(k)$
2. array
3. collision resolution strategies

(min)Heap

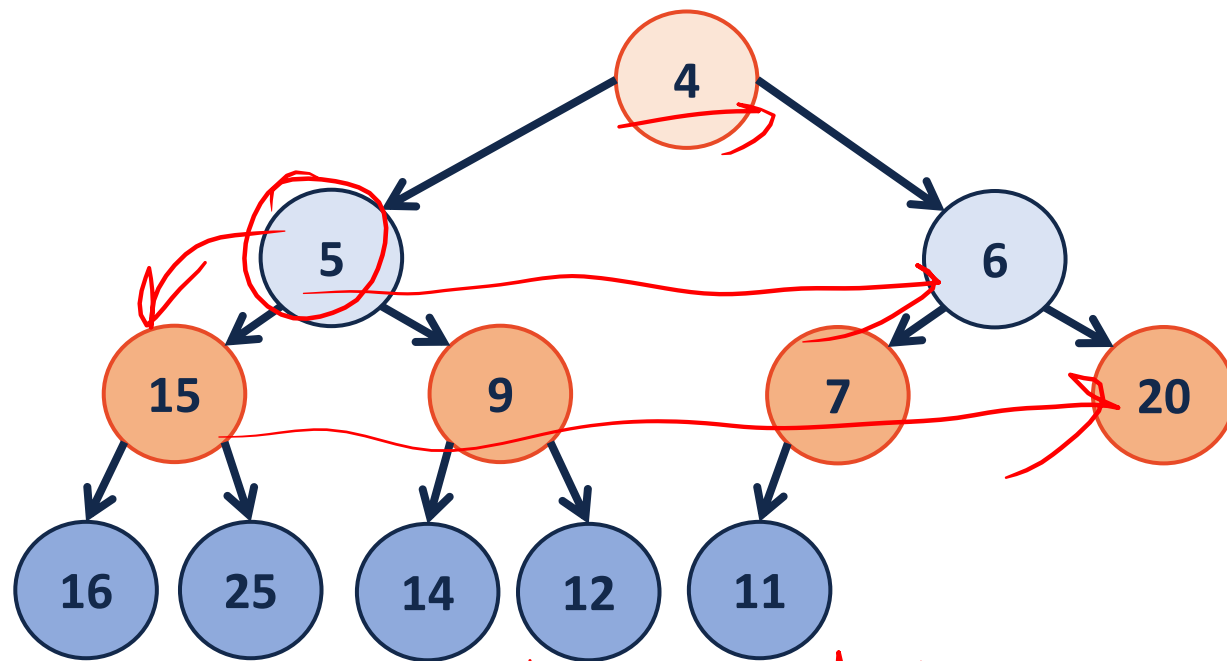
A complete binary tree T is a min-heap if:

- $T = \{\}$ or
- $T = \{r, T_L, T_R\}$, where r is less than the roots of $\{T_L, T_R\}$ and $\{T_L, T_R\}$ are min-heaps.



(min)Heap

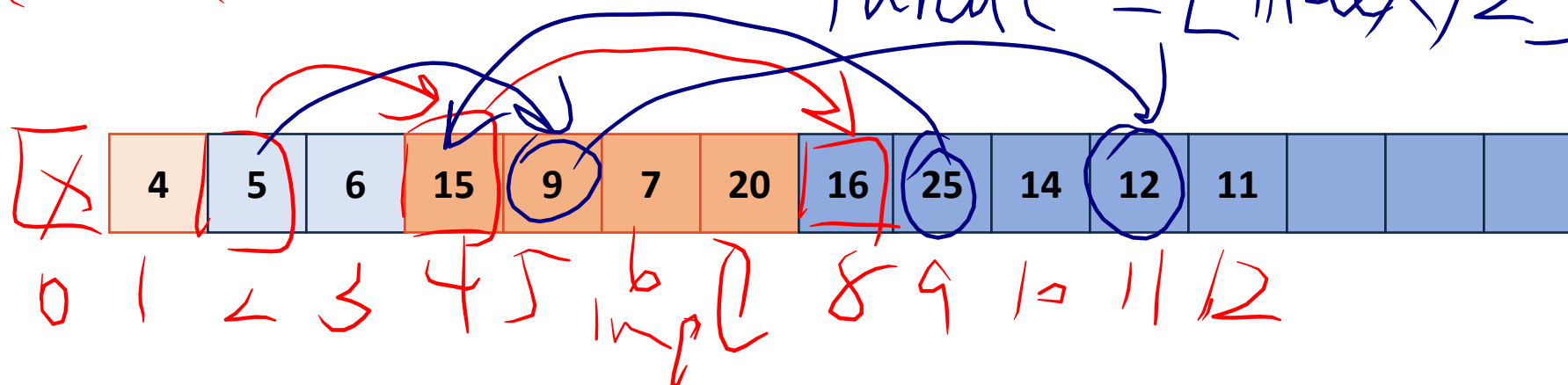
Level order TrAV
* In a heap impl
[0] is not used



Left child = $2 * \text{index}$
Right child = $2 * \text{index} + 1$

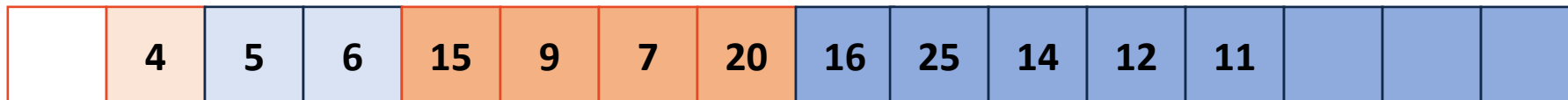
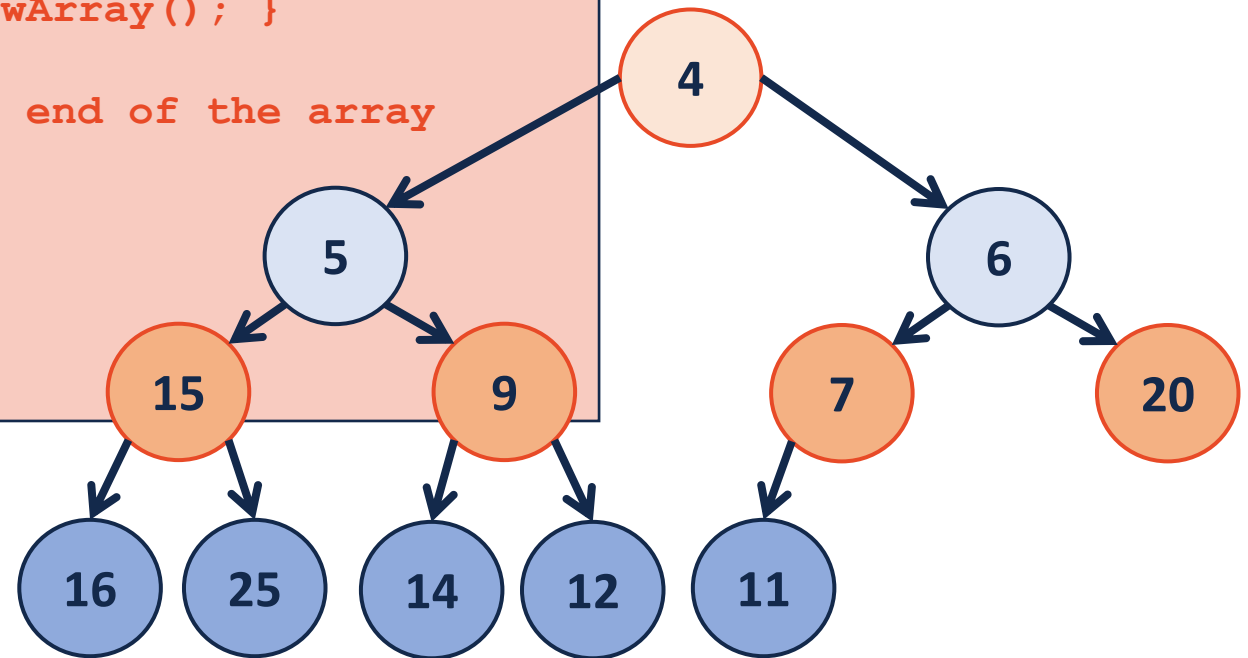
visual model

Parent = $\lfloor \text{index} / 2 \rfloor$



insert

```
1  template <class T>
2  void Heap<T>::_insert(const T & key) {
3      // Check to ensure there's space to insert an element
4      // ...if not, grow the array
5      if ( size_ == capacity_ ) { _growArray(); }
6
7      // Insert the new element at the end of the array
8      item_[++size_] = key;
9
10     // Restore the heap property
11     _heapifyUp(size_);
12 }
```



insert - heapifyUp

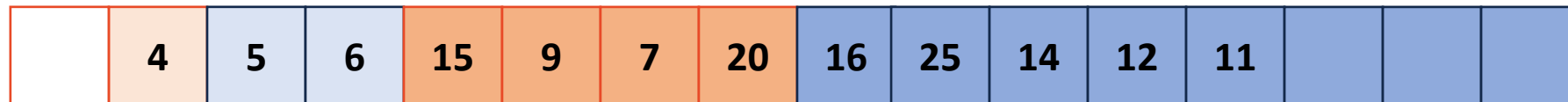
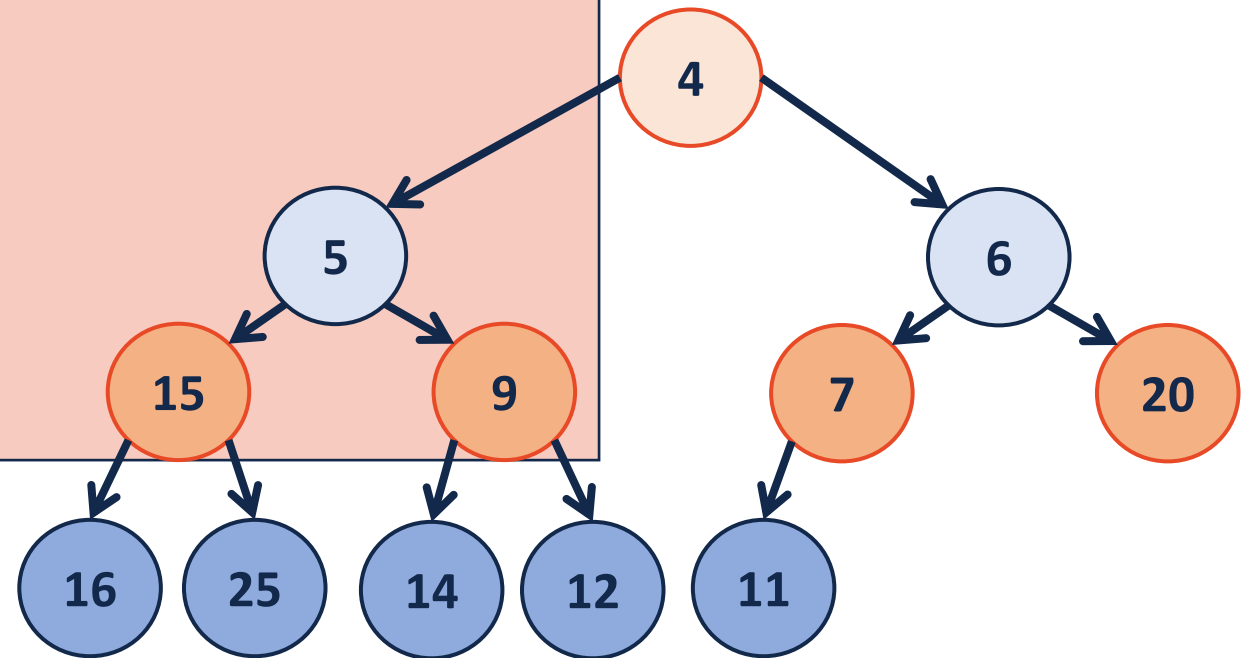
```
1 template <class T>
2 void Heap<T>::_insert(const T & key) {
3     // Check to ensure there's space to insert an element
4     // ...if not, grow the array
5     if ( size_ == capacity_ ) { _growArray(); }
6
7     // Insert the new element at the end of the array
8     item_[++size] = key;
9
10    // Restore the heap property
11    _heapifyUp(size);
12 }
```

```
1 template <class T>
2 void Heap<T>::_heapifyUp( unsigned index ) {
3     if ( index > 1 ) {
4         if ( item_[index] < item_[parent(index)] ) {
5             std::swap( item_[index], item_[parent(index)] );
6             _heapifyUp( parent(index) );
7         }
8     }
9 }
```

parent(index):
return index/2;

removeMin

```
1  template <class T>
2  void Heap<T>::_removeMin() {
3      // Swap with the last value
4      T minValue = item_[1];
5      item_[1] = item_[size_];
6      size--;
7
8      // Restore the heap property
9      heapifyDown(1);
10
11     // Return the minimum value
12     return minValue;
13 }
```



removeMin - heapifyDown

```
1  template <class T>
2  void Heap<T>::_removeMin() {
3      // Swap with the last value
4      T minValue = item_[1];
5      item_[1] = item_[size_];
6      size--;
7
8      // Restore the heap property
9      _heapifyDown(1);
10
11     // Return the minimum value
12     return minValue;
13 }
```

Base case check

```
1  template <class T>
2  void Heap<T>::_heapifyDown(int index) {
3      if ( !_isLeaf(index) ) {
4          int minChildIndex = _minChild(index);
5          if ( item_[index] > item_[minChildIndex] ) {
6              std::swap( item_[index], item_[minChildIndex] );
7              _heapifyDown( minChildIndex );
8          }
9      }
10 }
```

buildHeap

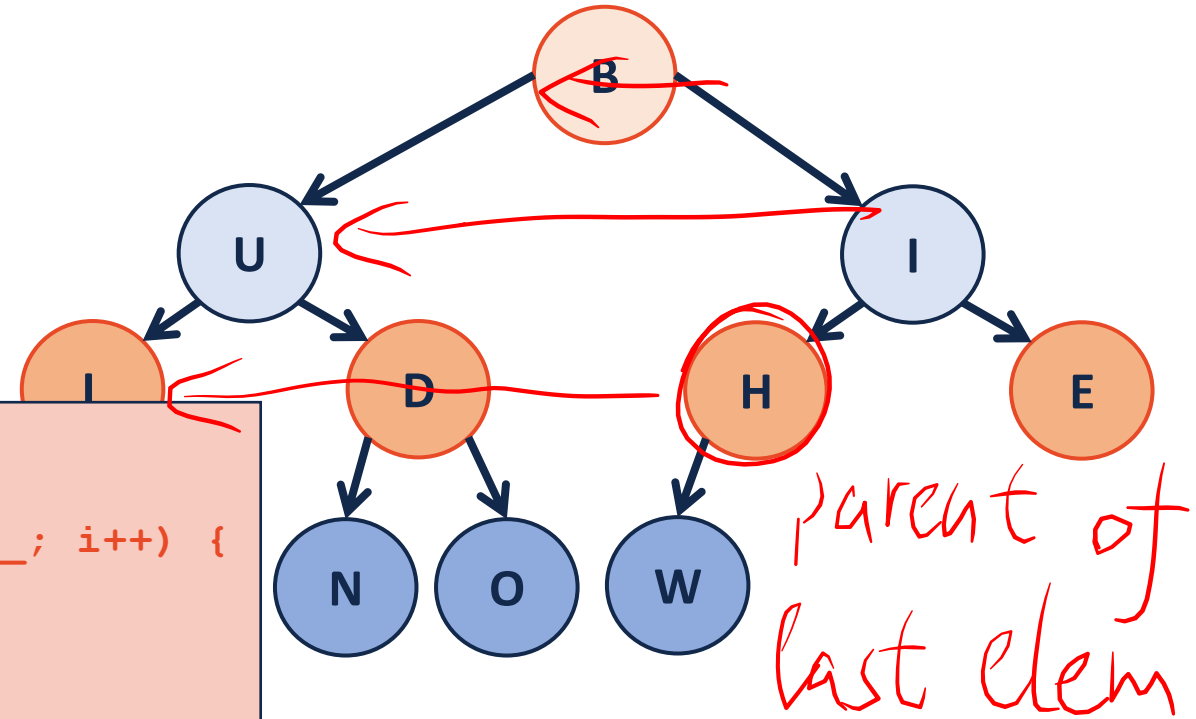
1. Sort the array – it's a heap!

2.

```
1 template <class T>
2 void Heap<T>::buildHeap() {
3     for (unsigned i = 0; i <= size_; i++) {
4         heapifyUp(i);
5     }
6 }
```

3.

```
1 template <class T>
2 void Heap<T>::buildHeap() {
3     for (unsigned i = parent(size); i > 0; i--) {
4         heapifyDown(i);
5     }
6 }
```



Proving buildHeap Running Time

Theorem: The running time of buildHeap on array of size n is:

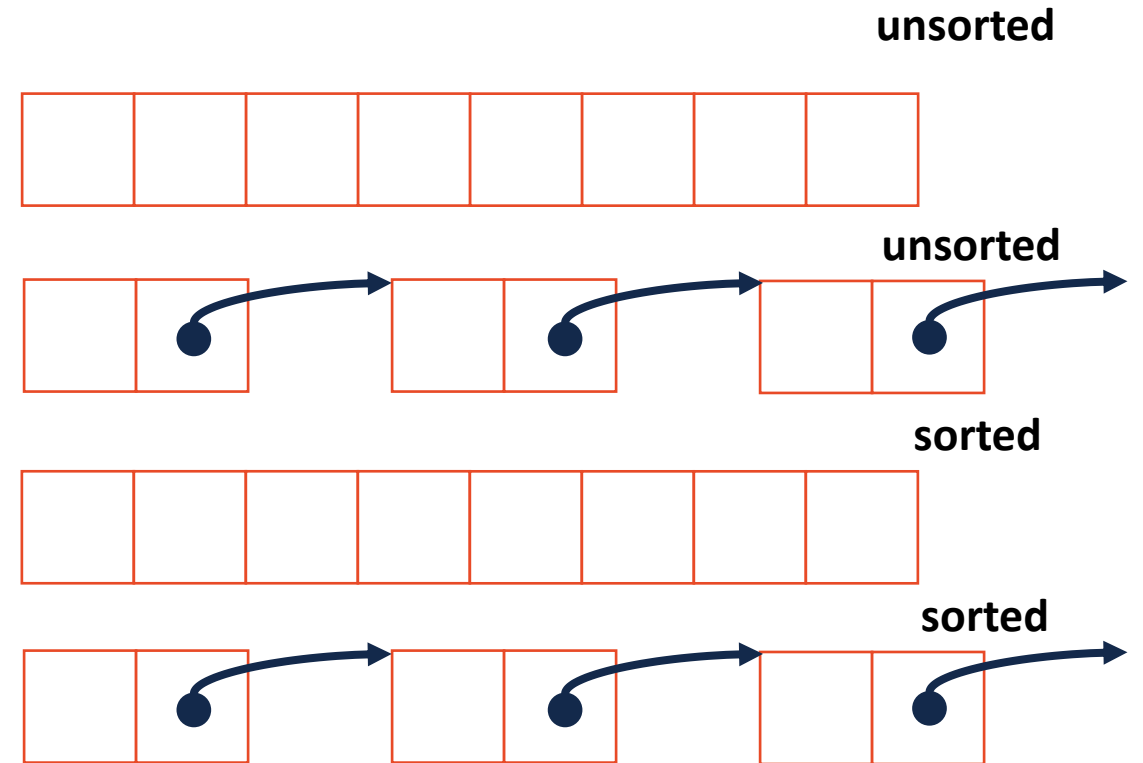
$O(n \log n)$

Strategy:

- We know that constant work is done based on the distance a node is away from the root (eg: it's height).
- Therefore, the running time is proportional to the sum of the heights of the nodes involved in heapifyDown operation.
- We will work towards creating a proof around the sum of the heights of all such nodes.

Priority Queue Implementation

insert	removeMin	build
$O(1)^A$	$O(n)$	$O(n)$
$O(1)$	$O(n)$	$O(n)$
$O(n)$	$O(1)$	$O(\lg n)$
$O(n)$	$O(1)$	$O(\lg n)$
$O(\lg n)$	$O(\lg n)$	$O(\lg n)$
$O(\lg n)$	$O(\lg n)$	$O(n)$



AVL Tree

Heap

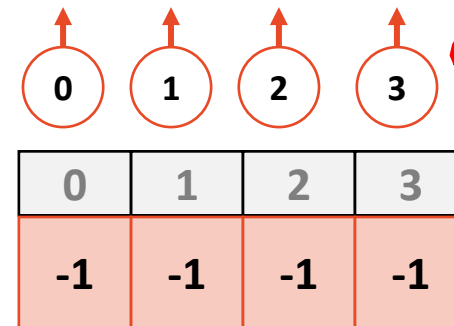
Disjoint Sets ADT

- Maintain a collection $S = \{s_0, s_1, \dots, s_k\}$
- Each set has a representative member.
- API:

```
void makeSet(const T & t);  
void union(const T & k1, const T & k2);  
T & find(const T & k);
```

Implementation #2

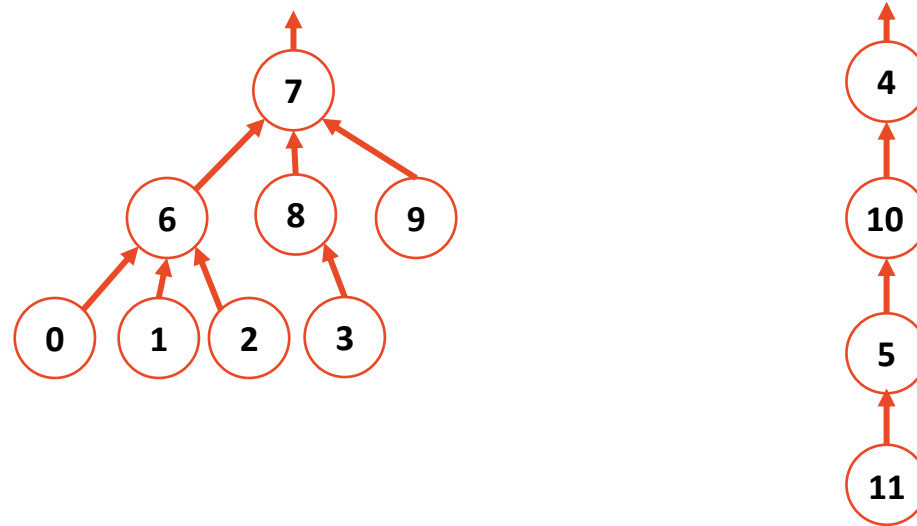
- We will continue to use an array where the index is the key
- The value of the array is:
 - **-1**, if we have found the representative element
 - **The index of the parent**, if we haven't found the rep. element
- We will call theses **UpTrees**:



Array
impl

visual
model

Disjoint Sets – Smart Union



Union by height

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8		10	7		7	7	4	5

Idea: Keep the height of the tree as small as possible.

Union by size

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8		10	7		7	7	4	5

Idea: Minimize the number of nodes that increase in height

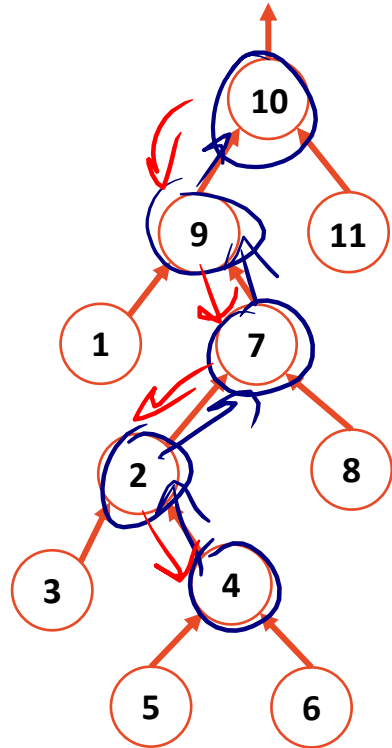
Both guarantee the height of the tree is: _____.

Disjoint Sets Find

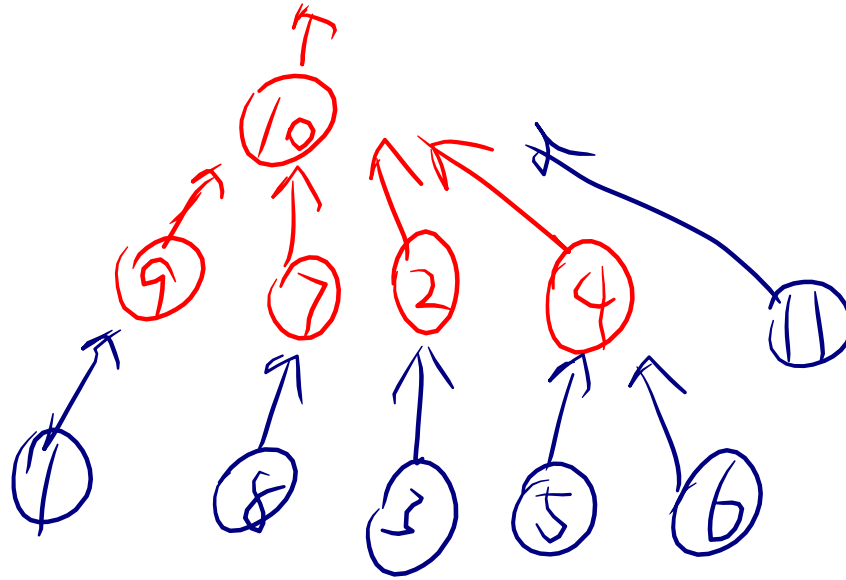
```
1 int DisjointSets::find(int i) {  
2     if ( s[i] < 0 ) { return i; }  
3     else { return find( s[i] ); }  
4 }
```

```
1 void DisjointSets::unionBySize(int root1, int root2) {  
2     int newSize = arr_[root1] + arr_[root2];  
3  
4     // If arr_[root1] is less than (more negative), it is the larger set;  
5     // we union the smaller set, root2, with root1.  
6     if ( arr_[root1] < arr_[root2] ) {  
7         arr_[root2] = root1;  
8         arr_[root1] = newSize;  
9     }  
10  
11     // Otherwise, do the opposite:  
12     else {  
13         arr_[root1] = root2;  
14         arr_[root2] = newSize;  
15     }  
16 }
```

Path Compression



$\text{find}(4) = 10$



ideal $O(1)$

Disjoint Sets Find

```
1 int DisjointSets::find(int i) {  
2     if ( arr_[i] < 0 ) { return i; }  
3     else { return      (arr[i] = find( arr_[i] )); }  
4 }
```

```
1 void DisjointSets::unionBySize(int root1, int root2) {  
2     int newSize = arr_[root1] + arr_[root2];  
3  
4     // If arr_[root1] is less than (more negative), it is the larger set;  
5     // we union the smaller set, root2, with root1.  
6     if ( arr_[root1] < arr_[root2] ) {  
7         arr_[root2] = root1;  
8         arr_[root1] = newSize;  
9     }  
10  
11     // Otherwise, do the opposite:  
12     else {  
13         arr_[root1] = root2;  
14         arr_[root2] = newSize;  
15     }  
16 }
```

Same !

Disjoint Sets Analysis

In an Disjoint Sets implemented with smart **unions** and path compression on **find**:

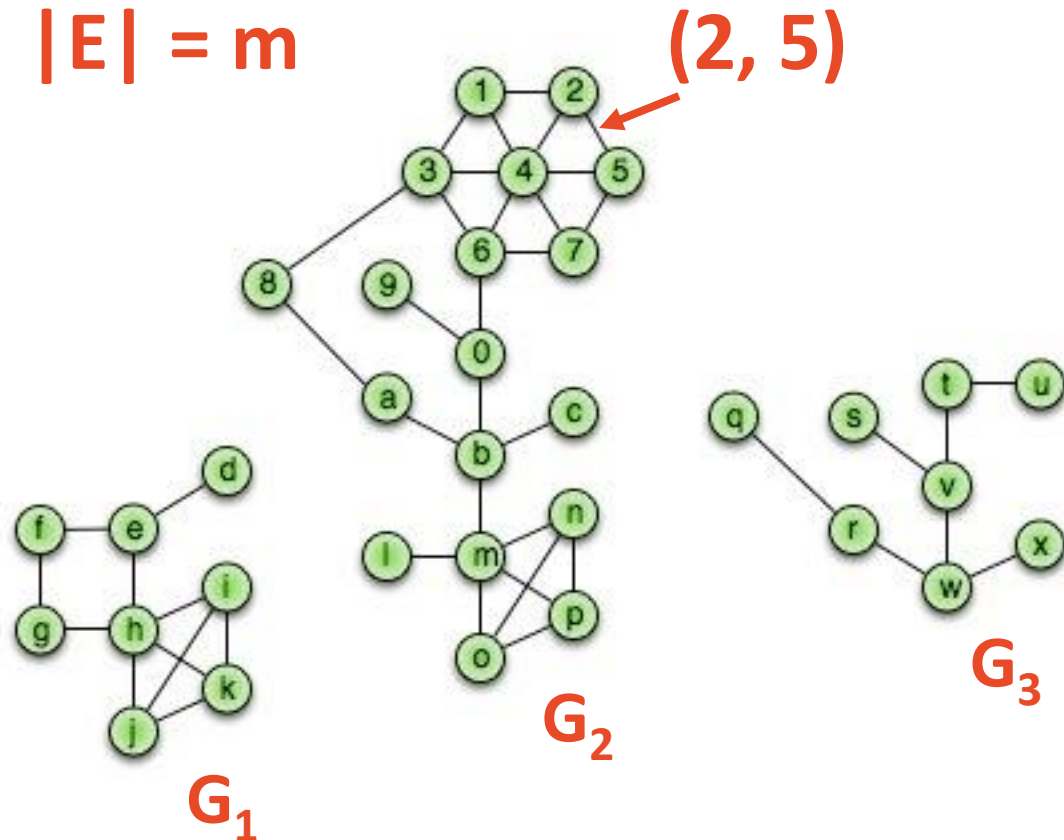
Any sequence of **m union** and **find** operations result in the worse case running time of $O(\underline{m \cdot \lg^*(n)})$,
where **n** is the number of items in the Disjoint Sets.

Graph Vocabulary

$$G = (V, E)$$

$$|V| = n$$

$$|E| = m$$



Incident Edges:

$$I(v) = \{ (x, v) \text{ in } E \}$$

Degree(v): $|I|$

Adjacent Vertices:

$$A(v) = \{ x : (x, v) \text{ in } E \}$$

Path(G_2): Sequence of vertices connected by edges

Cycle(G_1): Path with a common begin and end vertex.

Simple Graph(G): A graph with no self loops or multi-edges.

Running times are often reported by **n**, the number of vertices, but often depend on **m**, the number of edges.

How many edges? **Minimum edges:**

Not Connected:

① ② ③

Connected*:

$n-1$

Maximum edges:

Simple:

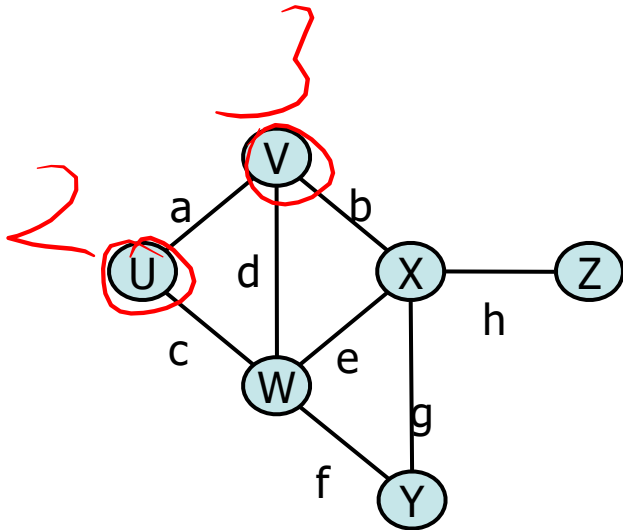
$\frac{n(n-1)}{2}$

Not simple:

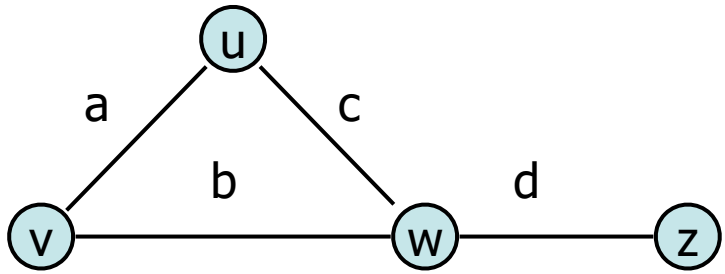
$+\infty \rightarrow$ no upper

$\sum_{v \in V} \deg(v) =$

$2m$, simple bound on non-simple



Edge List



Key Ideas:

- Given a vertex, $O(1)$ lookup in vertex list
 - Implement w/ a hash table, etc.
- All basic ADT operations runs in $O(m)$ time

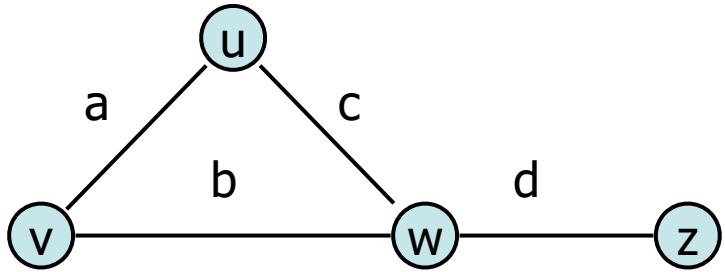
Vertex List

u
v
w
z

Edge List

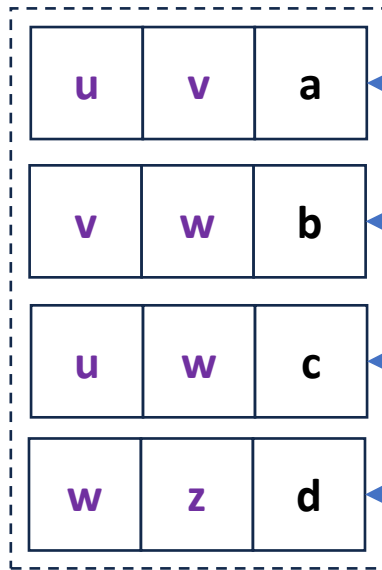
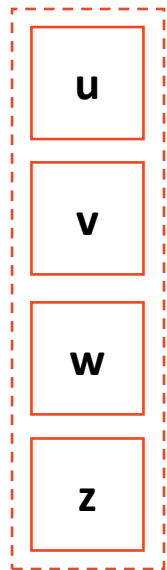
u	v	a
v	w	b
u	w	c
w	z	d

Adjacency Matrix



Key Ideas:

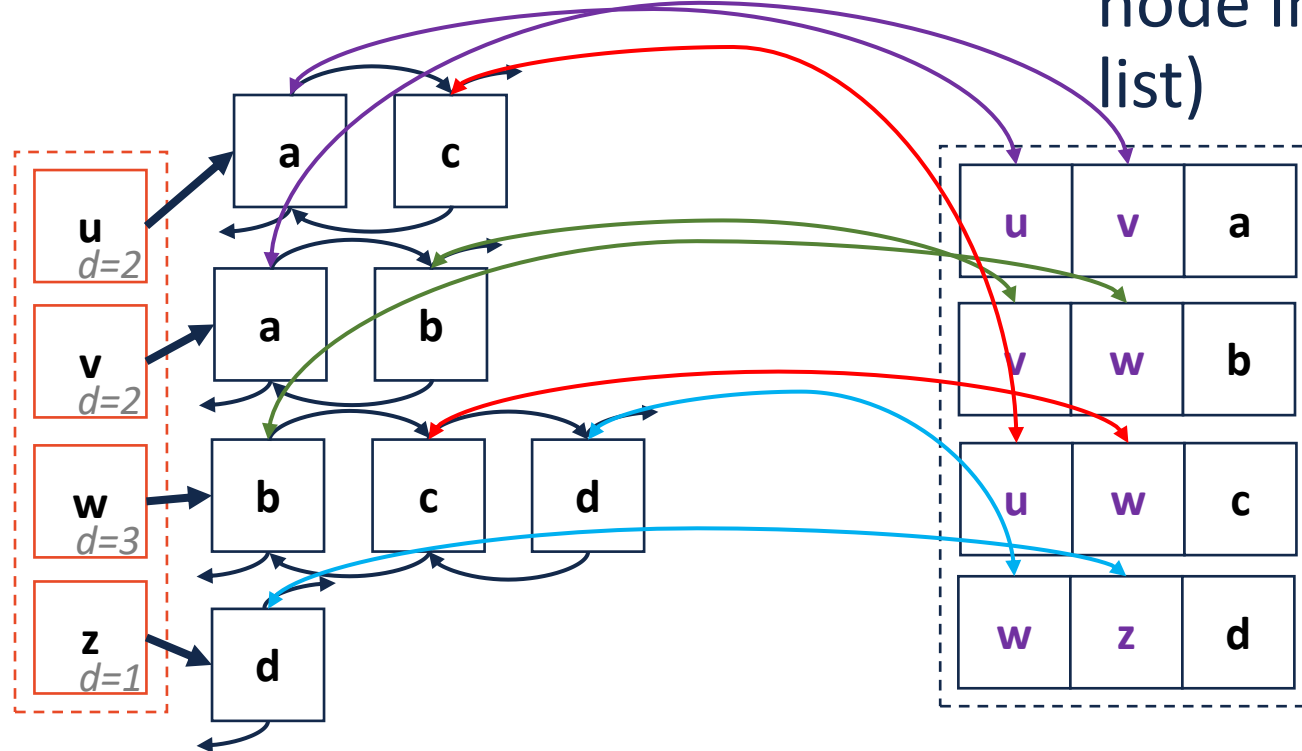
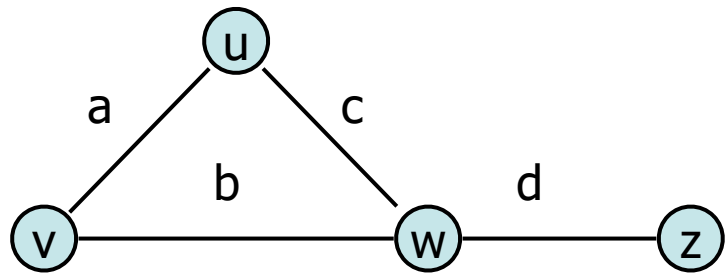
- Given a vertex, $O(1)$ lookup in vertex list
- Given a pair of vertices (an edge), $O(1)$ lookup in the matrix
- Undirected graphs can use an upper triangular matrix



	u	v	w	z
u		∅	●	∅
v			∅	∅
w				∅
z				

n^2 space

Adjacency List



Key Ideas:

- $O(1)$ lookup in vertex list
- Vertex list contains a doubly-linked adjacency list
 - $O(1)$ access to the adjacent vertex's node in adjacency list (via the edge list)
- Vertex list maintains a count of incident edges, or **deg(v)**
- Many operations run in $O(\deg(v))$, and $\deg(v) \leq n-1$, $O(n)$.

Expressed as big-O	Edge List	Adjacency Matrix	Adjacency List
Space	$n+m$	n^2	$n+m$
insertVertex(v)	1	n	1
removeVertex(v)	m	n	deg(v)
insertEdge(v, w, k)	1	1	1
removeEdge(v, w)	1	1	1
incidentEdges(v)	m	n	deg(v)
areAdjacent(v, w)	m	1	min(deg(v), deg(w))

```
1 BFS(G) :
2   Input: Graph, G
3   Output: A labeling of the edges on
4           G as discovery and cross edges
5
6   foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8   foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10  foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12          BFS(G, v)
```

```
14 BFS(G, v) :
15     Queue q
16     setLabel(v, VISITED)
17     q.enqueue(v)
18
19     while !q.empty():
20         v = q.dequeue()
21         foreach (Vertex w : G.adjacent(v)):
22             if getLabel(w) == UNEXPLORED:
23                 setLabel(v, w, DISCOVERY)
24                 setLabel(w, VISITED)
25                 q.enqueue(w)
26             elseif getLabel(v, w) == UNEXPLORED:
27                 setLabel(v, w, CROSS)
```

```
14 DFS(G, v):
15 Queue q
16   setLabel(v, VISITED)
17 q.enqueue(v)
18
19 while !q.empty():
20 v = q.dequeue()
21   foreach (Vertex w : G.adjacent(v)):
22     if getLabel(w) == UNEXPLORED:
23       setLabel(v, w, DISCOVERY)
24       setLabel(w, VISITED)
25       DFS(G, w)
26   elseif getLabel(v, w) == UNEXPLORED:
27     setLabel(v, w, BACK)
```

```
14 DFS(G, v):
15   Stack s
16   setLabel(v, VISITED)
17   s.push(v)
18
19   while !s.empty():
20     v = s.pop()
21     foreach (Vertex w : G.adjacent(v)):
22       if getLabel(w) == UNEXPLORED:
23         setLabel(v, w, DISCOVERY)
24         setLabel(w, VISITED)
25         s.push(w)
26       elseif getLabel(v, w) == UNEXPLORED:
27         setLabel(v, w, BACK)
```

Kruskal's Algorithm

Setup + Iter

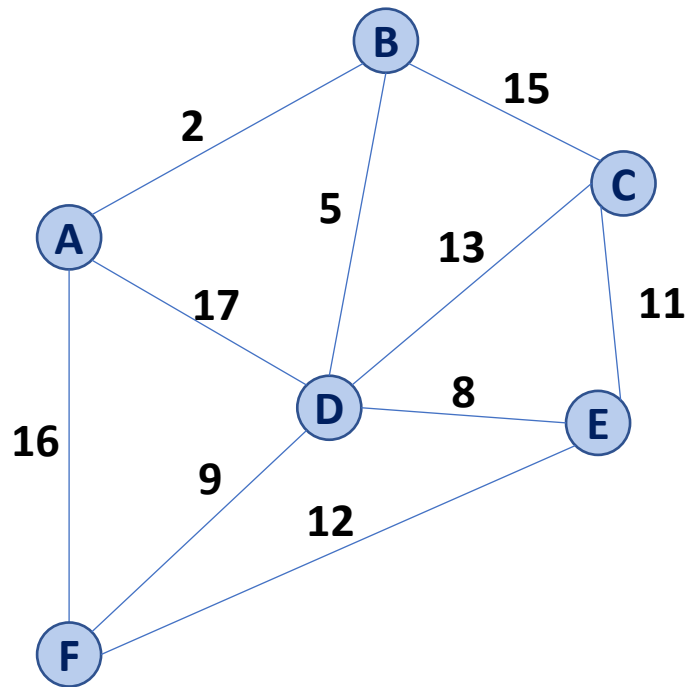
Priority Queue:	Total Running Time
Heap	$O(m \log(m))$
Sorted Array	$O(m \log(m))$

```
1 KruskalMST(G):
2   DisjointSets forest
3   foreach (Vertex v : G):
4       forest.makeSet(v)
5
6   PriorityQueue Q    // min edge weight
7   foreach (Edge e : G):
8       Q.insert(e)
9
10  Graph T = (V, {})
11
12  while |T.edges()| < n-1:
13      Edge (u, v) = Q.removeMin()
14      if forest.find(u) != forest.find(v):
15          T.addEdge(u, v)
16          forest.union( forest.find(u),
17                        forest.find(v) )
18
19  return T
```

Handwritten annotations in blue ink:

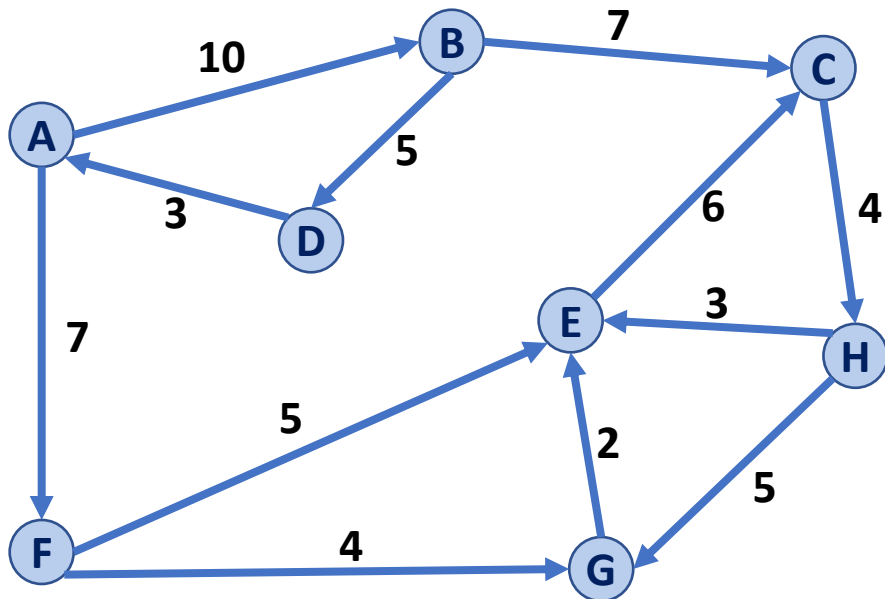
- A large closing bracket $)$ on the right side of lines 2-4, with $O(n)$ written next to it.
- A large closing bracket $)$ on the right side of lines 6-8, with $O(m)$ written next to it.
- A bracket $] O(n)$ on the right side of line 10.
- A bracket $] O(m)$ on the right side of lines 12-17.
- Arrows pointing from the $O(n)$ and $O(m)$ annotations to the $n-1$ in the while loop condition and the $removeMin()$ call, respectively.

Prim's Algorithm



```
1 PrimMST(G, s):
2   Input: G, Graph;
3         s, vertex in G, starting vertex
4   Output: T, a minimum spanning tree (MST) of G
5
6   foreach (Vertex v : G):
7     d[v] = +inf
8     p[v] = NULL
9   d[s] = 0
10
11   PriorityQueue Q // min distance, defined by d[v]
12   Q.buildHeap(G.vertices())
13   Graph T // "labeled set"
14
15   repeat n times:
16     Vertex m = Q.removeMin()
17     T.add(m)
18     foreach (Vertex v : neighbors of m not in T):
19       if cost(v, m) < d[v]:
20         d[v] = cost(v, m)
21         p[v] = m
22
23   return T
```

Dijkstra's Algorithm (SSSP)



```
DijkstraSSSP(G, s):
```

```
6   foreach (Vertex v : G):
7       d[v] = +inf
8       p[v] = NULL
9   d[s] = 0
10
11  PriorityQueue Q // min distance, defined by d[v]
12  Q.buildHeap(G.vertices())
13  Graph T          // "labeled set"
14
15  repeat n times:
16      Vertex u = Q.removeMin()
17      T.add(u)
18      foreach (Vertex v : neighbors of u not in T):
19          if cost(u, v) + d[u] < d[v]:
20              d[v] = cost(u, v) + d[u]
21              p[v] = u
```

Floyd-Warshall Algorithm

Floyd-Warshall's Algorithm is an alternative to Dijkstra in the presence of **negative-weight edges** (not **negative weight cycles**).

```
FloydWarshall(G):  
6   Let d be a adj. matrix initialized to +inf  
7   foreach (Vertex v : G):  
8       d[v, v] = 0  
9   foreach (Edge (u, v) : G):  
10      d[u, v] = cost(u, v)  
11  
12  foreach (Vertex w : G):  
13      foreach (Vertex u : G):  
14          foreach (Vertex v : G):  
15              if d[u, v] > d[u, w] + d[w, v]:  
16                  d[u, v] = d[u, w] + d[w, v]
```



May 28, 9:00-12:00
Good luck!